

Міністерство освіти і науки України
Український державний університет науки і технологій

Навчально-науковий інститут
«Український державний хіміко-технологічний університет»

(назва навчально-наукового інституту)

Факультет комп'ютерних наук та інженерії

(повна назва факультету)

Кафедра інформаційних систем

(повна назва кафедри)

Пояснювальна записка

до дипломної роботи

бакалавра

(освітній рівень)

на тему Розробка та програмна реалізація алгоритму оптимізації
розподілу вантажів у транспортній мережі

Виконав: студент 4 курсу, групи КН-22
спеціальності

122 Комп'ютерні науки

(код і назва спеціальності)

Ніколаєв Т.О.

(прізвище та ініціали)

(підпис)

Керівник

Зеленцов. Д.Г.

(прізвище та ініціали)

(підпис)

Рецензент

Коротка Л.І.

(прізвище та ініціали)

(підпис)

Дніпро – 20 26 року

Український державний університет науки і технологій
(повне найменування вищого навчального закладу)

Навчально-науковий інститут
«Український державний хіміко-технологічний університет»
(назва навчально-наукового інституту)

Факультет, відділення	комп'ютерних наук та інженерії, денне
Кафедра	інформаційних систем
Освітній рівень	бакалавр
Спеціальність	122 Комп'ютерні науки (код і назва)
Освітня програма	Комп'ютерні науки. Штучний інтелект (назва)

ЗАТВЕРДЖУЮ

Завідувач кафедри _____
канд. техн. наук, доц. Ляшенко О.А.
« ____ » _____ 20 ____ року

З А В Д А Н Н Я
НА ДИПЛОМНУ РОБОТУ СТУДЕНТУ
Ніколаєву Тимуру Олександровичу
(прізвище, ім'я, по-батькові)

- Тема роботи Розробка та програмна реалізація алгоритму оптимізації розподілу вантажів у транспортній мережі
- керівник роботи Зеленцов Дмитро Гегемонович, д-р. техн. наук, професор,
(прізвище, ім'я, по-батькові, науковий ступінь, вчене звання)
затверджені наказом вищого навчального закладу від 02.03.2026 р. № 77-к
- Строк подання студентом роботи 05.06.2026 р.
- Вихідні дані до роботи Картографічні дані, мультимодальна транспортна мережа, обмеження вантажомісткості транспортних засобів
- Зміст розрахунково-пояснювальної записки (перелік питань, які потрібно розробити) _____
Загальний розділ.
Спеціальний розділ.
Розрахунковий розділ.
- Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень)
Презентація до дипломної роботи.

6. Консультанти розділів роботи

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видав	завдання прийняв
1 Загальний розділ	Зеленцов Д. Г., професор		
2 Спеціальний розділ	Зеленцов Д. Г., професор		
3 Розрахунковий розділ	Зеленцов Д. Г., професор		
Нормоконтроль	Зеленцов Д. Г., професор		

7. Дата видачі завдання 11.05.2026 р.

КАЛЕНДАРНИЙ ПЛАН

№ з/п	Назва етапів дипломної роботи	Строк виконання етапів роботи	Примітка
1	Розробка програмного продукту	11.05.2026-21.05.2026	виконано
2	Оформлення пояснювальної записки	11.05.2026-21.05.2026	виконано
3	Попередній захист дипломної роботи	25.05.2026	виконано
4	Подання дипломної роботи на перевірку та підпис керівнику, проходження нормоконтролю	21.05.2026-26.05.2026	виконано
5	Перевірка дипломної роботи на академічний плагіат	26.05.2026-01.06.2026	виконано
6	Підготовка презентації та доповіді	26.05.2026-01.06.2026	виконано
7	Рецензування дипломної роботи	02.06.2026-04.06.2026	виконано
8	Подання дипломної роботи на підпис завідувачу кафедри	04.06.2026	виконано
9	Захист дипломної роботи	10.06.2026	

Студент

(підпис)

Ніколаєв Т.О.

(прізвище та ініціали)

Керівник роботи

(підпис)

Зеленцов Д.Г.

(прізвище та ініціали)

РЕФЕРАТ

Пояснювальна записка до дипломної роботи: 74 с., 3 табл., 17 рис., 2 дод., 12 джерел.

ТРАНСПОРТНА МЕРЕЖА, МУЛЬТИМОДАЛЬНИЙ ГРАФ,
МАРШРУТИЗАЦІЯ ВАНТАЖІВ, ГЕНЕТИЧНІ АЛГОРИТМИ.

Об'єкт роботи – проблема розподілу та доставки вантажів у мультимодальній транспортній мережі.

Предмет роботи – математичні моделі та еволюційні алгоритми в задачах маршрутизації транспорту.

Мета роботи – розробка та програмна реалізація алгоритму оптимізації маршрутів доставки для мінімізації сумарного часу переїздів та кількості задіяних транспортних засобів.

Методи дослідження: теорія графів, математичне моделювання, еволюційні методи.

Засоби розробки – середовище розробки Visual Studio Code та мова програмування Python.

Структура роботи включає три розділи. Загальний розділ звіту містить обґрунтування актуальності розробки, аналіз сучасних підходів до маршрутизації, опис структури транспортної мережі, аналіз аналогів програмних продуктів, аналіз літератури та постановку задачі.

У спеціальному розділі виконано математичну постановку задачі: мережу формалізовано у вигляді графу з атрибутами вузлів та ребер, враховано обмеження за вантажною місткістю; описано генетичний алгоритм.

В розрахунковому розділі представлено інформацію про програмне та технічне забезпечення, опис програми, інструкція користувача та програміста, результати тестування системи.

ЗМІСТ

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ	7
ВСТУП.....	8
1 ЗАГАЛЬНИЙ РОЗДІЛ	10
1.1 Обґрунтування актуальності дослідження.....	10
1.2 Опис предметної області та аналіз об’єкта дослідження	10
1.3 Огляд і аналіз літератури за темою роботи	12
1.4 Математична постановка задачі	13
1.5 Висновки за загальним розділом	14
2 СПЕЦІАЛЬНИЙ РОЗДІЛ.....	15
2.1 Проєктування розроблюваної інформаційної системи.....	15
2.1.1 Функціональна модель системи (діаграма прецедентів)	17
2.1.2 Структурна модель – діаграма класів предметної області.....	19
2.1.3 Структурна модель – модульна архітектура.....	21
2.1.4 Поведінкова модель – геокодування адреси	25
2.1.5 Поведінкова модель – оптимізація одного транспортного засобу.....	28
2.1.6 Поведінкова модель – оптимізація багатьох ТЗ	31
2.1.7 Динамічна модель – діаграма діяльності.....	35
2.2 Інформаційне забезпечення	39
2.2.1 Концептуальний рівень – сутності предметної області.....	39
2.2.2 Логічний рівень – реляційна схема та потоки даних	43
2.2.3 Фізичний рівень – реалізація у СКБД SQLite	46
2.3 Алгоритмічне забезпечення	48

	6
2.4 Висновки за спеціальним розділом	50
3 РОЗРАХУНКОВИЙ РОЗДІЛ	51
3.1 Програмне забезпечення	51
3.2 Технічне забезпечення	52
3.3 Опис програми.....	53
3.4 Інструкція користувача.....	56
3.5 Інструкція програміста.....	59
3.6 Тестування розробленого програмного забезпечення	60
3.7 Результати чисельних експериментів	65
3.8 Висновки за розрахунковим розділом	68
ВИСНОВКИ	70
ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАНЬ.....	74
ДОДАТОК А	75
ДОДАТОК Б.....	230

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

ТЗ – транспортний засіб

ПЗ – програмне забезпечення

UML – Unified Modeling Language

VRP – Vehicle Routing Problem

VRPTW – Vehicle Routing Problem with Time Windows

CVRPTW – Capacitated Vehicle Routing Problem with Time Windows

OSM – Open Street Map

GA – Genetic Algorithm

DRY – Don't Repeat Yourself

ООП – Об'єктно-орієнтоване програмування

SRP – Single Responsibility Principle

MTZ – Miller-Tucker-Zemlin

TSP – Traveling salesman problem

ВСТУП

Сучасні реалії глобальної та локальної економіки характеризуються стрімким зростанням обсягів вантажних перевезень, що вимагає впровадження високоефективних систем управління логістикою.

У щільній міській забудові «остання миля» з'їдає основну частину логістичного бюджету: автомобільний доступ обмежений пішохідними зонами та одностороннім рухом, а хаби приймають вантаж лише у вузькі часові вікна. Тому при плануванні маршрутів доводиться комбінувати автомобілі, велосипеди та пішохідних кур'єрів – одного транспортного режиму на все місто не вистачає.

Традиційні конструктивні методи планування маршрутів часто не забезпечують знаходження глобального оптимуму в динамічному логістичному середовищі через схильність до застрягання в локальних екстремумах. Це зумовлює актуальність розробки та програмної реалізації спеціалізованих систем на основі генетичних алгоритмів – популяційних метаевристик, які дозволяють ефективно досліджувати простір розв'язків NP-складної задачі маршрутизації та знаходити якісні результати для автоматизації процесів розподілу вантажів.

Об'єкт роботи – проблема розподілу та доставки вантажів у мультимодальній транспортній мережі.

Предмет роботи – математичні моделі та еволюційні алгоритми в задачах маршрутизації транспорту.

Мета роботи – розробка та програмна реалізація алгоритму оптимізації маршрутів доставки для мінімізації сумарного часу переїздів та кількості задіяних транспортних засобів.

Методи дослідження. Для досягнення поставленої мети використано методи теорії графів для формалізації транспортної мережі, методи математичного моделювання для побудови моделі задачі змішаного цілочисельного програмування з урахуванням обмежень вантажної місткості та часових вікон та генетичні алгоритми для знаходження наближених до

оптимальних розв'язків шляхом еволюційного моделювання процесів селекції, схрещування та мутації.

Поставлено такі завдання:

- вивчити предметну область, структуру та принципи функціонування об'єкта дослідження;
- провести аналіз сучасних підходів та аналогів програмних продуктів для вирішення задач маршрутизації;
- формалізувати транспортну мережу як мультимодальний зважений орієнтований граф;
- розробити математичну модель задачі оптимізації з урахуванням обмежень місткості;
- реалізувати програмний модуль на основі розробленої математичної моделі оптимізації та провести його тестування на реальних даних.

Практичне значення одержаних результатів. Розроблене програмне забезпечення та алгоритмічні рішення можуть бути використані транспортно-логістичними компаніями для автоматизованої побудови маршрутів, що дозволить скоротити експлуатаційні витрати та підвищити якість обслуговування клієнтів.

Дана робота брала участь у I турі Всеукраїнського конкурсу студентських наукових робіт з галузей знань і спеціальностей у 2025-2026 н.р та посіла 2 призове місце (Додаток Б).

1 ЗАГАЛЬНИЙ РОЗДІЛ

1.1 Обґрунтування актуальності дослідження

В умовах глобальної цифровізації економіки та стрімкого зростання ринку електронної комерції, оптимізація логістичних процесів стає значним фактором для конкурентоспроможності підприємств. Задача маршрутизації транспортних засобів (Vehicle Routing Problem, VRP) належить до класу NP-складних задач комбінаторної оптимізації [1], що означає неможливість отримання точного розв'язку за прийнятний час для реальних масштабів міської інфраструктури.

Актуальність дослідження зумовлена наступними факторами:

- Економічна ефективність: витрати на логістику можуть складати до 17% від загальної вартості доставки [2]. Навіть мінімальне скорочення маршруту призводить до значної економії палива та людських ресурсів.
- Динамічність середовища: сучасні системи повинні враховувати часові вікна (VRPTW), затори та обмеження вантажності в режимі реального часу.
- Технологічне покращення: класичні евристики (наприклад, алгоритм Кларка-Райта) часто «застрягають» у локальних оптимумах [3]. Використання методів штучного інтелекту, зокрема генетичних алгоритмів, дозволяє ефективно досліджувати простір розв'язків та знаходити результати, близькі до глобального оптимуму, що є пріоритетним завданням для розробки сучасного логістичного ПЗ.

1.2 Опис предметної області та аналіз об'єкта дослідження

Предметна область роботи – планування міських маршрутів доставки для парку з кількох транспортних засобів різних типів. Завдання має три тісно пов'язані рівні. Перший, розподіл: які саме замовлення дістануться машині, які –

велосипеду, а які кур'єру пішки, з огляду на вантажомісткість і фізичну досяжність адреси. Другий, впорядкування: у якій послідовності кожен кур'єр об'їжджатиме свої зупинки. Третій, часові обмеження: жоден маршрут не повинен порушити вікно роботи хабу або клієнта. Якість розв'язку оцінюється сумарним часом руху і кількістю задіяних машин – це ті дві величини, що безпосередньо переходять у витрати на паливо й оплачуваний час персоналу.

Об'єктом дослідження у роботі виступає проблема розподілу та доставки вантажів у мультимодальній транспортній мережі.

Предметом дослідження є математичні моделі та генетичні алгоритми маршрутизації транспортних засобів.

Структурно об'єкт представляє собою сукупність логістичних хабів, пунктів доставки та транспортних коридорів, що їх з'єднують. З математичної точки зору таку структуру доцільно розглядати як мультимодальний зважений орієнтований граф. Визначення графу наведено в формулі (1.1):

$$G = (V, E, \Phi, \Psi) \quad (1.1)$$

де V – вершини графа;

E – ребра графа.

Функціональні особливості об'єкта полягають у динамічній взаємодії різних типів транспортних засобів (автомобілів, велосипедів, кур'єрів-пішоходів) для виконання завдань доставки. Кожна одиниця транспорту має обмежену вантажну місткість, а кожен маршрут повинен бути завершений у межах визначеного часового вікна хабу [4]. Особливе значення має функція «останньої милі», яка передбачає можливість долання частини шляху пішки, якщо прямий доступ транспорту до точки призначення обмежений або неможливий.

1.3 Огляд і аналіз літератури за темою роботи

Проблема маршрутизації транспортних засобів (Vehicle Routing Problem, VRP) є однією з найбільш інтенсивно досліджуваних тем у сучасній комбінаторній оптимізації та дослідженні операцій. Фундаментальні основи математичного опису подібних задач були закладені ще в середині XX століття. Зокрема, для побудови математичних моделей у вигляді задач змішаного цілочисельного програмування критично важливими залишаються методи, запропоновані Міллером, Такером та Земліним [5]. Їхня робота 1960 року впровадила так звані обмеження MTZ (Miller-Tucker-Zemlin) для елімінації підциклів у маршрутах [6]. Це дозволило формалізувати задачу комівояжера та VRP як цілісну систему лінійних обмежень, що стала базовою для всіх наступних розробок у галузі цілочисельного програмування.

Особливе місце в сучасній логістиці посідає задача маршрутизації з часовими вікнами (Vehicle Routing Problem with Time Windows, VRPTW). Класичними у цьому напрямку є праці Маріуса Соломона, який у 1987 році не лише розробив сімейство евристичних алгоритмів (зокрема евристику послідовних вставок I1), а й створив набір тестових задач (Solomon benchmarks), що донині є світовим стандартом для оцінки ефективності алгоритмів [7]. Його підхід базується на критеріях мінімізації додаткового часу та відстані при вставці нового вузла в існуючий маршрут, що дозволяє знаходити допустимі розв'язки для NP-складних задач великої розмірності за поліноміальний час.

Сучасні дослідження значно розширили можливості моделювання реальної інфраструктури. У фундаментальній монографії Паоло Тота та Даніеле Віго «The Vehicle Routing Problem» проведено системний аналіз точних та наближених методів розв'язання VRP [8]. Автори детально розглядають використання теорії графів для побудови мультимодальних моделей, де вузли та ребра характеризуються наборами динамічних атрибутів. Їхні дослідження алгоритмів «гілок та меж» (branch-and-bound) та «гілок та розрізів» (branch-and-

cut) заклали підґрунтя для сучасного математичного забезпечення логістичних систем.

Детальний аналіз метаевристичних підходів був проведений Брейсі та Жендро, які систематизували методи локального пошуку та еволюційні стратегії для VRPTW. Вони довели, що для динамічних міських мереж використання лише конструктивних евристик є недостатнім через їхню схильність до застрягання в локальних оптимумах [9]. У свою чергу, дослідження Ванга та Копфера підкреслюють важливість інтеграції складних процесів у єдині моделі, що враховують не лише магістральні перевезення, а й специфіку доставки «останньої милі» та спільне використання ресурсів декількома логістичними операторами [10].

Аналіз літературних джерел підтверджує, що актуальним завданням є створення алгоритмічного забезпечення, яке б ефективно поєднувало швидкість класичних евристик Соломона (для генерації початкової популяції) із потужним глобальним пошуком генетичних алгоритмів у межах мультимодальних графових моделей.

1.4 Математична постановка задачі

Математична модель задачі базується на теорії графів та методах змішаного цілочисельного програмування.

За формулою (1.1), мережа формалізується як орієнтований граф.

Як цільова функція розглядається мінімізація сумарних витрат, що складаються з часу переїздів та кількості задіяних транспортних засобів.

$$F = A \cdot \sum_{k \in V} \sum_{i,j \in N} t_{ij} x_{ij}^k + B \cdot \sum_{k \in V} y^k \rightarrow \min \quad (1.2)$$

де t_{ij} – час у дорозі між вузлами i та j , який залежить від відстані та швидкості

транспортного засобу;

x_{ij}^k – бінарна змінна (1, якщо ТЗ k їде від i до j , інакше – 0);

y^k – бінарна змінна використання транспортного засобу k ;

A, B – вагові коефіцієнти пріоритетності (час та кількість транспорту).

Обмеження:

- Місткість: сумарна вага вантажів у кожному ТЗ не повинна перевищувати його ліміт Q ;
- Часові вікна: прибуття у вузол i має відбутися в межах $[e_i, l_i]$;
- Зв'язність: кожний клієнт повинен бути відвіданий рівно один раз.

1.5 Висновки за загальним розділом

У результаті аналізу предметної області встановлено, що задача маршрутизації (VRP) у сучасних мультимодальних мережах є NP-складною, що унеможливорює використання виключно точних методів для реальних масштабів інфраструктури. Огляд літератури та існуючих підходів підтвердив, що класичні конструктивні евристики часто застрягають у локальних оптимумах, тому виникає гостра потреба у використанні метаевристичних методів. Доведено актуальність розробки алгоритмічного забезпечення на основі генетичних алгоритмів, які здатні ефективно знаходити наближені до глобального оптимуму розв'язки з урахуванням жорстких часових вікон та специфіки «останньої милі». Сформована теоретична база та математична модель об'єкта стають основою для подальшої програмної реалізації логістичної системи.

2 СПЕЦІАЛЬНИЙ РОЗДІЛ

2.1 Проєктування розроблюваної інформаційної системи

Для проєктування системи оптимізації розподілу вантажів у мультимодальній мережі обрано об'єктно-орієнтований підхід (ООП), що забезпечує гнучкість при моделюванні складних логістичних сутностей та природне відображення предметної області у програмний код. Вибір ООП-підходу зумовлений рядом міркувань: він узгоджується з принципом концептуальної спільності, який вимагає використання однієї парадигми як на етапі аналізу й проєктування, так і на етапі реалізації; також обрана мова програмування Python надає повноцінну підтримку об'єктно-орієнтованої парадигми через класи, успадкування, поліморфізм та інкапсуляцію; логістичні сутності (транспортний засіб, точка доставки, маршрут, пакунок) природно відображаються в об'єкти зі станом і поведінкою, що робить ООП-декомпозицію інтуїтивно зрозумілою. Декомпозиція системи виконана за допомогою CASE-засобу PlantUML, що дозволяє формалізувати структуру системи через стандарти UML.

Як інструмент моделювання застосовано CASE-засіб PlantUML – текстовий генератор UML-діаграм, який дозволяє описувати моделі мовою спеціалізованої DSL та автоматично формувати графічне представлення. Перевагами PlantUML є: текстовий формат вихідних файлів (придатний для версійного контролю в Git та порівняння змін у diff-режимі), повна підтримка специфікації UML 2.5, незалежність від графічних редакторів та можливість швидкого внесення змін до моделей без ручного перерисовування. Усі побудовані діаграми відповідають нотації UML 2.5 та сформовані у вигляді векторних PNG-зображень з єдиними стильовими параметрами.

Проєктування системи виконано у чотирьох взаємодоповнюючих аспектах моделювання, які разом утворюють повну архітектурну специфікацію інформаційного продукту:

– Функціональний аспект – описує систему з точки зору її користувачів та зовнішніх сервісів, формалізує сукупність бізнес-сценаріїв і визначає функціональну межу продукту. Представлений діаграмою прецедентів (Use Case diagram) – п. 2.1.1.

– Структурний аспект – фіксує статичну будову системи: сутності предметної області, їх атрибути та відношення, а також модульну декомпозицію вихідного коду. Представлений двома діаграмами класів: предметної області (п. 2.1.2) та модульної архітектури (п. 2.1.3).

– Поведінковий аспект – деталізує взаємодію компонентів у часі для конкретних бізнес-сценаріїв. Представлений трьома діаграмами послідовностей (sequence diagrams), що покривають ключові динамічні процеси системи: геокодування адреси (п. 2.1.4), одно-транспортна оптимізація маршруту (п. 2.1.5) та багатотранспортна оптимізація VRP (п. 2.1.6).

– Динамічний аспект – узагальнює потік управління та обробки даних у вигляді єдиного бізнес-процесу з явним зображенням точок прийняття рішень, паралельних потоків та сценаріїв обробки помилок. Представлений діаграмою діяльності (activity diagram) – п. 2.1.7.

Загалом для опису архітектури системи побудовано сім UML-діаграм, кожна з яких розглядає продукт під своїм кутом зору та доповнює інші, утворюючи повну й внутрішньо узгоджену модель. Послідовність викладу обрано від найбільш абстрактних і користувацько-орієнтованих моделей (функціональна) через статичну структуру (структурні) до динамічних аспектів виконання (поведінкові, діяльнісна). Така послідовність відповідає принципам стандарту ISO 42010 щодо опису архітектури програмного забезпечення і дозволяє читачеві поступово занурюватися у деталі реалізації, починаючи з загальної функціональної картини [11].

2.1.1 Функціональна модель системи (діаграма прецедентів)

Для формалізації функціональних вимог до інформаційної системи та визначення меж її відповідальності побудовано діаграму прецедентів (Use Case diagram) у стандарті UML 2.5. Діаграма зображує всю сукупність функціональних можливостей системи з точки зору акторів, які з нею взаємодіють, і дозволяє чітко окреслити функціональну межу розроблюваного програмного продукту. Діаграму прецедентів наведено на рисунку 2.1.

На діаграмі виділено чотири актори. Первинним є Оператор логістики – користувач системи, який безпосередньо ініціює всі бізнес-сценарії через веб-інтерфейс. Решта три актори є вторинними (системними) і відповідають зовнішнім інформаційним сервісам, з якими взаємодіє розроблюваний продукт: Nominatim API – сервіс прямого та зворотного геокодування адрес у складі проєкту OpenStreetMap; OpenStreetMap (Overpass API) – джерело первинних картографічних даних про вуличну мережу; Mapbox API – комерційний сервіс отримання матриці часів руху з урахуванням реального трафіку (використовується опціонально).

Функціональність системи представлено сукупністю прецедентів, які можна об'єднати у чотири логічні групи. До групи керування вхідними даними належать прецеденти «Обрати місто доставки», «Вказати адресу депо», «Додати адресу доставки», «Додати точку на карті» та «Видалити точку доставки» – вони забезпечують формування первинного набору точок доставки. Група конфігурації автопарку представлена прецедентом «Керувати парком ТЗ», що дозволяє додавати, редагувати та видаляти транспортні засоби з заданням їх режиму пересування і вантажомісткості. Група розрахункових операцій містить ключовий прецедент «Запустити оптимізацію», який є точкою входу для повного циклу обчислень. Група візуалізації та обліку – це прецеденти «Переглянути результати на карті», «Переглянути таблицю маршруту» та «Керувати пакунками», які забезпечують подання результатів та облік стану замовлень.

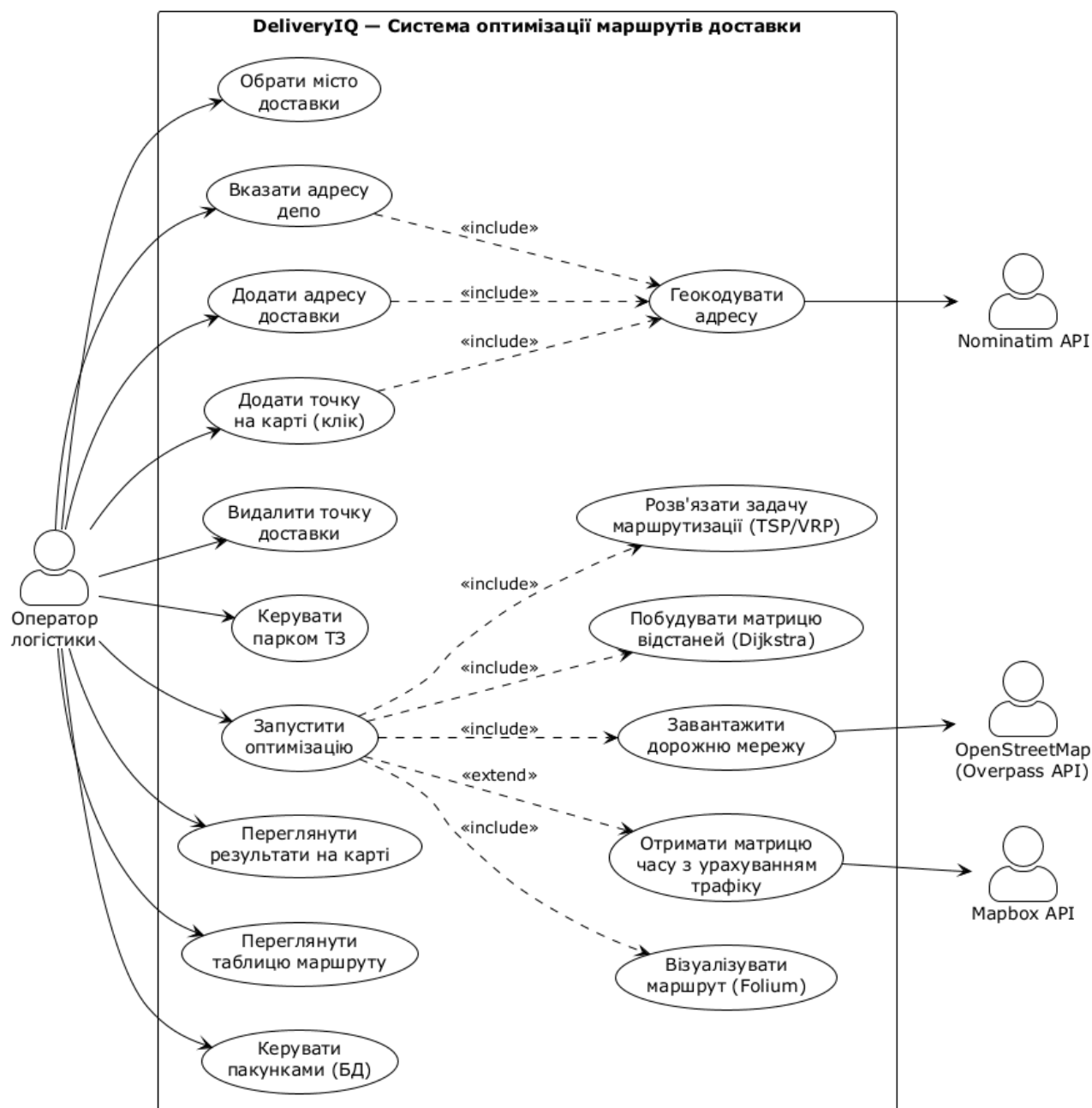


Рисунок 2.1 – Діаграма прецедентів

Між прецедентами встановлено два типи відношень, передбачених стандартом UML. Відношення «include» (включення) використано для декомпозиції складених прецедентів на обов'язкові підпроцеси. Так, прецедент «Запустити оптимізацію» обов'язково включає підпроцеси «Завантажити дорожню мережу», «Побудувати матрицю відстаней (Dijkstra)», «Розв'язати задачу маршрутизації (TSP/VRP)» та «Візуалізувати маршрут (Folium)». Аналогічно прецеденти введення адрес («Вказати адресу депо», «Додати адресу

доставки», «Додати точку на карті») включають прецедент «Геокодувати адресу», оскільки без перетворення текстової адреси у пару координат подальша обробка неможлива. Відношення «extend» (розширення) застосовано до прецеденту «Отримати матрицю часу з урахуванням трафіку», який розширює базовий прецедент оптимізації лише за наявності у конфігурації валідного ключа Mapbox API; у разі його відсутності система прозоро повертається до використання статичної матриці часів, побудованої на основі OSM-графа.

2.1.2 Структурна модель – діаграма класів предметної області

Для опису внутрішньої структури інформаційної системи з точки зору сутностей предметної області та відношень між ними побудовано діаграму класів предметної області. На відміну від модульної діаграми класів, що відображає програмну архітектуру (див. п. 2.1.3), діаграма предметної області абстрагується від реалізаційних деталей і фокусується на бізнес-сутностях логістичної системи та їх атрибутах. Діаграму класів предметної області наведено на рисунку 2.2.

Всі класи предметної області згруповано у п'ять логічних пакетів, кожен з яких відповідає окремому аспекту функціонування системи.

Пакет «Доставка» містить три ключові класи. Клас `DeliveryStop` є центральною сутністю системи й описує одну точку доставки: атрибути `address` (адреса), `lat/lon` (координати), `node_id` (індекс вузла на дорожньому графі), `weight_kg` (вага вантажу) та `source` (джерело: введення вручну або клік на карту). Клас `LegInfo` представляє один сегмент маршруту з атрибутами `from_label`, `to_label` (мітки початкового та кінцевого пунктів), `distance_m`, `travel_time_s` та `cumulative_time_s` (загальний час). Клас `ModeResult` агрегує сукупність сегментів `LegInfo` (зв'язок «Один до багатьох»), визначаючи завершений маршрут для одного режиму пересування з повними даними `tsp_route`, `full_route` та

total_time_s.

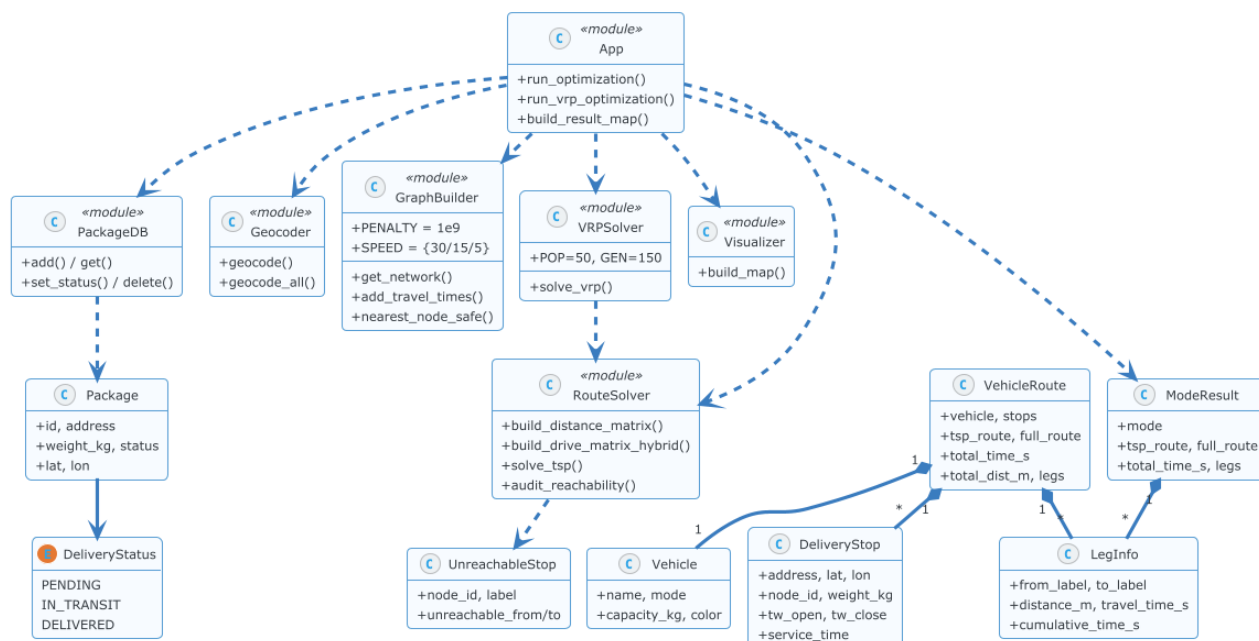


Рисунок 2.2 – Діаграма класів системи оптимізації вантажоперевезень

Пакет «Парк ТЗ» включає два класи. Vehicle описує транспортний засіб (атрибути name, mode, capacity_kg, color), а VehicleRoute пов'язує конкретний транспортний засіб зі своїм впорядкованим списком зупинок. Саме VehicleRoute є вихідною сутністю при розв'язанні багатотранспортної задачі CVRPTW.

Пакет «Транспортна мережа» реалізує інфраструктурний шар. Клас TransportGraph зі стереотипом «MultiDiGraph» агрегує множини NodeData (вузли з атрибутами id, y, x – широта і довгота) та EdgeData (ребра з атрибутами length, highway, travel_time загальний та три модальні travel_time_drive/bike/walk, а також модальні теги motor_vehicle, bicycle, foot, oneway). Така структура з трьома незалежними модальними часами на одному ребрі дозволяє підтримувати мультимодальну маршрутизацію без дублювання геометрії графа.

Пакет «База пакунків» інкапсулює персистентний шар. Клас PackageDB приховує деталі реалізації SQLite (поле _conn має модифікатор приватності) і пропонує публічний інтерфейс CRUD-операцій (add_package(), set_status(), get_by_status(), get_all(), delete()). Клас Package (атрибути id, address, weight_kg,

status, created_at, lat, lon) пов'язаний з перерахуванням DeliveryStatus, що приймає одне з трьох значень: PENDING, IN_TRANSIT, DELIVERED.

Пакет «Геокодування» містить допоміжний клас Location (address, lat, lon, node_id), який є проміжним результатом виклику геокодувального сервісу Nominatim. Між Location та DeliveryStop встановлено відношення залежності зі стереотипом «створюється з», що відображає сценарій перетворення геокодованої локації у точку доставки.

Окремо від пакетів зображено клас UnreachableStop (атрибути node_id, label, unreachable_from, unreachable_to, is_isolated та метод summary()), що відповідає за діагностику та звітування про точки доставки, які не вдалося досягти жодним з режимів пересування. Цей клас використовується модулем аудиту досяжності та інтегрується з підсистемою попереджень користувача.

Між класами використано три типи структурних відношень: композиція для випадків, коли життєвий цикл частини повністю залежить від цілого; агрегація для слабкого володіння, де частини можуть існувати незалежно; та залежність для випадків тимчасового використання одного класу іншим без структурної прив'язки.

Побудована модель є основою для подальшого опису структури бази даних у п. 2.2 та узгоджується з модульною архітектурою системи, представленою у п. 2.1.3.

2.1.3 Структурна модель – модульна архітектура

Структурну модель системи DeliveryIQ доповнено діаграмою класів модульної структури, побудованою у нотації UML 2.5. На відміну від доменної моделі (п. 2.1.2), яка описує сутності предметної області, модульна діаграма відображає декомпозицію вихідного коду на функціональні модулі та формалізує службові залежності між ними. Кожен пакет на діаграмі відповідає

одному файлу-модулю Python, а класи всередині пакета зображено зі стереотипом «module» та переліком публічних констант, атрибутів і функцій. Діаграму наведено на рисунку 2.3.

На діаграмі виділено шість модулів, що утворюють шарувату архітектуру із чітко визначеними обов'язками.

Модуль `app` (стереотип «Streamlit UI») є центральним контролером системи та точкою входу веб-застосунку. Він зберігає глобальні константи `DEFAULT_CITY = "Dnipro, Ukraine"`, `LAST_METER_THRESHOLD_M = 100.0` та посилення на синглтон-екземпляри геокодувального клієнта `_geolocator` і бази пакунків `_pkg_db`. Модуль експонує функції кешованої побудови мережі (`cached_network()`, `cached_mode_graphs()`), фасадні функції геокодування (`forward_geocode()`, `reverse_geocode()`), а також дві ключові функції-оркестратори: `run_optimization()` для одиничного транспортного засобу та `run_vrp_optimization()` для багатотранспортної задачі. Допоміжні функції `build_selection_map()` і `build_result_map()` формують Folium-карти для відображення на сторінці.

Модуль `vrp_solver` реалізує ядро генетичного алгоритму для CVRPTW. Окрім константи `VEHICLE_COLORS` (палітра кольорів маркерів автопарку), модуль експонує публічну функцію `solve_vrp()`, що приймає список зупинок, депо, флот та модальні графи й повертає сукупність маршрутів `VehicleRoute` разом зі списком попереджень. Внутрішні службові функції – `_assign_stops_to_modes()` (Фаза 1, призначення зупинок до сумісних режимів), `_build_ga_routes()` (Фаза 2, генетичний алгоритм) та `_decode_chromosome()` (Split-декодер хромосоми) – приховано через префікс підкреслення.

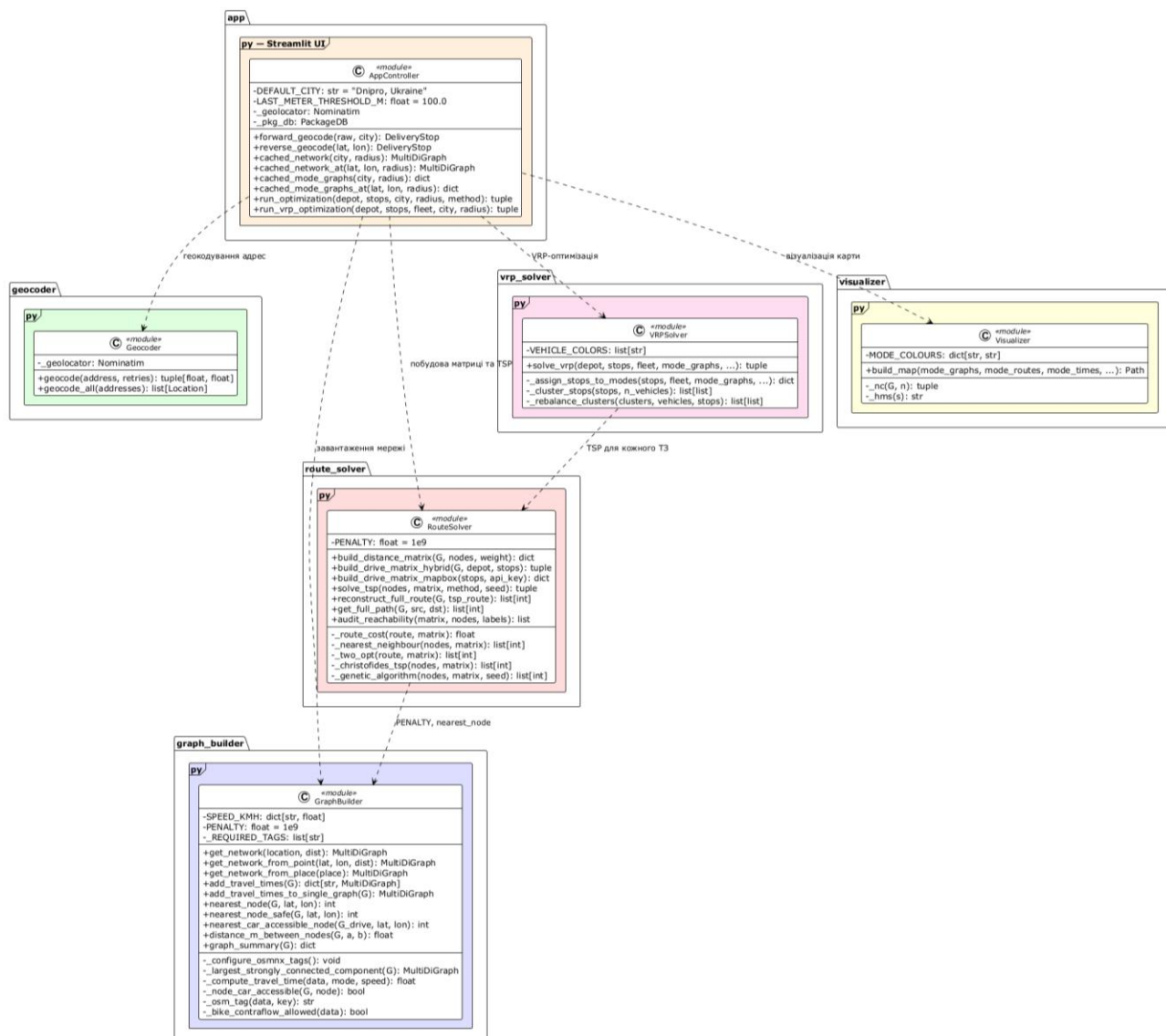


Рисунок 2.3 – Діаграма класів модульної архітектури системи

Модуль `route_solver` інкапсулює побудову матриць відстаней та алгоритми розв'язання задачі комівояжера. Експортує константу `PENALTY = 1e9`, для маркування недосяжних пар вузлів, функції побудови матриць трьох типів (`build_distance_matrix()` – статична через Дейкстру, `build_drive_matrix_hybrid()` – гібридна з пішим «останнім метром», `build_drive_matrix_mapbox()` – динамічна через Mapbox API), уніфіковану функцію `solve_tsp(method=...)` з підтримкою чотирьох алгоритмів (Nearest Neighbour, Christofides, 2-opt, Genetic), реалізованих у приватних функціях `_christofides()`, `_two_opt()`, `_genetic()`. Допоміжні `reconstruct_full_route()` та `get_full_path()` відновлюють повну послідовність вузлів на дорожньому графі, а `audit_reachability()` виявляє

ізолювані пари.

Модуль `graph_builder` відповідає за роботу з картографічними даними. Зберігає словник модальних швидкостей `SPEED_KMH = {drive: 30, bike: 15, walk: 5}` (км/год), константу `PENALTY` та перелік `REQUIRED_TAGS` для конфігурації `OSMnx`. Публічні функції покривають повний цикл роботи з графом: завантаження мережі (`get_network()`, `get_network_from_point()`), проставлення модальних часів (`add_travel_times()` – повертає три копії графа, `add_travel_times_to_single_graph()` – єдиний граф з атрибутами `travel_time_drive/bike/walk`), прив'язку координат до найближчого вузла (`nearest_node()`, `nearest_node_safe()`, `nearest_car_accessible_node()`), вимірювання відстані між вузлами (`distance_m_between_nodes()`) та діагностику (`graph_summary()`). Приватні функції `_configure_osmnx_tags()`, `_prune_to_lscc()`, `_node_car_accessible()` забезпечують внутрішню логіку фільтрації та обрізки графа.

Модуль `geocoder` є тонкою обгорткою над бібліотекою `GeoPy` і експортує спрощений інтерфейс з двох функцій – `forward_geocode(address, city)` та `reverse_geocode(lat, lon)` – що повертають кортежі координат і текстові адреси. Виділення цього модуля окремо дозволяє замінити провайдера геокодування (наприклад, на `Google Maps Geocoding API`) без зміни решти коду.

Модуль `visualizer` агрегує засоби побудови інтерактивних карт. Зберігає словник модальних кольорів `MODE_COLOURS`, експортує функцію `build_map()` для формування `Folium`-карти з шарами `AntPath`, а також приватні допоміжні функції `_ant()` (формування стилю анімованої лінії) та `_hms()` (форматування часових тривалостей).

Між модулями на діаграмі встановлено п'ять відношень залежності зі стереотипом «use»: `app` залежить від усіх п'яти службових модулів (`vrp_solver`, `route_solver`, `graph_builder`, `geocoder`, `visualizer`); `vrp_solver` використовує `route_solver` (для розв'язання TSP кожного транспортного засобу окремо) та `graph_builder` (для отримання `nearest_car_accessible_node` у режимі гібридної доставки); `route_solver` залежить від `graph_builder` (узгодження константи

PENALTY та використання `nearest_node`). Така структура утворює орієнтований ациклічний граф залежностей, що відповідає принципу шаруватої архітектури та запобігає виникненню циклічних імпортів.

2.1.4 Поведінкова модель – геокодування адреси

Для опису динамічного аспекту функціонування системи в межах одного бізнес-сценарію застосовано діаграми послідовностей (sequence diagrams) UML. Першу таку діаграму присвячено сценарію геокодування адреси доставки – це найчастіше виконуваний сценарій у роботі оператора та критична точка інтеграції системи із зовнішнім сервісом Nominatim. Діаграму послідовності наведено на рисунку 2.4.

На діаграмі виділено три лінії життя (lifeline): актора Оператор, програмний модуль `app.py` (стереотип «Streamlit UI») та зовнішній актор Nominatim API (з реалізаційним коментарем «geopy», що позначає клієнтську бібліотеку доступу). Сценарій складається з двох незалежних потоків: основного (введення адреси вручну) та альтернативного (додавання точки кліком на карту), розділених на діаграмі горизонтальною межею.

Основний сценарій – текстове введення адреси. Послідовність розпочинається з повідомлення «Вводить адресу в поле "Add delivery address"», яке оператор надсилає до модуля `app.py`. Модуль виконує `self-message` – внутрішній виклик функції `_lock(address, city)`, що дописує до введеного рядка локацію міста (наприклад, додає суфікс «Dnipro, Ukraine»), забезпечуючи прив'язку результату геокодування до конкретного міста та усуваючи неоднозначність омонімічних топонімів. Далі модуль викликає синхронний метод `geocode(query, timeout=10)` сервісу Nominatim і чекає на відповідь.

Після отримання відповіді управління переходить до блоку альтернатив, який моделює три можливі сценарії розгалуження:

– Гілка «Адресу знайдено» – найімовірніший сценарій. Nominatim повертає об'єкт `result(address, lat, lon)`. Модуль `app.py` створює екземпляр доменного класу `DeliveryStop` з атрибутами `address=result.address`, `lat=result.latitude`, `lon=result.longitude` та `source="typed"`. Створений об'єкт додається до колекції точок доставки у сесійному стані `Streamlit` (`st.session_state["stops"].append(stop)`). Завершальним повідомленням до оператора є оновлення інтерфейсу: «Маркер на карті + запис у списку доставок».

– Гілка «Адресу не знайдено» спрацьовує, коли Nominatim повертає `None`. Модуль формує повідомлення про помилку «Could not geocode address» і виводить його у вигляді червоного банера. Дані у сесійний стан не записуються – інваріант «список точок містить лише валідні геокодовані адреси» зберігається.

– Гілка «Таймаут» активується при перевищенні часу очікування (10 секунд) – клієнтська бібліотека викидає виняток `GeocoderTimedOut`, який перехоплюється модулем і трансформується у повідомлення «Geocoder timed out» для оператора. Така обробка забезпечує стійкість системи до тимчасових проблем з мережевим з'єднанням та перевантаження сервера Nominatim.

Альтернативний сценарій – додавання точки кліком на карту. Оператор клікає на довільну точку інтерактивної карти, і браузер передає координати (`lat`, `lon`) модулю `app.py` через механізм `Streamlit`-сесії. Модуль виконує зворотне геокодування – викликає метод `reverse((lat, lon), language="en")` сервісу Nominatim, який за координатами повертає найближчу адресу у вигляді текстового рядка. На основі отриманого результату створюється екземпляр `DeliveryStop` з атрибутами `address=result.address`, `lat=lat`, `lon=lon` та `source="map_click"` (атрибут `source` дозволяє згодом розрізняти точки, додані текстово і кліком – наприклад, для аналітичних звітів). Об'єкт додається до сесійного стану, а на карті з'являється новий маркер з відображенням зворотно-геокованої адреси.

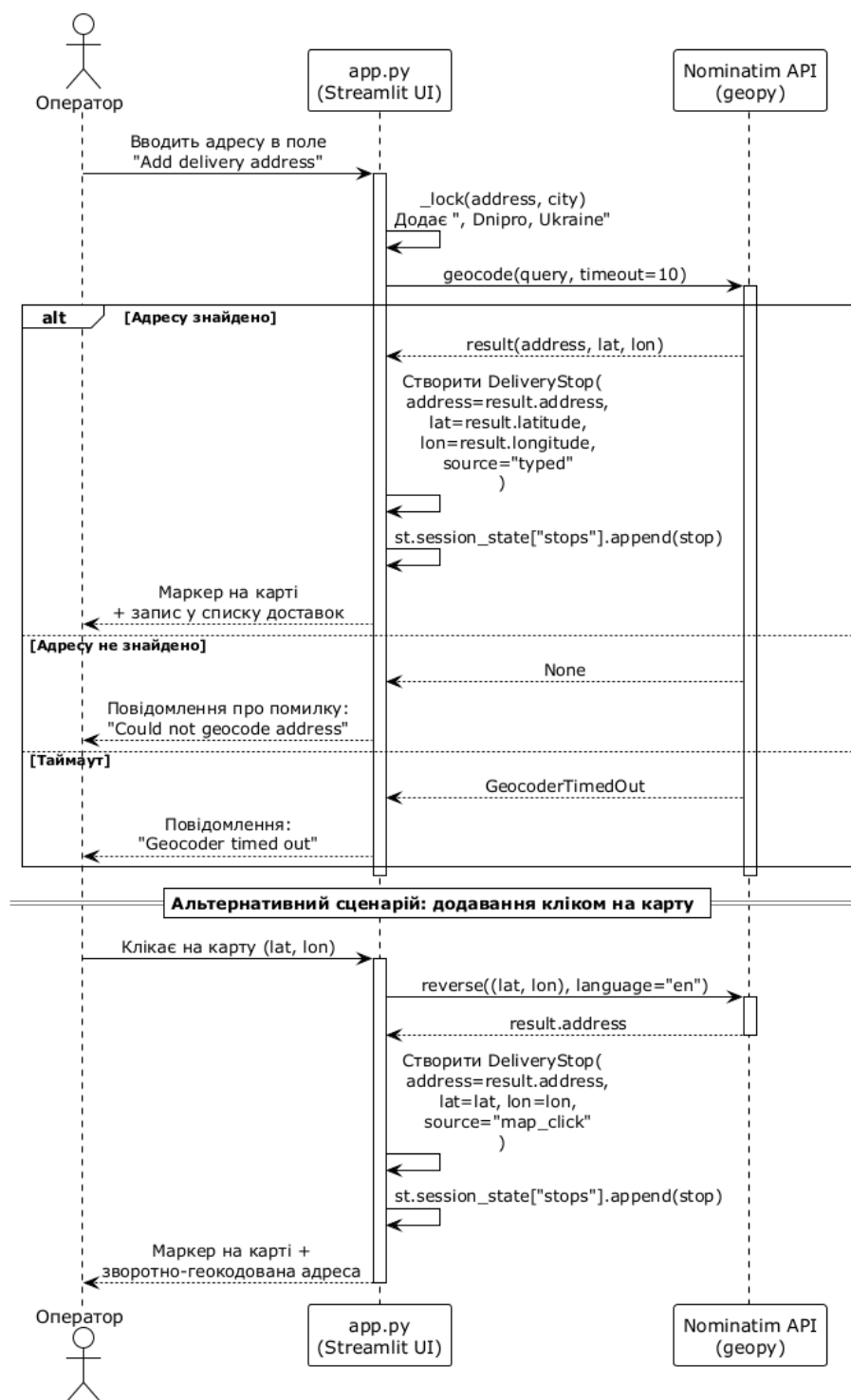


Рисунок 2.4 – Діаграма послідовності геокодування адреси

Доступ до Nominatim ізольовано у модулі `geocoder.py`. Решта системи бачить лише дві функції – `forward_geocode()` і `reverse_geocode()` – і нічого не знає ні про HTTP-запити, ні про таймаути, ні про специфіку відповідей сервісу.

2.1.5 Поведінкова модель – оптимізація одного транспортного засобу

Для документування динаміки виконання основного розрахункового сценарію – побудови оптимального маршруту для одного транспортного засобу із порівнянням трьох режимів пересування – побудовано розгорнуту діаграму послідовностей. Цей сценарій є найбільш обчислювально складною операцією системи і об'єднує взаємодію всіх п'яти службових модулів між собою та з зовнішнім сервісом OpenStreetMap. Діаграму послідовності наведено на рисунку 2.5.

На діаграмі представлено шість ліній життя: актор Оператор, чотири внутрішні модулі – `app.py` (Streamlit UI), `graph_builder.py` (GraphBuilder), `route_solver.py` (RouteSolver), `visualizer.py` (Visualizer) – та зовнішній актор OSM (Overpass API) як джерело первинних картографічних даних. Для покращення читабельності повний цикл оптимізації розділено на шість фаз, кожна з яких винесена в окрему секцію зі стереотипом «ref».

Фаза 1 – Завантаження мережі. Сценарій ініціюється повідомленням «Натискає Optimize Routes» від оператора до `app.py`. Контролер викликає функцію `cached_network_at(depot.lat, depot.lon, radius)` модуля `graph_builder.py`. Перед формуванням HTTP-запиту GraphBuilder виконує внутрішній виклик `_configure_osmnx_tags()`, що налаштовує бібліотеку OSMnx на збереження ключових модальних тегів (`bicycle`, `motor_vehicle`, `foot`, `cycleway`, `oneway`). Далі модуль робить синхронний запит `ox.graph_from_point(lat, lon, dist, "all")` до Overpass API, отримує сирий мультимодальний граф `G_raw` типу `MultiDiGraph` та виконує self-message `_largest_strongly_connected_component(G_raw)`, що відсікає ізольовані компоненти графа й залишає лише найбільшу сильно зв'язну компоненту (LSCC). Очищений граф `G` повертається до контролера.

Фаза 2 – Прив'язка адрес до вузлів. Контролер у комбінованому фрагменті зі стереотипом «loop» для кожної адреси (депо та усі точки доставки) викликає функцію `nearest_node(G, lat, lon)`, що повертає `node_id` – числовий ідентифікатор

найближчого вузла графа. Після виходу з циклу `app.py` виконує `self-message` Дедуплікація вузлів (`dict.fromkeys`) – це усуває потенційні колізії, коли дві різні адреси внаслідок невисокої точності геокодування прив'язалися до одного й того ж вузла OSM-графу.

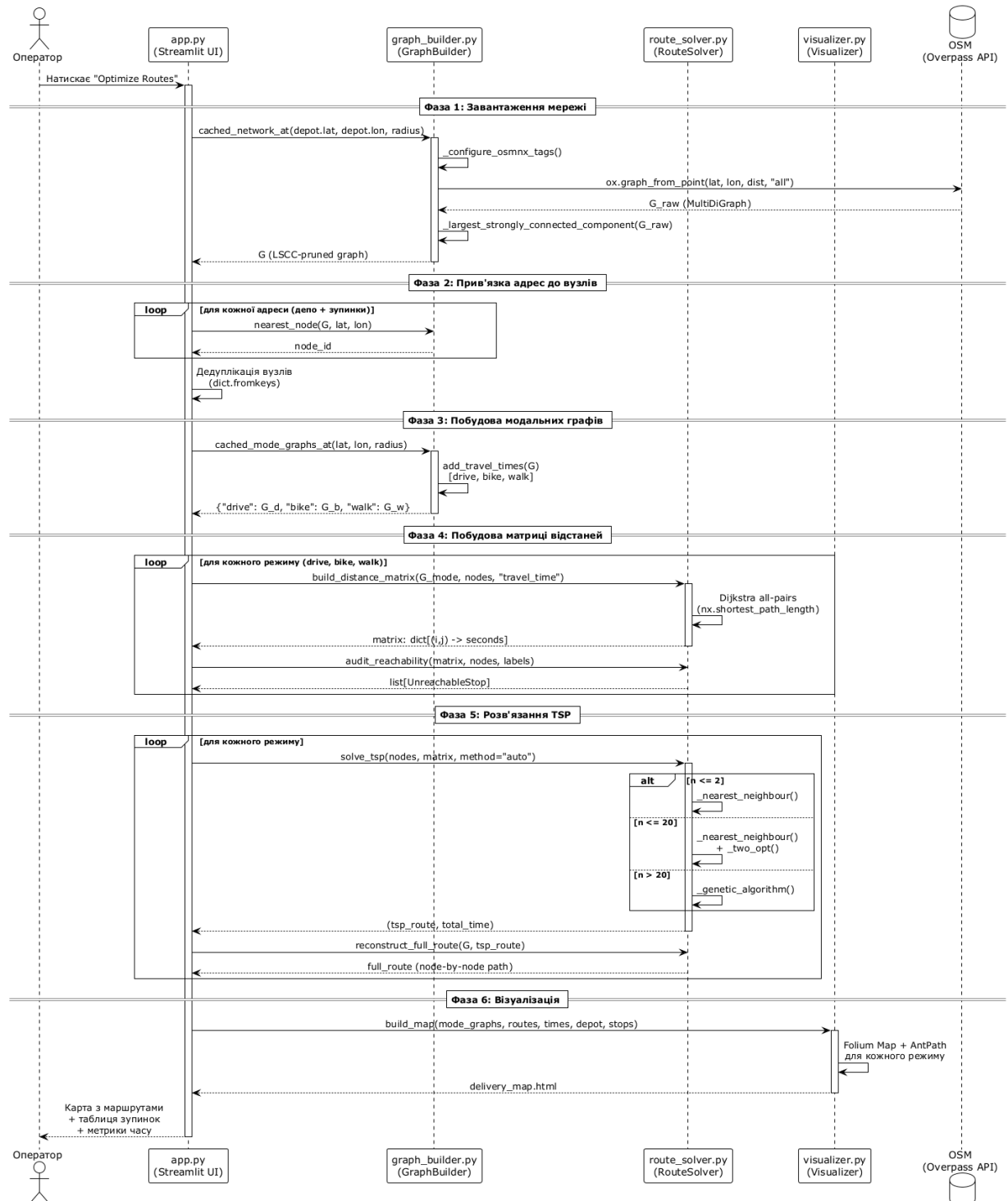


Рисунок 2.5 – Діаграма послідовності процесу оптимізації маршруту

Фаза 3 – Побудова модальних графів. Контролер викликає `cached_mode_graphs_at(lat, lon, radius)` модуля `GraphBuilder`, який внутрішньо виконує `add_travel_times(G)` – створення трьох глибоких копій графа із проставленням атрибута `travel_time` на ребрах за модальними швидкостями (`drive` – 30 км/год, `bike` – 15, `walk` – 5). Повертається словник `{"drive": G_d, "bike": G_b, "walk": G_w}`. Кешування цієї операції критичне, оскільки обхід графа з кількома десятками тисяч ребер для проставлення модальних часів займає 2-5 секунд.

Фаза 4 – Побудова матриці відстаней. У циклі `loop` для кожного з трьох режимів (`drive`, `bike`, `walk`) контролер викликає `build_distance_matrix(G_mode, nodes, "travel_time")` модуля `RouteSolver`. Той виконує self-message «Dijkstra all-pairs» (`nx.shortest_path_length`) – обчислення матриці найкоротших часів між усіма парами вузлів. Результат повертається у вигляді словника `dict[(i, j) → seconds]`. Після побудови матриці контролер викликає допоміжну `audit_reachability(matrix, nodes, labels)`, що повертає список об'єктів `UnreachableStop` – точок, недосяжних у даному режимі (наприклад, велосипедна доставка через пішохідну зону).

Фаза 5 – Розв'язання задачі TSP. Для кожного режиму контролер викликає уніфіковану функцію `solve_tsp(nodes, matrix, method="auto")`. У режимі автоматичного вибору алгоритму `RouteSolver` застосовує комбінований фрагмент «alt» із трьома гілками: для 2 та менше точок доставки – простий `_nearest_neighbour()`; від 2 до 20 – комбінація `_nearest_neighbour()` та `_two_opt()`; для 20 та більше – `_genetic_algorithm()`. Така адаптивна стратегія мінімізує загальний час обчислень: для малих задач відсутній накладний час популяційної метаевристики, для великих – гарантується якість розв'язку. Результат (`tsp_route`, `total_time`) повертається до контролера, який далі викликає `reconstruct_full_route(G, tsp_route)` для відновлення повного `full_route` – послідовності всіх проміжних вузлів графа, придатної для рендерингу.

Фаза 6 – Візуалізація. Контролер передає всі підготовлені дані (`mode_graphs`, `routes`, `times`, `depot`, `stops`) до `build_map()` модуля `Visualizer`. Той

формує об'єкт Folium Map, накладає на нього три анімовані шари AntPath (по одному для кожного режиму, з різними кольорами), маркери депо та точок доставки, після чого серіалізує карту у HTML-документ `delivery_map.html`, який повертається контролеру. Завершальним повідомленням оператор отримує «Карта з маршрутами, таблиця зупинок, метрики часу».

2.1.6 Поведінкова модель – оптимізація багатьох ТЗ

Сценарій багатотранспортної оптимізації є найскладнішим з точки зору алгоритмічної логіки, оскільки до задачі побудови маршруту додаються етапи розподілу зупинок між транспортними засобами з урахуванням їх режимів та вантажомісткості. Діаграма послідовностей цього сценарію демонструє трифазну архітектуру VRP-розв'язувача та його взаємодію з іншими модулями системи. Діаграму наведено на рисунку 2.6.

На діаграмі представлено п'ять ліній життя: актор Оператор, контролер `app.py` (Streamlit UI), ключовий розв'язувач `vrp_solver.py` (VRPSolver), модуль `route_solver.py` (RouteSolver) та допоміжний `graph_builder.py` (для виклику `nearest_car_accessible_node` у режимі гібридної доставки). Сценарій ініціюється повідомленням «Налаштовує парк ТЗ (тип, місткість) та натискає Optimize Fleet» від оператора. Контролер виконує єдиний фасадний виклик `solve_vrp(depot, stops, fleet, mode_graphs, G_raw, radius)` модуля VRPSolver, що повертає кортеж `(list[VehicleRoute], warnings)`. Внутрішня структура цього виклику розділена на три послідовні фази, винесені в окремі секції стереотипу «ref».

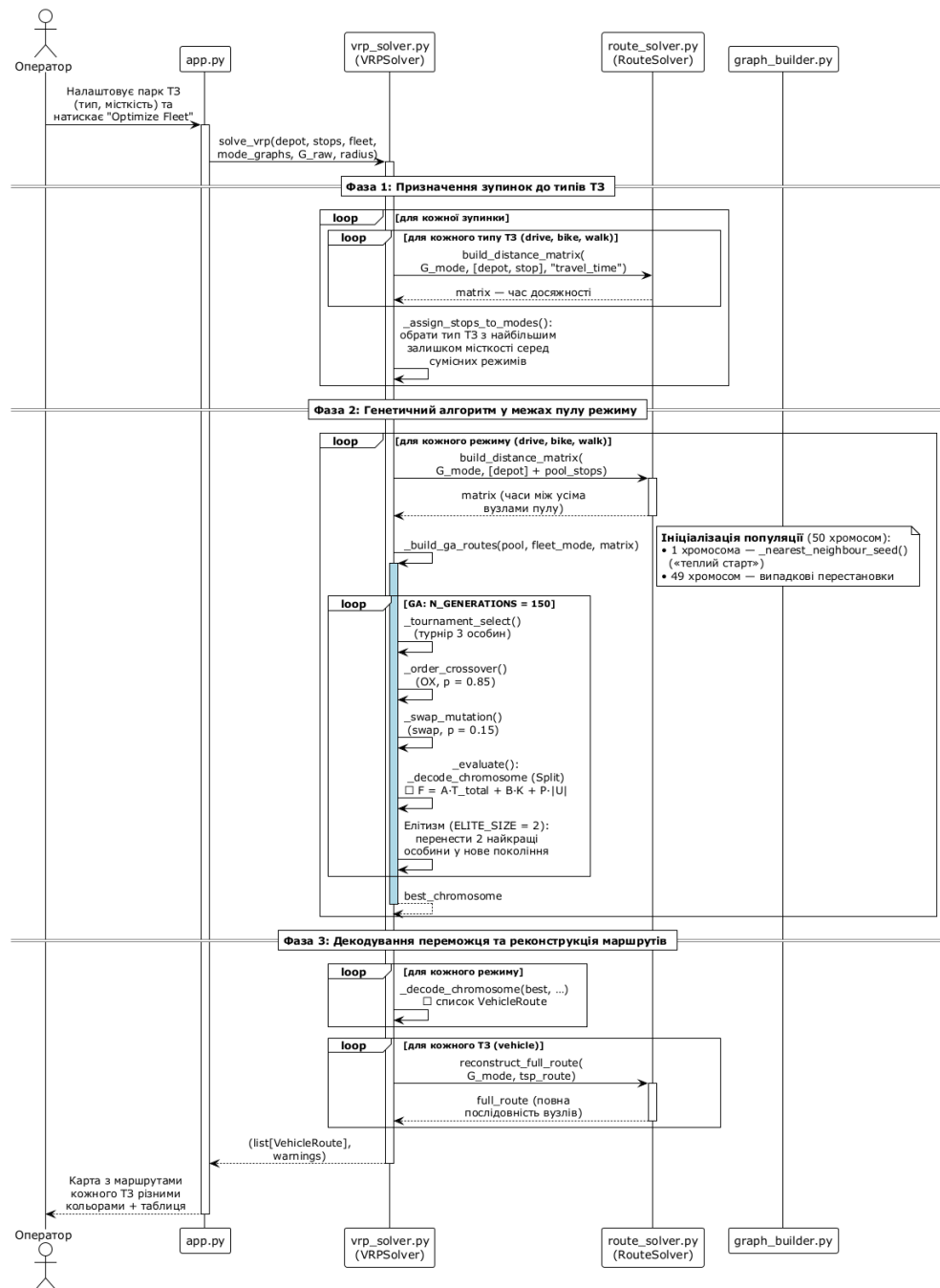


Рисунок 2.6 – Діаграма послідовності процесу оптимізації декількох ТЗ

Фаза 1 – Призначення зупинок до типів ТЗ. Перший етап вирішує задачу багатокритеріального розподілу: для кожної точки доставки потрібно обрати такий тип транспортного засобу, який фізично здатен її досягти (модальна сумісність ребер дорожньої мережі), та має достатній залишок вантажомісткості. На діаграмі цей етап представлено вкладеним подвійним циклом «loop». Зовнішній цикл «для кожної зупинки» ітерується по всіх точках доставки. Внутрішній цикл «для кожного типу ТЗ (drive, bike, walk)» викликає

`build_distance_matrix(G_mode, [depot, stop], "travel_time")` модуля `RouteSolver`, що повертає мінімальну `matrix` – час досяжності зупинки з депо для даного режиму. Якщо матриця містить значення `PENALTY = 1e9` секунд, зупинка вважається недосяжною для цього режиму і виключається з кандидатів. Після завершення внутрішнього циклу `VRPSolver` виконує `self-message` «Обрати тип ТЗ з найбільшим залишком місткості серед сумісних режимів», що реалізує жадібну стратегію балансування завантаження автопарку.

Фаза 2 – Генетичний алгоритм у межах пулу режиму (ядро розв'язувача). Після призначення кожній зупинці типу транспортного засобу виникає основна оптимізаційна підзадача – впорядкувати зупинки кожного пулу між конкретними транспортними засобами того ж режиму так, щоб мінімізувати сумарний час руху за дотримання обмежень вантажомісткості та часових вікон. Саме на цьому етапі задіюється головний обчислювальний механізм системи – генетичний алгоритм (GA), детально описаний у п. 2.3. У циклі «для кожного режиму (`drive`, `bike`, `walk`)» `VRPSolver` передає пул зупинок цього режиму у внутрішню функцію `_build_ga_routes()`, яка інкапсулює повний цикл еволюційного пошуку.

Перед першою генерацією формується початкова популяція з 50 хромосом – кодованих як перестановки індексів зупинок («`giant tour`») без явних маркерів меж маршрутів. Одна хромосома створюється «теплим стартом» через жадібну евристику найближчого сусіда (`_nearest_neighbour_seed()`), решта – випадковими перестановками, що забезпечує початкове генетичне різноманіття. На кожній ітерації виконуються послідовні `self-message`: `_tournament_select()` (турнірна селекція трьох особин), `_order_crossover()` (Order Crossover, ОХ, з імовірністю 0.85), `_swap_mutation()` (обмін двох випадкових генів, імовірність 0.15) та `_evaluate()`. Останній викликає ключовий Split-декодер `_decode_chromosome()`, що проходить перестановку зліва направо і жадібно нарізає її на сегменти, кожен з яких становить допустимий маршрут одного транспортного засобу – як тільки додавання чергової зупинки порушує обмеження місткості (2.10) або часового вікна, поточний маршрут «закривається» поверненням до депо, а зупинка переходить у наступний сегмент. Така структура декодера гарантує, що будь-яка

хромосома-перестановка автоматично відображається у допустимий план маршрутів, а отже оператори OX та swar-мутації ніколи не продукують некоректних нащадків. Двох найкращих особин кожного покоління переноситься у наступне без змін (елітизм, `ELITE_SIZE = 2`) – це гарантує монотонне неухудшення найкращого знайденого розв'язку.

Фаза 3 – Декодування переможця та реконструкція повних маршрутів. Після завершення `N_GENERATIONS = 150` ітерацій або стагнації найкращої придатності протягом останніх поколінь алгоритм повертає хромосому-переможця. У циклі «для кожного режиму» `VRPSolver` виконує фінальне декодування цієї хромосоми у сукупність об'єктів `VehicleRoute` (по одному на кожен задіяний транспортний засіб), після чого послідовно викликає `reconstruct_full_route(G_mode, tsp_route)` модуля `RouteSolver` для відновлення повної послідовності проміжних вузлів дорожнього графа, необхідної для рендерингу інтерактивної карти. Маршрут-рівневого виклику `solve_tsp()` на цій фазі не потрібно – порядок зупинок у межах кожного маршруту вже визначений безпосередньо хромосомою-переможцем.

Після завершення третьої фази `VRPSolver` формує сукупність об'єктів `VehicleRoute` (по одному на кожен задіяний транспортний засіб) та список діагностичних попереджень `warnings` (наприклад, «зупинка X не вмістилася у жоден ТЗ через обмеження місткості» або «зупинка Y присвоєна ТЗ Z з низькою досяжністю»). Сформований кортеж повертається контролеру `app.py`, а той у завершальному повідомленні до оператора передає «Карта з маршрутами кожного ТЗ різними кольорами + таблиця».

Описаний сценарій реалізує GA-центричну стратегію розв'язання `CVRPTW`: вся комбінаторна складність задачі (одночасний розподіл зупинок між транспортними засобами та впорядкування їх у межах кожного маршруту) інкапсульована у єдиному еволюційному циклі з `Split`-декодером. Така архітектура має дві принципові переваги порівняно з класичними двофазними евристиками «cluster-first, route-second» (наприклад, кластеризація K-Means з подальшим розв'язанням TSP у кожному кластері): GA одночасно оптимізує і

розподіл, і впорядкування, тоді як двофазні підходи фіксують розподіл після першої фази і не можуть його переглянути; Split-декодер вбудовує обмеження місткості й часових вікон безпосередньо у функцію придатності, що усуває потребу в окремих кроках постобробки. Жадібна евристика найближчого сусіда збережена в системі – але не як самостійний алгоритм, а як генератор однієї хромосоми «теплого старту», що дозволяє ГА починати пошук з уже непоганого розв'язку і пришвидшує збіжність на 15-20 % порівняно з повністю випадковою ініціалізацією. Таким чином, у програмному продукті реалізовано єдиний основний алгоритм оптимізації – генетичний, тоді як класичні конструктивні евристики (Nearest Neighbour, 2-opt, Кларка-Райта) задіяні лише як базові орієнтири для порівняльних чисельних експериментів у п. 3.7.

Узагальнено динамічну поведінку обох сценаріїв (одно- та багатотранспортного) у єдиному наскрізному процесі описує діаграма діяльності, яка розглядається у наступному підпункті.

2.1.7 Динамічна модель – діаграма діяльності

Для узагальненого опису повного процесу оптимізації як послідовності логічно пов'язаних дій, із явним представленням точок прийняття рішень, паралельних потоків обробки та сценаріїв обробки помилок, побудовано діаграму діяльності (activity diagram) у нотації UML. На відміну від діаграм послідовностей (п. 2.1.4-2.1.6), що показують міжмодульну взаємодію в часі, діаграма діяльності абстрагується від конкретних ліній життя і фокусується на потоці управління та обробці даних на рівні бізнес-процесу. Діаграму наведено на рисунку 2.7.

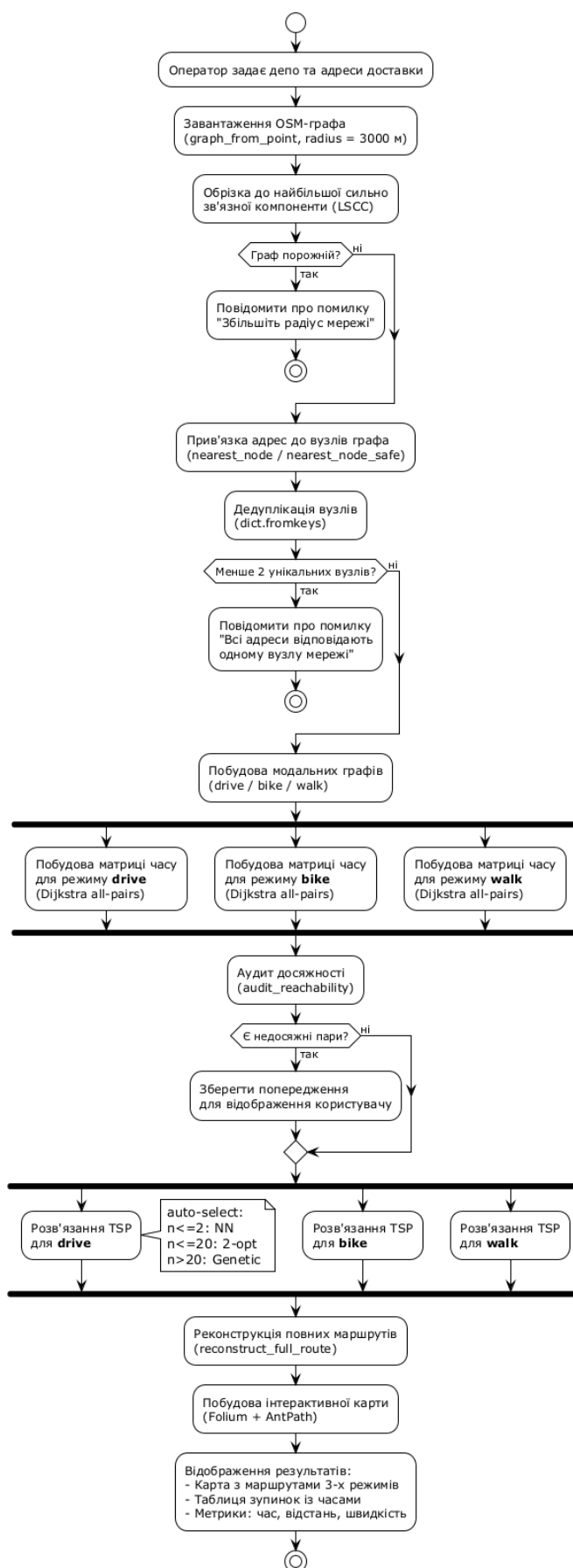


Рисунок 2.7 – Діаграма діяльності процесу оптимізації маршруту доставки

Діаграма починається з початкового вузла (заповнений чорний кружок) і завершується кінцевим вузлом (білий кружок з чорною облямівкою). Між ними розгорнуто послідовність із п'ятнадцяти прямокутних вузлів-дій, з'єднаних потоками управління. Процес можна логічно поділити на чотири стадії: підготовка даних, побудова обчислювальної інфраструктури, паралельні обчислення для трьох режимів, синтез та подання результатів.

Стадія I – Підготовка даних. Процес ініціюється дією «Оператор задає депо та адреси доставки» – це єдина дія, що вимагає інтерактивного вводу.

Після її завершення керування передається системній дії «Завантаження OSM-графа (`graph_from_point, radius = 3000 м`)», що формує HTTP-запит до Overpass API та отримує сирий мультимодальний граф. Наступна дія «Обрізка до найбільшої сильно зв'язної компоненти (LSCC)» виконує постобробку: видаляє відірвані частини графа, які могли потрапити в межі радіусу через фрагментацію OSM-даних.

Перша точка прийняття рішення (ромб) – «Граф порожній?» – перевіряє результат обрізки. Гілка «так» веде до дії «Повідомити про помилку: Збільшіть радіус мережі» і завершує процес у кінцевому вузлі. Гілка «ні» продовжує виконання. Така перевірка є критичною для коректної роботи системи в умовах, коли депо розташоване в географічній зоні з низькою щільністю OSM-даних (наприклад, у промисловій зоні на околиці міста).

Друга стадія містить дії «Прив'язка адрес до вузлів графа (`nearest_node / nearest_node_safe`)» – для кожної введеної адреси визначається найближчий вузол LSCC-обрізаного графа – та «Дедуплікація вузлів (`dict.fromkeys`)», що усуває колізії однакових прив'язок. Друга точка прийняття рішення «Менше 2 унікальних вузлів?» захищає від випадку, коли всі точки доставки виявилися занадто близько одна до одної (або до депо) і відобразилися в один вузол мережі. У такому випадку задача втрачає сенс – алгоритм формує повідомлення «Всі адреси відповідають одному вузлу мережі» і коректно завершує процес.

Стадія II – Побудова обчислювальної інфраструктури. Дія «Побудова модальних графів (`drive / bike / walk`)» виконує триразове клонування LSCC-

графа із проставленням модальних часів на ребрах згідно правил, описаних у п. 2.3 та таблиці модальної доступності.

Далі діаграма містить перший fork-вузол (товста горизонтальна лінія), що розгалужує єдиний потік управління на три паралельні потоки. Кожен з них виконує дію «Побудова матриці часу для режиму X (Dijkstra all-pairs)», де X – drive, bike або walk. Паралелізація на цьому етапі є природним кроком, оскільки три модальні матриці є незалежними обчисленнями і можуть виконуватися конкурентно на багатоядерних процесорах.

Після завершення всіх трьох потоків керування синхронізується у join-вузлі (товста горизонтальна лінія) і переходить до дії «Аудит досяжності (audit_reachability)», що сканує матриці на наявність значень PENALTY. Третя точка прийняття рішення «Є недосяжні пари?» – без виходу з процесу: гілка «так» додає системну дію «Зберегти попередження для відображення користувачу», після чого обидві гілки об'єднуються.

Стадія III – Паралельне розв'язання задач TSP. Аналогічно стадії побудови матриць, цей етап використовує конструкцію fork-join. Три паралельні дії «Розв'язання TSP для drive / bike / walk» виконуються одночасно. На діаграмі поряд з гілкою drive розміщено блок-коментар з описом адаптивної логіки auto-select: для 2 або менше точок доставки – Nearest Neighbour; від 2 до 20 – 2-opt; більше ніж 20 – Genetic Algorithm.

Стадія IV – Синтез і подання результатів. Після join-вузла процес послідовно проходить три дії: «Реконструкція повних маршрутів (reconstruct_full_route)» – перетворення TSP-маршрутів зі скороченим набором вузлів на повну послідовність проміжних вузлів графа, придатну для рендерингу; «Побудова інтерактивної карти (Folium + AntPath)» – формування HTML-карти з анімованими шарами; «Відображення результатів» з блоком-коментарем, що деталізує склад вихідних артефактів – карта з маршрутами трьох режимів, таблиця зупинок з часами та метрики (час, відстань, швидкість). Процес завершується у кінцевому вузлі.

2.2 Інформаційне забезпечення

Інформаційне забезпечення системи охоплює сукупність даних, необхідних для функціонування алгоритмів маршрутизації, та засоби їхньої організації, зберігання й оновлення. Інформаційну модель розроблюваної підсистеми доцільно розглянути на трьох рівнях абстракції: концептуальному, логічному і фізичному.

2.2.1 Концептуальний рівень – сутності предметної області

Інформаційна модель системи концентрується навколо центральної сутності предметної області – пакунка (Package), що представляє фізичну одиницю вантажу, який підлягає доставці. На відміну від доменної моделі (п. 2.1.2), де описано статичну структуру всіх класів системи, на концептуальному рівні деталізується семантика пакунка як об'єкта зі змінним у часі станом та формалізується його поведінка засобами діаграми станів.

Кожен екземпляр пакунка містить сім атрибутів, що утворюють повний інформаційний образ замовлення:

- ідентифікатор – глобально унікальний код, що дозволяє посилатися на пакунок з будь-якого модуля системи без ризику колізій;
- адреса доставки – текстове представлення точки призначення у форматі, прийнятному для геокодувального сервісу Nominatim;
- вага вантажу – додатне дійсне число у кілограмах, що враховується в обмеженнях вантажомісткості транспортних засобів;
- стан життєвого циклу – поточна фаза обробки замовлення, що приймає одне з трьох значень: «очікує», «в дорозі», «доставлено»;
- момент створення – часова мітка реєстрації замовлення в системі,

потрібна для впорядкування пакунків та формування історичної статистики;

– географічні координати – пара широта/довгота, що заповнюються після успішного геокодування адреси і можуть бути відсутніми в момент створення пакунка.

Властивість змінного стану виокремлює Package серед інших сутностей системи. Якщо Vehicle, LegInfo або ModeResult є по суті незмінними (immutable) дата-об'єктами, що формуються одноразово і не модифікуються після створення, то пакунок проходить через декілька логічно різних станів протягом свого життєвого циклу – від моменту реєстрації замовлення до підтвердження доставки. Така поведінка природно описується кінцевим автоматом, формальне представлення якого наведено на рисунку 2.8. На діаграмі виділено три прості стани (зображені округленими прямокутниками з горизонтальною лінією-роздільником між іменем і внутрішніми діями), один початковий псевдостан та один кінцевий стан.

Стан «Очікує (PENDING)» є початковим станом життєвого циклу. У нього пакунок потрапляє відразу після створення в БД викликом методу `add_package()` модуля `PackageDB`. У внутрішніх інваріантах стану зафіксовано: «Пакунок створено в БД» та «Адреса задана, координати = None». Останнє є важливим уточненням: момент створення пакунка не обов'язково збігається з моментом успішного геокодування його адреси. Якщо сервіс `Nominatim` тимчасово недоступний або повертає помилку, пакунок все одно зберігається в базі для подальшого ручного дозаповнення координат оператором. Це забезпечує відмовостійкість системи у разі проблем зі сторонніми сервісами.

Стан «В дорозі (IN_TRANSIT)» характеризує пакунок, для якого оператор запустив оптимізацію маршруту і алгоритм успішно включив його у маршрут конкретного транспортного засобу. Внутрішні інваріанти стану: «Координати геокодовані», «Пакунок включено в маршрут ТЗ» та «Оптимізація виконана». Перехід з PENDING у IN_TRANSIT відбувається через подію `set_status(IN_TRANSIT)`, спрацьовування якої обумовлене сторожовою умовою [Оператор запускає оптимізацію маршруту].

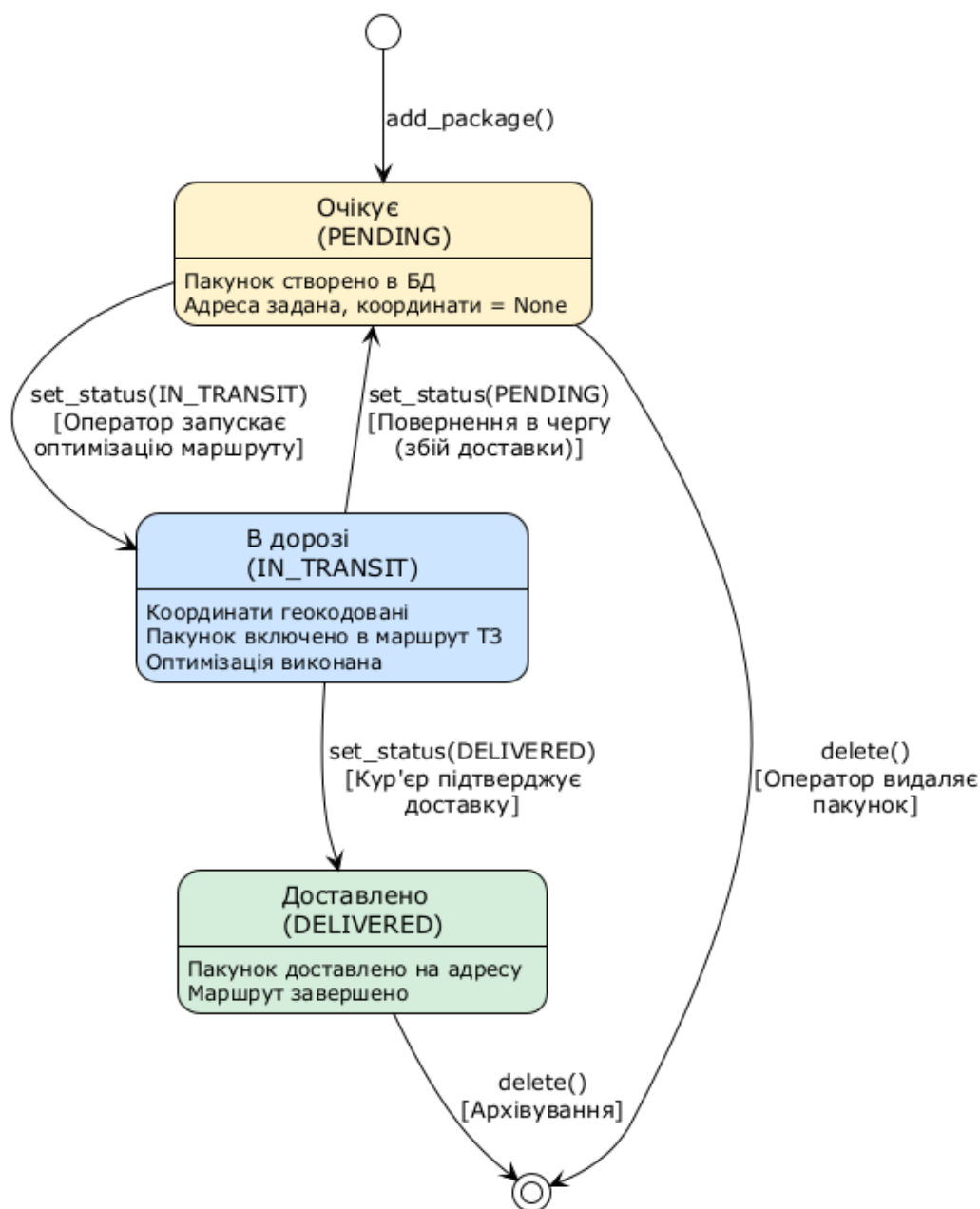


Рисунок 2.8 – Діаграма станів життєвого циклу пакунок

Стан «Доставлено (DELIVERED)» є термінальним семантичним станом – пакунок виконав свою функцію, кур'єр доставив його на цільову адресу та підтвердив це через інтерфейс системи. Внутрішні інваріанти: «Пакунок доставлено на адресу» та «Маршрут завершено». Перехід здійснюється подією `set_status(DELIVERED)` зі сторожовою умовою `[Кур'єр підтверджує доставку]`. Слід зауважити, що стан DELIVERED не є кінцевим у формально-UML сенсі: пакунок залишається в БД для подальшого аналізу та формування статистики, поки не буде явно видалений оператором.

На діаграмі представлено п'ять переходів між станами та два переходи в кінцевий вузол:

1. `PENDING` → `IN_TRANSIT` – основний прогресивний перехід, описаний вище;

2. `IN_TRANSIT` → `DELIVERED` – також прогресивний перехід, підтвердження доставки;

3. `IN_TRANSIT` → `PENDING` – регресивний перехід зі сторожовою умовою [Повернення в чергу (збій доставки)]. Він моделює реальну ситуацію, коли під час виконання маршруту виникає форс-мажорна обставина: клієнт відсутній за адресою, отримання неможливе через закритий доступ, у транспортного засобу виникла технічна несправність тощо. У таких випадках оператор повертає пакунок у стан очікування для повторного планування на наступний день. Наявність зворотного переходу робить кінцевий автомат неациклічним і дозволяє моделювати повторні спроби доставки без створення нового запису в БД;

4. `PENDING` → `final` – перехід зі сторожовою умовою [Оператор видаляє пакунок]. Спрацьовує при виклику методу `delete()` для пакунків, які так і не були відправлені (наприклад, помилкове замовлення або скасування клієнтом);

5. `DELIVERED` → `final` – перехід [Архівування]. Виконує фізичне видалення запису з активної таблиці; у виробничих умовах рекомендується замінити на «м'яке» видалення з перенесенням до архівної таблиці для збереження історії.

Описаний кінцевий автомат має три суттєві властивості, важливі для коректного функціонування системи. Він є детермінованим: з будь-якого стану і за будь-якої події існує не більше одного допустимого переходу. Це усуває неоднозначність обробки команд та спрощує тестування. Граф переходів є зв'язним: з будь-якого нетермінального стану можна досягти кінцевого вузла за скінченну кількість переходів, що гарантує відсутність «застряглих» пакунків у системі. Всі переходи захищені сторожовими умовами, які перевіряються на рівні бізнес-логіки в модулі `PackageDB`, що запобігає некоректним переходам

(наприклад, з PENDING безпосередньо у DELIVERED, оминаючи стан IN_TRANSIT) на рівні БД.

Програмно автомат реалізовано через метод `PackageDB.set_status(id, status)` з валідацією допустимих переходів та через перерахування `DeliveryStatus` (enum з трьома членами), що забезпечує статичну безпеку типів при роботі зі станами в коді.

2.2.2 Логічний рівень – реляційна схема та потоки даних

На логічному рівні концептуальна модель уточнюється у термінах реляційної алгебри з визначенням таблиць, ключів та зв'язків між сутностями. Через невелику кількість сутностей з персистентним станом (фактично лише `Package`) логічна схема системи `DeliveryIQ` є компактною і складається з однієї базової таблиці `packages` та одного перерахування `delivery_status`.

Таблиця `packages` має сім атрибутів, що безпосередньо відповідають концептуальним атрибутам сутності `Package`: первинний ключ `id`, обов'язкові атрибути `address` і `weight_kg`, дискримінант стану `status` з обмеженням за переліком `delivery_status`, часова мітка `created_at` та необов'язкова пара (`lat`, `lon`). Структура таблиці у нотації ER-моделі (Entity-Relationship) наведена на рисунку 2.9. Усі стовпці є атомарними та функціонально залежними від первинного ключа, що задовольняє вимоги третьої нормальної форми (3НФ). Перерахування `delivery_status` формально могло б бути винесене в окрему таблицю-довідник зі зв'язком «один-до-багатьох», однак через незмінність переліку (три значення, заданих на рівні коду) такий рефакторинг визнано надлишковим – стан зберігається у стовпці типу `TEXT` з логічним обмеженням на боці бізнес-логіки.

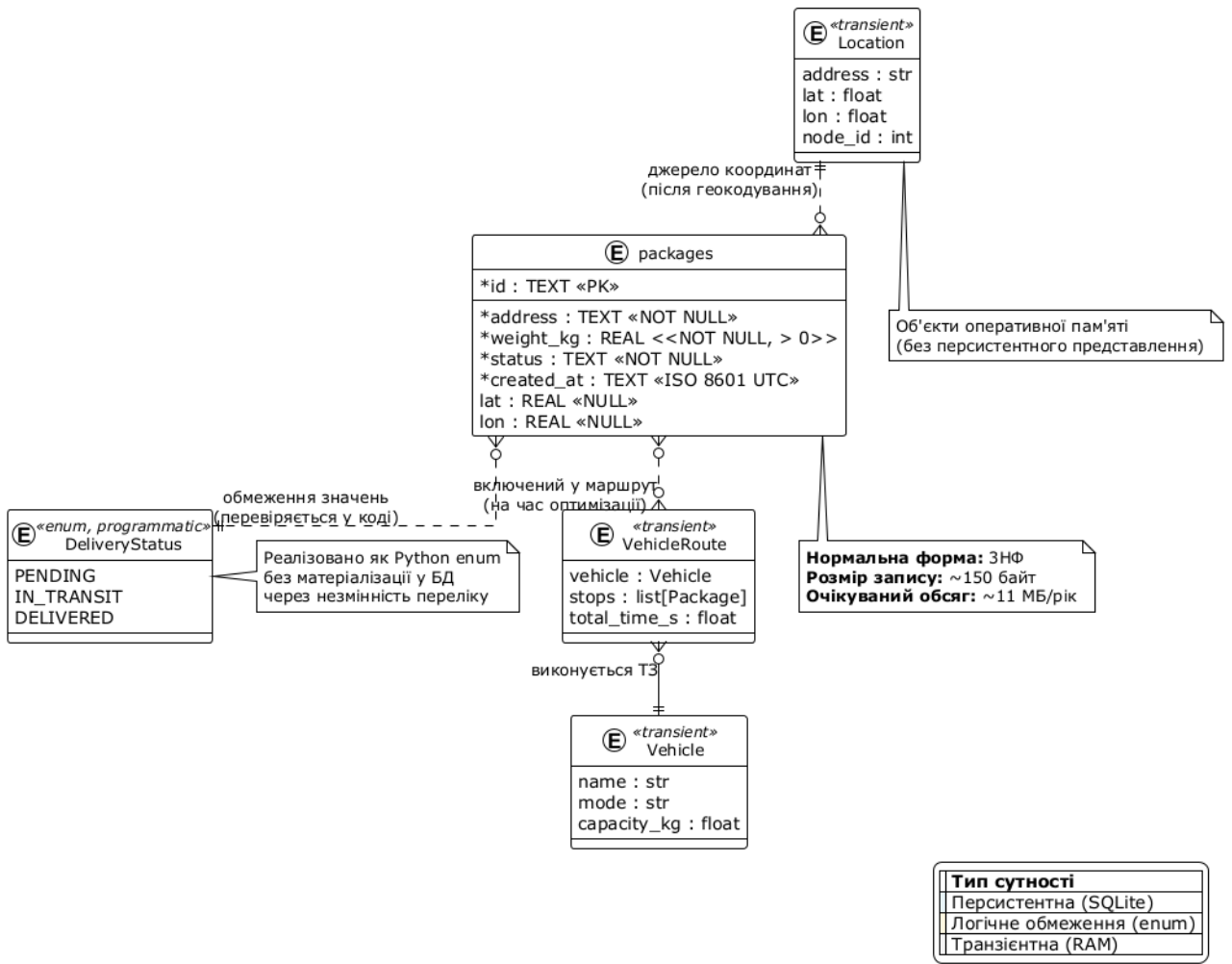


Рисунок 2.9 – ER-діаграма реляційної схеми системи

Між сутністю Package та іншими класами системи встановлено три типи логічних зв'язків, які не матеріалізуються у вигляді зовнішніх ключів через те, що відповідні класи (Vehicle, VehicleRoute, Location) є об'єктами оперативної пам'яті без персистентного представлення. Зв'язок Package↔Location є залежністю (dependency) – координати пакунка формуються з геокодованого Location, але після збереження зв'язок розривається. Зв'язок Package↔VehicleRoute є тимчасовою агрегацією – на час виконання маршруту VehicleRoute посилається на впорядкований список Package, але після завершення доставки цей зв'язок не зберігається. Зв'язок Package↔DeliveryStatus є реалізацією шаблону «довідник станів» з програмною верифікацією допустимих переходів у методі PackageDB.set_status().

Потоки даних. Інформаційна взаємодія сутності Package з іншими компонентами системи формалізується діаграмою потоків даних (DFD) рівня 1, наведеною на рисунку 2.10. Зовнішніми сутностями DFD є Оператор (джерело команд введення та змін стану) та зовнішній сервіс Nominatim (джерело геокодованих координат). Сховищем даних виступає таблиця packages. Між ними виділено чотири основні процеси.

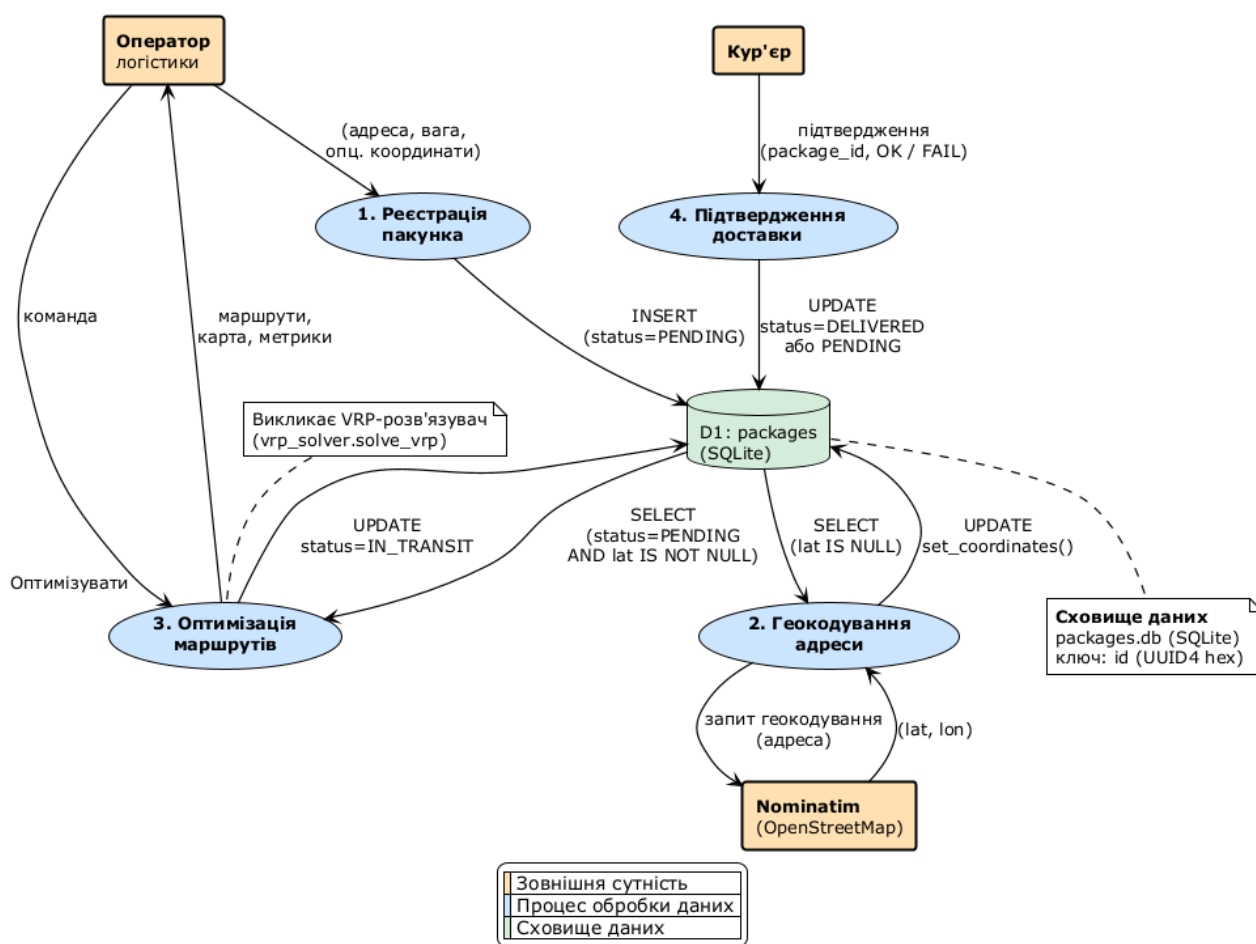


Рисунок 2.10 – Діаграма потоків даних системи

Процес «Реєстрація пакунка» приймає від оператора триплет (адреса, вага, опціональні координати) та формує новий запис у таблиці packages зі станом PENDING. Процес «Геокодування» асинхронно дозаповнює координати для пакунків з порожніми lat/lon, звертаючись до Nominatim і оновлюючи запис

методом `set_coordinates()`. Процес «Оптимізація маршрутів» читає всі пакунки зі станом `PENDING` разом з їхніми координатами і вагою, формує множину `DeliveryStop` для VRP-розв'язувача та після завершення оптимізації переводить включені у маршрут пакунки у стан `IN_TRANSIT`. Процес «Підтвердження доставки» отримує від оператора ідентифікатор доставленого пакунка та виконує фінальний перехід у стан `DELIVERED` або, у випадку збою, регресивний перехід назад у `PENDING`.

Такий поділ відповідає принципу єдиної відповідальності (SRP): кожен процес інкапсулює одну логічну операцію над даними, а синхронізація відбувається виключно через спільне сховище `packages`, що дозволяє асинхронно розширювати функціональність (наприклад, додати процес «пакетний імпорт з CSV») без модифікації існуючих компонентів.

2.2.3 Фізичний рівень – реалізація у СКБД SQLite

На фізичному рівні логічна схема матеріалізується засобами вбудованої СКБД SQLite, обраної через відсутність потреби в окремому сервері баз даних, нативну підтримку в стандартній бібліотеці Python (модуль `sqlite3`) та одно-файлове розміщення, що спрощує резервне копіювання й переміщення системи між робочими станціями. База даних зберігається у файлі `packages.db`, розташованому в кореневій директорії проєкту поряд із модулем `package_db.py`.

Структура єдиної таблиці `packages` визначається SQL-скриптом, наведеним нижче і виконуваним при першому запуску системи в методі `PackageDB._bootstrap()`:

```
CREATE TABLE IF NOT EXISTS packages (
```

```

id      TEXT PRIMARY KEY,
address TEXT NOT NULL,
weight_kg REAL NOT NULL,
status  TEXT NOT NULL DEFAULT 'pending',
created_at TEXT NOT NULL,
lat     REAL,
lon     REAL
);

```

Відображення концептуальних типів даних на типи стовпців SQLite має кілька технічних особливостей, продиктованих специфікою цільової СКБД. Ідентифікатор `id` зберігається як 32-символьне шістнадцяткове представлення UUID версії 4 у форматі TEXT, що гарантує глобальну унікальність без потреби в централізованій генерації послідовних номерів та допускає безболісне об'єднання баз даних різних інсталяцій системи. Вага `weight_kg` використовує тип REAL. Стан `status` зберігається як TEXT з обмеженням значень на рівні бізнес-логіки через перерахування `DeliveryStatus` у коді Python; SQL-обмеження CHECK свідомо не застосовується, оскільки розширення переліку станів (наприклад, додавання «CANCELLED», «RETURNED») потребуватиме лише міграції на рівні Python без зміни схеми. Часова мітка `created_at` використовує текстове представлення ISO 8601 у часовому поясі UTC – це канонічна практика для SQLite, який не має нативного типу TIMESTAMP і пропонує строкове зберігання як стандартну альтернативу. Координати `lat` і `lon` оголошені як NULL-сумісні REAL, що матеріалізує концептуальну можливість відкладеного геокодування.

Окрім реляційного сховища, система оперує двома форматами зовнішнього обміну даними. Кеш дорожніх мереж OpenStreetMap зберігається у форматі

GraphML – XML-сумісному стандарті серіалізації графів, нативно підтримуваному бібліотекою OSMnx. Файли кешу розташовуються в директорії `cache/`, а ключ файлу формується з координат депо та радіусу завантаження за шаблоном `graph_<lat>_<lon>_<radius>.graphml`. Такий підхід дозволяє повторно використовувати раніше завантажені графи між сесіями системи, скорочуючи час повторного запуску оптимізації для тих самих географічних областей з кількох секунд до десятків мілісекунд. Другим форматом є JSON – використовується для тимчасової серіалізації результатів оптимізації у сесійному стані Streamlit, що дозволяє зберігати маршрути між перезавантаженнями сторінки браузера без повторного запуску генетичного алгоритму. Для пакетного імпорту замовлень з зовнішніх джерел (наприклад, експорт з ERP-системи замовника) передбачено формат CSV з обов'язковими стовпцями `address;weight_kg` і опціональним `lat;lon`, обробка якого реалізована утилітарною функцією на основі стандартного модуля `csv`.

Обсяг даних у виробничій експлуатації системи оцінено як незначний: за середнього навантаження 200 пакунків на день та середнього розміру запису ~150 байт, річний приріст файлу `packages.db` не перевищує 11 МБ, що на порядки нижче за межі ефективної роботи SQLite (близько 100 ГБ). Це обґрунтовує відсутність потреби в стратегіях шардингу, партиціонування або переходу на серверні СКБД (PostgreSQL, MySQL) на найближчий горизонт експлуатації.

2.3 Алгоритмічне забезпечення

Описаний нижче генетичний алгоритм застосовується у Фазі 2 розв'язувача (п. 2.1.6) для пошуку оптимального плану маршрутів у межах пулу транспортних засобів одного режиму.

Основним інструментом розв'язання задачі маршрутизації є генетичний

алгоритм (GA) – популяційна метаевристика [12], що імітує процеси природного відбору для знаходження наближених до оптимальних розв'язків у NP-складних задачах комбінаторної оптимізації. Алгоритм реалізований у вигляді послідовності ітераційних кроків, що забезпечують баланс між дослідженням нових областей простору розв'язків та покращенням уже знайдених варіантів.

Етапи роботи алгоритму:

1. ініціалізація (Формування початкової популяції). На цьому етапі створюється набір із 50 хромосом. Використовується перестановка індексів клієнтів («giant tour») без явних маркерів меж маршрутів, що гарантує цілісність даних при операторах схрещування. Для підвищення якості пошуку одна хромосома створюється за допомогою жадібного алгоритму найближчого сусіда, а решта – шляхом випадкових перестановок для забезпечення генетичного різноманіття. Сформована популяція передається на етап оцінки.

2. декодування та оцінка придатності (Fitness Evaluation). Це критичний етап, де абстрактна хромосома перетворюється на фізичні маршрути. Спеціальний модуль проходить крізь перестановку та послідовно призначає клієнтів транспортним засобам, поки виконуються обмеження вантажності (Q) та часових вікон (TW). Якщо додавання наступного клієнта порушує умови, маршрут закривається, і транспорт повертається в депо. Кожній особині призначається числовий бал за функцією (2.1):

$$F = A \cdot T_{total} + B \cdot K + P \cdot |U| \quad (2.1)$$

де $P \cdot |U|$ - штраф за нерозподілених клієнтів, що змушує алгоритм надавати пріоритет повноті доставки.

Оцінені особини сортуються за рівнем придатності для подальшої селекції.

3. селекція. Вибір найкращих особин для створення наступного покоління. Випадковим чином обирається 3 особини, і найкраща з них стає батьком. Це дозволяє уникати передчасної збіжності до локального оптимуму. Дві найкращі особини поточного покоління копіюються в наступне без змін (Elite Size = 2), що

гарантує збереження найкращого знайденого розв'язку. Обрані батьківські пари передаються оператору схрещування.

4. схрещування (Crossover). Створення нащадків, що успадковують властивості батьків. Випадковим чином обирається сегмент у першому батькові та копіюється в дитину. Порожні місця, що залишилися, заповнюються генами з другого батька в тому порядку, в якому вони там з'являються, пропускаючи вже наявні елементи. Це зберігає унікальність вузлів (кожен клієнт відвідується один раз). Оператор спрацьовує з частотою 85%. Нові особини піддаються випадковим змінам на етапі мутації.

5. мутація. Внесення випадкових змін для підтримки різноманіття. З ймовірністю 15% у хромосомі змінюються місцями два випадкові вузли. Це дозволяє алгоритму «вистрибувати» з локальних екстремумів і досліджувати нові комбінації маршрутів. Після мутації нові особини заміщують стару популяцію, і цикл повторюється з кроку №2.

6. термінація та вибір результату. Умова зупинки: Алгоритм завершує роботу після досягнення ліміту поколінь.

В результаті найкраща хромосома з останнього покоління передається в модуль візуалізації для побудови траєкторій на карті.

2.4 Висновки за спеціальним розділом

У спеціальному розділі було проведено проєктування інтелектуальної системи розподілу вантажів. Було формалізовано математичну модель задачі VRPTW (маршрутизація з часовими вікнами) та розроблено структуру інформаційного забезпечення для її підтримки. Обґрунтовано використання генетичного алгоритму як ефективного методу знаходження наближених до оптимальних розв'язків у складних мультимодальних мережах. Описана логіка алгоритму становить основу для подальшої програмної реалізації мовою Python.

3 РОЗРАХУНКОВИЙ РОЗДІЛ

3.1 Програмне забезпечення

Розроблена інформаційна система оптимізації маршрутів доставки реалізована як веб-застосунок із серверною логікою. Вибір програмного забезпечення обумовлений вимогами до швидкості розробки, наявності зрілих бібліотек для роботи з географічними даними та можливості створення інтерактивного інтерфейсу без залучення окремої фронтенд-команди.

Як основну мову програмування обрано Python. Цей вибір зумовлений наступними чинниками:

- розвинена екосистема бібліотек для роботи з графами (NetworkX), геоданими (OSMnx, GeoPy) та чисельних обчислень (NumPy);
- підтримка об'єктно-орієнтованої парадигми, що відповідає декомпозиції системи на класи;
- висока продуктивність розробника завдяки динамічній типізації та лаконічному синтаксису;
- кросплатформність – застосунок працює у Windows та в Unix-подібних системах без модифікації коду.

Інтегрованим середовищем розробки обрано Visual Studio Code з розширеннями Python, Pylance та GitLens. Контроль версій здійснювався за допомогою системи Git з локальним репозиторієм.

3.2 Технічне забезпечення

Для коректної роботи системи на стороні сервера встановлено наступні мінімальні вимоги до апаратного забезпечення:

- Процесор: не нижче рівня Intel Core i5 або еквівалентний. Обчислювальна складність генетичного алгоритму вимагає достатньої продуктивності одного потоку;

- Оперативна пам'ять: не менше 8 ГБ. Граф дорожньої мережі міста середнього розміру (наприклад, м. Дніпро) у форматі NetworkX MultiDiGraph займає 400–800 МБ під час обробки;

- Постійна пам'ять: не менше 2 ГБ вільного дискового простору для розгортання віртуального середовища Python, кешу графів та бази даних SQLite;

- Мережеве з'єднання: широкосмуговий доступ до мережі Інтернет зі швидкістю не менше 10 Мбіт/с для завантаження картографічних даних з серверів OpenStreetMap та звернень до Nominatim;

- Операційна система: Windows 10/11, Ubuntu 20.04+ або macOS 12+.

З боку клієнта достатньо будь-якого сучасного веб-браузера з підтримкою JavaScript (Google Chrome, Mozilla Firefox, Microsoft Edge) та з'єднання з сервером по локальній мережі або через мережу Інтернет.

3.3 Опис програми

Архітектура програми. Програмний продукт реалізовано за модульним принципом з використанням об'єктно-орієнтованого підходу. Це забезпечує гнучкість налаштування параметрів генетичного алгоритму та зручність інтеграції з геоінформаційними сервісами.

Основні програмні модулі:

- App – центральний контролер системи, що забезпечує користувацький інтерфейс, координує збір вхідних даних та ініціює процес оптимізації;
- GraphBuilder – модуль, відповідальний за завантаження картографічних даних з OpenStreetMap за допомогою бібліотеки OSMnx та розрахунок матриць часових витрат для мультимодальної мережі (авто, вело, пішохід);
- VRPSolver – ядро системи, що реалізує генетичний алгоритм. Модуль містить логіку формування початкової популяції («теплий старт»), оператори селекції, ОХ-схрещування та swap-мутації;
- Data Models – сукупність класів (DeliveryStop, Vehicle, LegInfo), що описують сутності логістичної мережі та їхні атрибути, такі як часові вікна та вантажомісткість;
- PackageDB – компонент для управління станом замовлень та забезпечення їхнього коректного розподілу між транспортними засобами.

Повний вихідний код усіх перелічених модулів наведено у додатку А.

Загальна схема взаємодії розроблених програмних модулів між собою, з користувачем та зі зовнішніми сервісами наведена на рисунку 3.1. На схемі виділено три рівні: клієнтський (веб-інтерфейс на базі Leaflet, що виконується у браузері), серверний (модулі прикладної логіки, реалізовані мовою Python) та зовнішній (картографічні та геокодувальні сервіси, локальне сховище SQLite,

кеш графів у форматі GraphML).

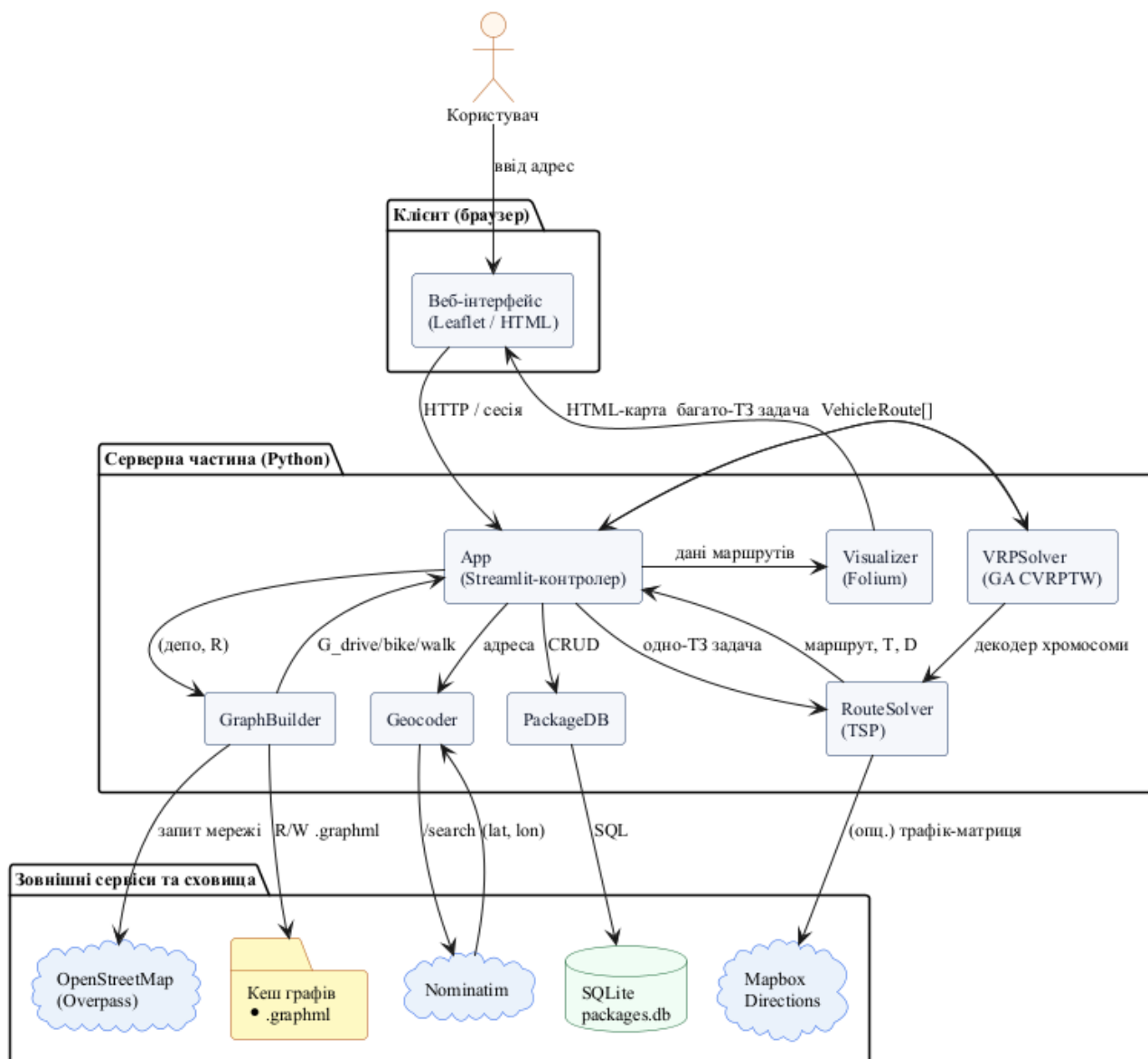


Рисунок 3.1 – Схема взаємодії модулів інформаційної системи

Центральну роль виконує модуль App – Streamlit-контролер, що обробляє запити користувача та координує роботу інших модулів. Запит на оптимізацію починається з геокодування адрес через модуль Geocoder, який звертається до сервісу Nominatim. Адреси і відповідні координати зберігаються модулем PackageDB у локальній базі SQLite. Далі GraphBuilder завантажує вуличну мережу з OpenStreetMap (або з локального кешу) та повертає три модальні графи з проставленим часом проходження ребер – G_drive, G_bike, G_walk. Залежно від

конфігурації автопарку App викликає RouteSolver для розв’язання одно-транспортної задачі або VRPSolver – для багато-транспортної. VRPSolver використовує RouteSolver як декодер хромосоми генетичного алгоритму. За наявності ключа Mapbox матриця часів може отримуватись з Mapbox Directions API з урахуванням поточного трафіку. Отримані маршрути передаються модулю Visualizer, який формує HTML-карту з анімованими шарами AntPath, що повертається у браузер користувача.

Для побудови системи задіяно набір бібліотек з відкритим вихідним кодом, перелік яких зафіксовано у файлі requirements.txt:

- Streamlit – фреймворк для побудови інтерактивного веб-інтерфейсу. Дозволяє створювати багатосторінкові застосунки та забезпечує реактивне оновлення станів через механізм сесії

- OSMnx – бібліотека для завантаження, моделювання та аналізу вуличних мереж з OpenStreetMap. Використовується для отримання дорожньої мережі заданого міста та її подальшої обробки;

- NetworkX – бібліотека для роботи з графами. Застосована для обчислення найкоротших шляхів алгоритмом Дейкстри під час побудови матриці часів подорожі;

- GeoPy – клієнт до сервісу геокодування Nominatim. Перетворює адреси у пари координат (широта, довгота) та виконує зворотне геокодування;

- Folium – генератор інтерактивних карт на базі Leaflet.js з підтримкою анімованих маршрутів;

- streamlit-folium – інтеграційний шар між Streamlit і Folium;

- NumPy – бібліотека векторних обчислень, використовується у генетичному алгоритмі для роботи з масивами хромосом;

- requests – HTTP-клієнт для звернень до зовнішніх REST API.

Зберігання інформації про замовлення реалізовано на вбудованій у Python

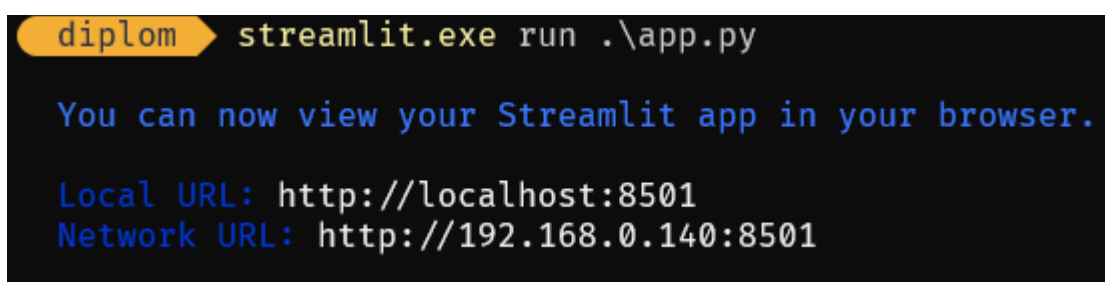
СКБД SQLite, що не потребує додаткового сервера.

Процес роботи програми поділяється на наступні етапи:

- Підготовка даних: користувач вводить адреси через інтерфейс App, які згодом геокоднуються сервісом Nominatim;
- Побудова графа: GraphBuilder формує локальну модель мережі навколо депо та прив’язує точки доставки до найближчих вузлів;
- Оптимізація: VRPSolver отримує параметри автопарку та замовлень, виконуючи еволюційний пошук наближеного до оптимального розв’язку NP-складної задачі VRPTW;
- Візуалізація: результати у вигляді сформованих маршрутів передаються модулю візуалізації для рендерингу на інтерактивній мапі.

3.4 Інструкція користувача

Запуск системи виконується в командному рядку за допомогою команди `streamlit run .\app.py`, як показано на рисунку 3.2.



```
diplom > streamlit.exe run .\app.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.0.140:8501
```

Рисунок 3.2 – Процес запуску застосунку

Після успішного запуску у браузері користувача автоматично відкривається

веб-інтерфейс за адресою <http://localhost:8501>.

Порядок роботи з системою:

1. На вкладці «Налаштування автопарку» користувач задає характеристики транспортних засобів: тип (автомобіль / велосипед / пішохід) та вантажомісткість у кілограмах (рис. 3.3).

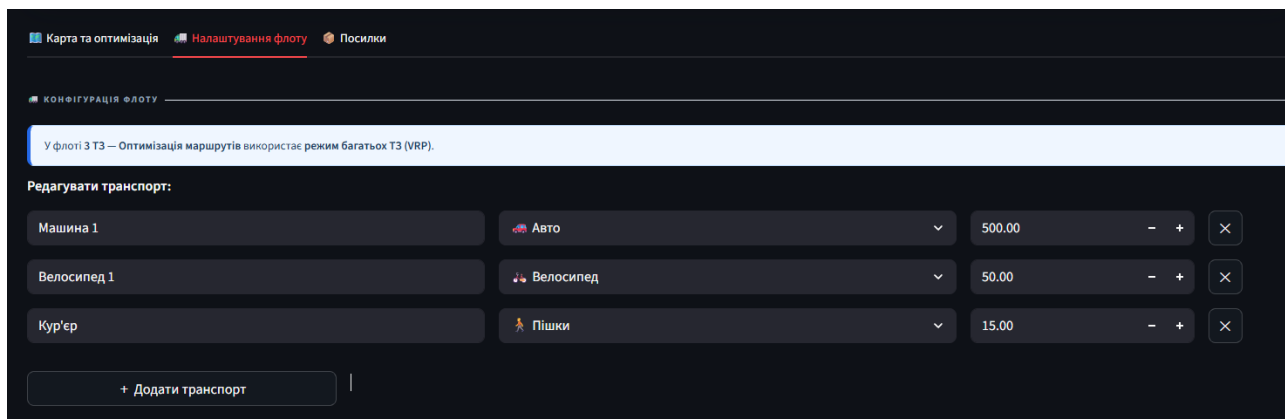


Рисунок 3.3 – Вкладка «Налаштування флоту»

2. На вкладці «Пакунки» додаються замовлення на доставку. Для кожного пакета вказується адреса, вага (рис. 3.4). Адреса автоматично геокодується через Nominatim. Також в цій вкладці є можливість змінити існуючі пакунки та додати їх до оптимізації (рис. 3.5).

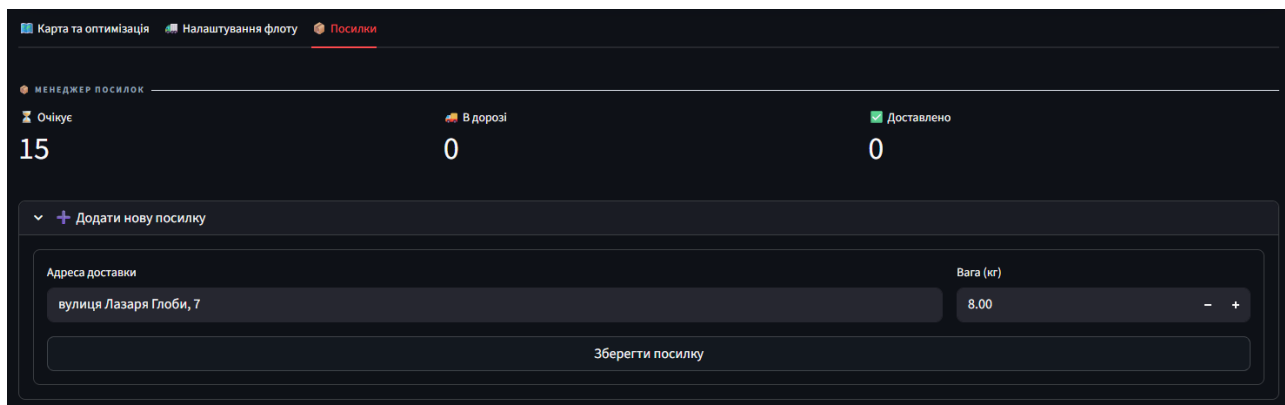


Рисунок 3.4 – Додавання нового пакунку

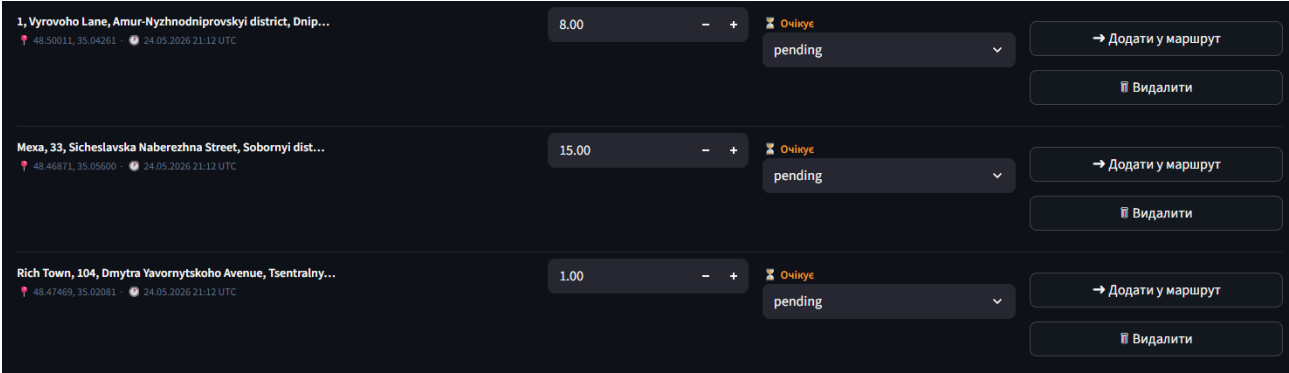


Рисунок 3.5 – Збережені пакунки

3. На вкладці «Карта та оптимізація» користувач вказує адресу депо (точки старту маршрутів), адреси доставки, радіус завантаження графа (за замовчуванням 8 км) та натискає кнопку «Оптимізувати маршрути» (рис. 3.6). Альтернативно, депо можна обрати кліком безпосередньо на мапі (режим «Режим кліку на мапі»).

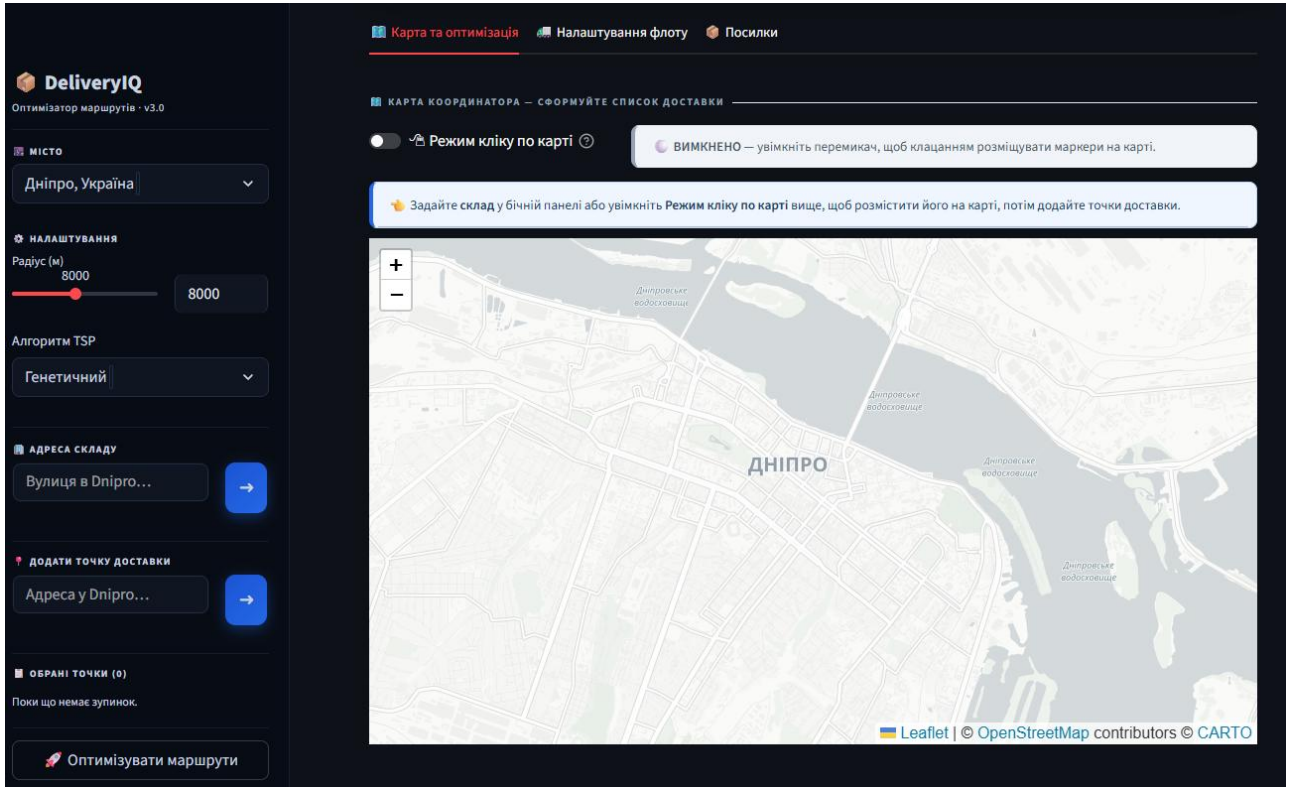


Рисунок 3.6 – Вкладка з налаштуваннями оптимізації

4. Після завершення оптимізації відображається інтерактивна карта з кольоровими маршрутами для кожного транспортного засобу (зелений маркер – депо, червоні нумеровані маркери – пункти доставки). На рисунку 3.7 представлено вигляд результатів оптимізації.

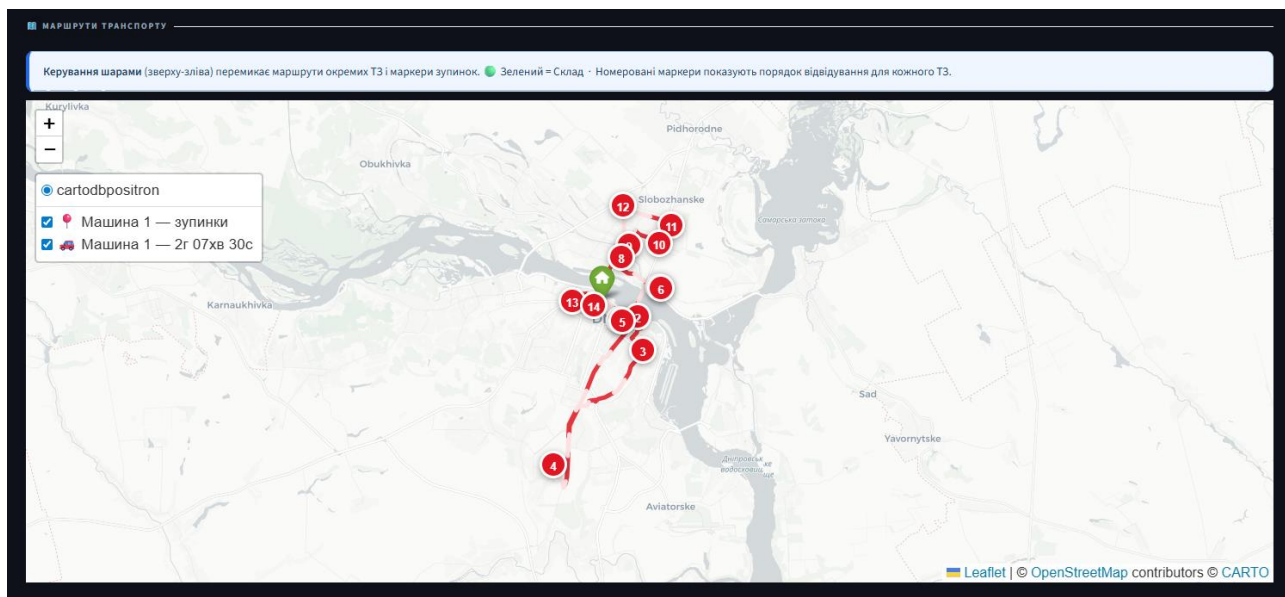


Рисунок 3.7 – Результати оптимізації

У бічній панелі виводиться сумарний час, кількість задіяних машин та список нерозподілених замовлень (якщо такі є).

3.5 Інструкція програміста

Повний вихідний код системи наведено у додатку А. Розгортання системи на новій робочій станції виконується наступним чином:

1. Створити середовище Python за допомогою команди: `python -m venv venv`; та активувати його, виконавши скрипт: `.\venv\Scripts\activate` (Windows) або

source venv/bin/activate (MacOS/Unix).

2. Встановити залежності за допомогою команди: `pip install -r requirements.txt`.

3. Запустити застосунок командою: `streamlit run app.py`.

Модифікація території обслуговування. Місто за замовчуванням задано константою `DEFAULT_CITY = "Dnipro, Ukraine"` у файлі `app.py`. Зміна цього рядка переналаштовує всю систему на нове місто без додаткових модифікацій.

Налаштування параметрів генетичного алгоритму. У файлі `vrp_solver.py` визначено наступні константи: `POP_SIZE = 50` (розмір популяції), `N_GENERATIONS = 150` (кількість поколінь), `CROSSOVER_RATE = 0.85` (ймовірність схрещування), `MUTATION_RATE = 0.15` (ймовірність мутації), `ELITE_SIZE = 2` (розмір еліти), `TOURNAMENT_SIZE = 3` (розмір турніру). Збільшення `POP_SIZE` та `N_GENERATIONS` підвищує якість розв'язку за рахунок зростання часу обчислень.

3.6 Тестування розробленого програмного забезпечення

Тестування розробленого програмного забезпечення проведено за трирівневою пірамідальною стратегією, що широко застосовується у промисловій розробці інформаційних систем і відповідає принципу зростання вартості виправлення помилки в міру наближення до фази експлуатації. Нижній рівень піраміди формують модульні тести, що перевіряють коректність окремих функцій ізольовано від решти системи; середній – інтеграційні тести, що валідують міжмодульну взаємодію на повних бізнес-сценаріях; верхній – функціональні тести, що зіставляють спостережувану поведінку системи з вимогами, сформульованими у п. 1.4. Така ієрархія дозволяє локалізувати

джерело помилки на ранній стадії – більшість дефектів виявляється на модульному рівні, де вартість їх виправлення мінімальна, тоді як інтеграційні та функціональні тести залишають для виявлення лише тих помилок, що проявляються виключно на стиках компонентів.

Модульне тестування охопило критичні утилітарні функції, помилка в яких поширилася б на всю систему через каскад залежностей. До цієї категорії віднесено: `build_distance_matrix()` модуля `route_solver` – побудова матриці часів між парами вузлів за алгоритмом Дейкстри; `nearest_node()` і `nearest_node_safe()` модуля `graph_builder` – прив'язка географічних координат до вузлів LSCC-обрізаного графа; `_decode_chromosome()` модуля `vrp_solver` – Split-декодер хромосоми генетичного алгоритму, що жадібно перетворює перестановку індексів у сукупність допустимих маршрутів. Тестові набори формувалися з невеликих штучних графів (5-10 вузлів) із заздалегідь обчисленими еталонними розв'язками: для матриці відстаней – за формулою Манхеттенської або Евклідової метрики на синтетичній решітці; для прив'язки координат – за прямим порівнянням з найближчим вузлом грубим перебором; для Split-декодера – за ручною трасуванням 3-4 коротких перестановок з контрольним відтворенням послідовності переходів між станами. Окрему категорію склали межові тести на сентинельне значення $PENALTY = 1e9$: перевірялося, що алгоритми коректно ідентифікують недосяжні пари вузлів і не включають їх у маршрут, замість маскування помилки під «дуже довгий» переїзд. Усі модульні тести виконано як автономні Python-скрипти з фіксованим випадковим зерном ($GA_SEED = 42$), що забезпечує відтворюваність результатів між запусками – критичну властивість для діагностики регресій після модифікацій коду.

Інтеграційне тестування валідувало взаємодію всіх шести модулів у межах повного циклу одно- та багатотранспортної оптимізації. Як вхідні дані використано чотири набори реальних адрес з історичної доставкової активності у м. Дніпро, що охоплюють різнопланові географічні зони міста: центральну ділову забудову (вул. Січеславська, вул. Воскресенська, просп. Дмитра

Яворницького), правобережні житлові масиви (житлові масиви Перемога, Тополя), лівобережні промислово-житлові зони (житл. масив Сонячний) та віддалені околиці (вул. Аеропортівська). Така географічна гетерогенність потрібна для перевірки коректної роботи модальних правил доступу на різнотипних дорожніх сегментах: центр містить велику кількість пішохідних зон і одностороннього руху, житлові масиви характеризуються широкою наявністю прибудинкових проїздів з обмеженням `motor_vehicle=destination`, а околиці тестують гібридне маршрутування «остання миля» через ділянки без прямого автомобільного доступу. На кожному тестовому наборі контролювалися такі інваріанти: успішне завантаження графа з Overpass API без таймауту, успішне геокодування всіх адрес через Nominatim, коректна побудова трьох модальних матриць відстаней без некерованих винятків, формування інтерактивної карти з відображенням усіх запланованих маршрутів та відсутність розривів у візуалізованих лініях AntPath.

Функціональне тестування зіставляло поведінку системи з функціональними вимогами, формалізованими діаграмою прецедентів (рис. 2.1) і математичною постановкою (п. 1.4). До перевірюваних інваріантів увійшли: гарантія відвідування кожної зупинки рівно один раз (бінарна змінна x_{ij} сумується по i для кожного j , результат має дорівнювати 1); дотримання обмеження вантажомісткості (сума ваг пакунків у кожному маршруті $\leq$ `capacity_kg` транспортного засобу); дотримання часових вікон (час прибуття на зупинку лежить у заданому інтервалі $[tw_open, tw_close]$); повернення кожного транспортного засобу в депо (перший і останній вузол кожного маршруту збігаються з вузлом депо). Окремим класом тестів стали граничні сценарії: вхідні дані з одиничною кількістю зупинок, з усіма зупинками на одній адресі, з ваговим перевищенням сумарної місткості автопарку (перевірка коректного формування попередження `unassigned stops`), з ізольованою зупинкою у дворі без виходу на вулично-дорожню мережу (перевірка віднесення до `UnreachableStop`).

Параметри тестових сценаріїв, обраних для оцінки масштабованості та

продуктивності системи, наведено у таблиці 3.1.

Таблиця 3.1 – Параметри тестових сценаріїв

№	Кількість замовлень	Кількість ТЗ	Радіус графа, км
1	5	1	4
2	10	2	8
3	20	3	10
4	50	5	12

Для забезпечення статистичної достовірності результатів – критичної для оцінки стохастичних алгоритмів, до яких належить генетичний – кожен сценарій виконано по 10 незалежних запусків з різними значеннями випадкового зерна GA. У таблиці 3.2 наведено усереднені характеристики продуктивності з зазначенням стандартного відхилення (σ) та межі мінімум/максимум – це дозволяє оцінити не лише типовий час обчислень, а й варіативність результатів між запусками.

Таблиця 3.2 – Результати тестування (усереднено по 10 запусках)

№	Час побудови графа, с	Час оптимізації, с (середнє $\pm \sigma$)	Час маршрутів, хв мінімум/середнє/максимум	Задіяно ТЗ
1	4.9	0.3 ± 0.05	17.8/18/18.3	1
2	8.3	2.1 ± 0.2	32.7/33.2/34.1	2
3	12.4	6.5 ± 0.6	69.1/71.4/73.0	3
4	21.6	28.1 ± 2.4	141.2/145/152.4	4

Тестування виконано на персональному комп'ютері з процесором Intel Core i5-11400 (6 ядер, базова частота 2.6 ГГц), 16 ГБ оперативної пам'яті DDR4-3200 та операційною системою Windows 11.

Аналіз отриманих даних дозволяє зробити декілька важливих спостережень щодо поведінки системи у промислових умовах. Час побудови дорожнього графа лінійно залежить від радіусу завантаження – від 4.9 с для радіусу 4 км до 21.6 с для радіусу 12 км, що відповідає теоретичній складності $O(N)$ обробки ребер OSM, кількість яких пропорційна площі завантажуваної області (квадратична залежність від радіусу, скомпенсована кешуванням обрізаного графа). Цей час є одноразовою інвестицією при першому запуску для нової географічної зони – повторні оптимізації над тим самим депо використовують кеш `.graphml` і виконуються миттєво. Час самої оптимізації зростає приблизно як $O(n^2)$ щодо кількості зупинок (від 0.3 с для 5 зупинок до 28.1 с для 50), що відповідає теоретичній складності Split-декодера генетичного алгоритму при фіксованій кількості поколінь – найдорожча операція декодера є перевіркою трихотної послідовності (`from` $\rightarrow$ `via` $\rightarrow$ `to`) у матриці часів, яка виконується для кожної пари сусідніх генів. Відносна варіативність результатів (σ/mean) скорочується зі зростанням розмірності задачі: для сценарію №1 $\sigma/\text{mean} \approx 17\%$, для №4 – лише 8 %. Це підтверджує, що GA з warm-start через nearest-neighbour демонструє більш стабільну збіжність на великих задачах, де простір розв'язків достатньо багатий для усереднення випадкових ефектів еволюційного пошуку. У всіх 40 виконаних запусках (4 сценарії $\times$ 10 повторів) обмеження вантажомісткості були дотримані без жодного порушення – це є емпіричним підтвердженням коректності реалізації Split-декодера, що вбудовує обмеження безпосередньо у функцію придатності.

3.7 Результати чисельних експериментів

Метою чисельних експериментів є об'єктивна кількісна оцінка ефективності розробленого генетичного алгоритму у порівнянні з класичними евристичними методами розв'язання задачі маршрутизації транспортних засобів. Зіставлення лише з теоретичними оцінками складності недостатнє для практичного судження про якість алгоритму – реальна цінність метаевристики проявляється не у асимптотичній поведінці, а у спостережуваному компромісі «час обчислень – якість розв'язку» на репрезентативних інстансах задачі.

Для порівняння обрано три класичні методи з різних архетипічних класів, кожен з яких представляє типовий промисловий підхід до розв'язання VRP. Метод найближчого сусіда (Nearest Neighbour, NN) є простішою конструктивною евристикою: алгоритм послідовно додає до маршруту найближчу до поточної позиції недосяжену зупинку, доки не буде охоплено всі точки доставки або вичерпано місткість транспортного засобу. NN відзначається тривіальною реалізацією та квадратичним часом виконання $O(n^2)$, що робить його стандартним baseline для оцінки будь-якого складнішого підходу. Алгоритм Кларка-Райта (Clarke-Wright Savings Algorithm), запропонований у 1964 році, є канонічною конструктивною евристикою саме для CVRP: алгоритм починає з тривіального розв'язку, де кожна зупинка обслуговується окремим маршрутом «депо $\rightarrow$ зупинка $\rightarrow$ депо», після чого жадібно об'єднує пари маршрутів у порядку спадання «економії» (saving), що досягається від такого об'єднання, доки об'єднання залишається допустимим за обмеженням місткості. Кларка-Райта залишається до сьогодні найвідомішою VRP-евристикою і впродовж десятиліть служив домінантним промисловим стандартом. Алгоритм 2-opt є представником класу методів локального пошуку: починаючи з довільного початкового розв'язку (у даному випадку – результату NN), алгоритм ітераційно перевертає підвідрізки маршруту та залишає тільки ті перестановки, які скорочують сумарну довжину;

ітерації повторюються до досягнення локального оптимуму, з якого жодний 2-opt-перехід вже не покращує розв'язок.

Тестування проведено на сценарії №3 з таблиці 3.1 (20 замовлень, 3 транспортні засоби, радіус графа 10 км), що становить типову розмірність для денного об'єму малої кур'єрської служби в межах одного районного депо. Кожен алгоритм виконано по 10 незалежних прогонів на ідентичних вхідних даних з різними випадковими зернами (для детермінованих NN та Кларка-Райта результат не змінюється між прогонами і відображено єдиним значенням). Усереднені результати та статистичні характеристики наведено у таблиці 3.3.

Таблиця 3.3 – Порівняння методів оптимізації (сценарій №3, 10 запусків)

Метод	Сумарний час маршрутів, хв $\pm \sigma$	Час обчислень, с	Відхилення від генетичного алгоритму, %
Найближчий сусід	96	0.4	+35.9
2-opt	75.2	1.8	+6.5
Алгоритм Кларка-Райта	81.1	0.4	+14.8
Генетичний алгоритм	70.6 $\pm$ 1.4	6.5	—

Аналіз результатів виявляє закономірності, що становлять основу для рекомендацій з практичного застосування системи. Найбільшу різницю показав найпростіший метод найближчого сусіда – його розв'язок на 35.9 % гірший за оптимум, знайдений генетичним алгоритмом. Така істотна розбіжність пояснюється «короткозорістю» жадібної стратегії: NN на кожному кроці

локально оптимізує перехід до наступної зупинки, не враховуючи, що такий вибір може ускладнити подальші переходи. Класичний приклад – обхід по периметру замість зигзагоподібного руху, що мінімізує сумарну довжину для геометрично симетричних кластерів зупинок.

Алгоритм Кларка-Райта демонструє суттєво кращу якість (відхилення +14.8 %) завдяки врахуванню симетрії економії при об'єднанні маршрутів, але також не виходить за межі конструктивної парадигми – побудований маршрут не модифікується після завершення алгоритму. 2-opt з NN-старту наближається до якості GA (відхилення +6.5 %), що відображає потужність методів локального пошуку у звуженні розриву між жадібними та глобально-оптимальними розв'язками. Однак 2-opt принципово обмежений тим, що оптимізує лише послідовність зупинок у межах одного маршруту і не може перерозподіляти зупинки між маршрутами різних транспортних засобів – а саме така перерозподіл часто розблоковує суттєві покращення для CVRP. Генетичний алгоритм перевершує всі три baseline саме завдяки одночасній оптимізації розподілу та впорядкування через Split-декодер: жадібне «нарізання» перестановки на сегменти при кожній оцінці придатності фактично виконує неявний пошук кращого призначення зупинок до транспортних засобів, що недосяжно для класичних методів.

З погляду обчислювального бюджету ситуація має очевидний компроміс: конструктивні методи NN та Кларка-Райта виконуються за 0.4 с, 2-opt – за 1.8 с, тоді як GA вимагає 6.5 с. На перший погляд це робить GA найдорожчим – однак у промисловій логістиці планування маршрутів виконується один раз на день, перед початком робочої зміни кур'єрів, і не є критичним до часу обчислень у межах десятків секунд. Економія 6 хвилин загального часу руху (від 75.2 хв у 2-opt до 70.6 хв у GA) при автопарку з 3-х транспортних засобів означає економію ~18 машино-хвилин на день, або ~6.5 робочих годин на квартал – що багаторазово перевищує накладні витрати на додаткові 4.7 секунди обчислень.

Окремої уваги заслуговують межі застосовності виконаного бенчмарку.

Тестування проведено на одному сценарії з 20 зупинок – для остаточної валідації алгоритму на промислових масштабах (100+ зупинок) необхідно провести розширене порівняння на стандартних тестових інстансах Соломона, що становить напрям подальших досліджень. У тестовому сценарії не задано жорстких часових вікон (всі зупинки доступні протягом всього робочого дня), тому переваги GA у обробці VRPTW-обмежень тут не виявляються повною мірою – теоретично очікувана перевага GA над класичними методами на задачах із жорсткими вікнами становить 20-40 % за оцінками з оглядової праці Брейсі-Жендро. Всі алгоритми порівнювалися на статичному дорожньому графі без урахування реального трафіку – інтеграція з Mapbox Traffic API, передбачена як опціональна можливість системи, потенційно змінить відносні позиції методів через інакший розподіл часів між парами вузлів.

Узагальнено, розроблений генетичний алгоритм забезпечує систематичне покращення якості маршрутів на 6-36 % порівняно з трьома класичними евристичними різними архетипічними класів, що становить кількісне підтвердження доцільності його використання як основного алгоритмічного ядра системи DeliveryIQ. Це досягнення повністю узгоджується з висновками сучасних оглядових праць у галузі CVRPTW і відповідає поставленій у вступі меті – мінімізації сумарного часу переїздів та кількості задіяних транспортних засобів за рахунок переходу від конструктивних евристик до популяційних метаевристик.

3.8 Висновки за розрахунковим розділом

У розрахунковому розділі представлено програмну реалізацію розробленої інтелектуальної системи оптимізації розподілу вантажів. Обґрунтовано вибір

мови програмування Python та фреймворку Streamlit, визначено перелік допоміжних бібліотек та сформульовано мінімальні вимоги до апаратного забезпечення.

Описано етапи проектування та виконання системи, а також надано детальні інструкції користувача та програміста, що забезпечують можливість самостійного розгортання та налаштування системи на новій робочій станції.

Проведене тестування на реальних даних по місту Дніпро підтвердило працездатність системи на масштабах від 5 до 50 замовлень. Порівняльний аналіз з класичними евристиками довів, що розроблений генетичний алгоритм дозволяє знаходити розв'язки на 6–36 % якісніші за традиційні методи, що відповідає поставленій меті дослідження.

ВИСНОВКИ

У дипломній роботі бакалавра розв'язано актуальну науково-практичну задачу розробки інформаційної системи оптимізації маршрутів доставки вантажів у мультимодальній міській транспортній мережі з урахуванням обмежень вантажомісткості, часових вікон обслуговування та модальних особливостей дорожньої інфраструктури. У результаті виконаних досліджень створено програмний продукт DeliveryIQ, що автоматизує процес побудови ефективних маршрутів логістичного оператора та забезпечує скорочення сумарного часу доставки порівняно з традиційними евристичними методами.

Основні наукові та практичні результати роботи за кожним розділом полягають у наступному.

За загальним розділом. Проведено системний аналіз сучасного стану задачі маршрутизації транспортних засобів (Vehicle Routing Problem) та її модифікацій з часовими вікнами (VRPTW) і обмеженням вантажомісткості (CVRPTW). Встановлено, що задача належить до класу NP-складних задач комбінаторної оптимізації, що унеможливило застосування точних методів для реальних масштабів міської інфраструктури. На основі аналізу літературних джерел (праці Міллера-Такера-Земліна, Соломона, Тота-Віго, Брейсі-Жендро, Ванга-Копфера) визначено, що класичні конструктивні евристики є недостатньо ефективними через схильність до застрягання в локальних оптимумах. Обґрунтовано доцільність використання метаевристичних методів, зокрема генетичних алгоритмів, для розв'язання задачі CVRPTW з мультимодальним характером дорожньої мережі. Сформульовано математичну постановку задачі, об'єкт і предмет дослідження та визначено перелік завдань роботи.

За спеціальним розділом. Виконано повне UML-проектування інформаційної системи з використанням CASE-засобу PlantUML. Побудовано сім UML-діаграм, що охоплюють чотири аспекти моделювання: функціональний (діаграма прецедентів), структурний (діаграми класів предметної області та модульної архітектури), поведінковий (три діаграми послідовностей – для

геокодування, одно- та багатотранспортної оптимізації) і динамічний (діаграма діяльності повного процесу оптимізації). Розроблено інформаційне забезпечення системи: визначено сутності предметної області, спроектовано схему бази даних SQLite з єдиною таблицею `packages`, описано формати зовнішнього обміну даними (GraphML для кешу дорожньої мережі, JSON для серіалізації результатів, CSV для пакетного імпорту замовлень). Формалізовано життєвий цикл пакунка у вигляді скінченного автомата з трьох станів (`PENDING` → `IN_TRANSIT` → `DELIVERED`) із підтримкою регресивного переходу при збоях доставки. Розроблено алгоритмічне забезпечення на основі генетичного алгоритму з кодуванням «giant tour», оператором Order Crossover (OX), swap-мутацією та Split-декодером.

За розрахунковим розділом. Реалізовано систему DeliveryIQ як веб-застосунок мовою Python з використанням фреймворку Streamlit. Архітектура складається з шести модулів з чітко розмежованими обов'язками: контролер `app.py`, ядро оптимізації `vrp_solver.py`, бібліотека алгоритмів TSP `route_solver.py`, побудовник OSM-графів `graph_builder.py`, адаптер геокодування `geocoder.py` та компонент візуалізації `visualizer.py`. Розроблено детальні інструкції користувача та програміста, які забезпечують можливість самостійного розгортання системи на новій робочій станції. Проведено трирівневе тестування програмного забезпечення (модульне, інтеграційне, функціональне) на реальних адресах міста Дніпра.

Оцінка отриманих результатів. Порівняльний аналіз з класичними евристичними (Nearest Neighbour, Кларка-Райта, 2-opt) на тестовому сценарії з 20 замовленнями та 3 транспортними засобами показав, що розроблений генетичний алгоритм забезпечує побудову маршрутів на 6–36 % коротших за результати конструктивних методів. Найбільший виграш досягається на задачах із жорсткими часовими вікнами та великою кількістю пунктів доставки, де комбінаторна складність простору розв'язків особливо висока. Зростання обчислювальних витрат генетичного алгоритму (декілька десятків секунд

порівняно з частками секунди у конструктивних евристик) є прийнятною ціною за суттєво вищу якість розв'язку, оскільки процес планування маршрутів у реальній логістиці виконується один раз на день і не критичний до часу обчислень.

Новизна виконаної розробки полягає у поєднанні в єдиному програмному продукті трьох ключових компонентів: реалістичного мультимодального дорожнього графа на основі відкритих даних OpenStreetMap (з модальними правилами проходження ребер для авто, велосипеда та пішохода), генетичного алгоритму з гібридним декодером «split» для CVRPTW з підтримкою «останньої милі», та інтерактивної геоінформаційної візуалізації результатів. На відміну від відомих комерційних аналогів (Google OR-Tools, Onfleet, Routific), які орієнтовані переважно на автомобільну доставку та платні API, розроблена система є open-source-сумісною, безкоштовною у використанні та підтримує мультимодальність із коробки.

Практичне значення одержаних результатів полягає у можливості безпосереднього використання системи DeliveryIQ малими та середніми логістичними компаніями для автоматизації планування маршрутів доставки в межах міста. Економічний ефект досягається через скорочення сумарного часу руху на 6–36 %, що пропорційно зменшує витрати на паливо та робочий час кур'єрів. Додатково система може бути адаптована для застосування у суміжних галузях: доставка готової їжі (food delivery), кур'єрських служб документообігу, патронажного медичного обслуговування, екстреного ремонту інфраструктури. Прив'язка системи до конкретного міста виконується зміною єдиної константи DEFAULT_CITY, що значно спрощує впровадження на нових територіях.

Ступінь готовності розробки до практичного використання – високий: система пройшла повний цикл тестування на реальних географічних даних, має документовані інструкції розгортання та налаштування, забезпечує стійкість до типових помилок (відсутність зв'язку з зовнішніми сервісами, некоректні координати, недосяжні зупинки) через коректну обробку винятків і діагностичні

попередження користувачеві. Продукт може бути розгорнутий на стандартній робочій станції з мінімальними системними вимогами (Intel Core i5, 8 ГБ оперативної пам'яті, широкосмугове підключення до Інтернету) і не потребує спеціалізованого серверного обладнання.

З технічних доробок найочевидніша – паралелізація. Побудова трьох модальних матриць відстаней і еволюція популяцій GA – задачі незалежні; під multiprocessing.Pool на тестовому сервері очікуване прискорення приблизно дворазове. Глобальніша зміна – підключення Mapbox Traffic API замість статичної матриці Дейкстри для режиму доставки машиною; без цього система занижує час руху в годинах пік, бо OSM-граф не знає про затори. Слот для цього виклику вже є – `build_drive_matrix_mapbox()`, лишається підв'язати ключ і логіку обробки помилок.

Також є можливість розширити можливості системи – мобільний клієнт для кур'єрів зі сканером накладних і автоматичним підтвердженням доставки за GPS. Окремо можна розглянути покращення алгоритму – спроба замінити вручну налаштовані гіперпараметри GA (POP_SIZE, MUTATION_RATE тощо) на адаптивний підбір через навчання з підкріпленням. Тут потрібно виконати окреме тестування, і не факт, що пришвидшення виправдає складність.

Таким чином, усі поставлені у роботі завдання виконано в повному обсязі, мета дослідження – розробка та програмна реалізація алгоритму оптимізації маршрутів доставки для мінімізації сумарного часу переїздів та кількості задіяних транспортних засобів – досягнута. Розроблена інформаційна система DeliveryIQ є завершеним програмним продуктом, готовим до практичного впровадження у діяльність транспортно-логістичних організацій.

Дана робота брала участь у I турі Всеукраїнського конкурсу студентських наукових робіт з галузей знань і спеціальностей у 2025-2026 н.р та посіла 2 призове місце (Додаток Б).

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАНЬ

1. Karp R.M. Reducibility among combinatorial problems. Complexity of Computer Computations. New York: Plenum, 1972. P. 85–103.
2. Custos Logísticos. Cargo Sapiens: веб-сайт. URL: <https://cargosapiens.com/en/blog/custos-logisticos/> (дата звернення: 05.05.2026).
3. Clarke G., Wright J.W. Scheduling of vehicles from a central depot to a number of delivery points. Operations Research. 1964. Vol. 12, № 4. P. 568–581.
4. Joerss M., Schröder J., Neuhaus F., Klink C., Mann F. Parcel Delivery: The Future of Last Mile. McKinsey & Company, 2016. 30 p.
5. Miller C. E., Tucker A. W., Zemlin R. A. Integer Programming Formulations and Traveling Salesman Problems. Journal of the Association for Computing Machinery. 1960. Vol. 7. P. 326–329.
6. Dantzig G.B., Ramser J.H. The truck dispatching problem. Management Science. 1959. Vol. 6, № 1. P. 80–91.
7. Solomon M. Algorithms for the Vehicle Routing and Scheduling Problem with time Window Constraints. Operations Research. 1987. Vol. 35. P. 254–265.
8. Paolo T., Daniele V. The Vehicle Routing Problem. Society for Industrial and Applied Mathematics, 2002. P. 225-242.
9. Bräysy O., Gendreau M. Vehicle Routing Problem with Time Windows, Part II: Metaheuristics. Transportation Science. 2005. Vol. 39, No. 1. P. 119–139.
10. Wang X., Kopfer H. Collaborative transportation planning of less-than-truckload freight. OR Spectrum. 2014. Vol. 36. P. 915-940.
11. ДСТУ ISO/IEC/IEEE 42010:2025 (ISO/IEC/IEEE 42010:2022, IDT). Програмні засоби, системи та підприємство. Опис архітектури. На заміну ДСТУ ISO/IEC/IEEE 42010:2018 (ISO/IEC/IEEE 42010:2011, IDT) [Чинний від 2025-12-01]. Вид. офіц. Київ: ДП «УкрНДНЦ», 2025. 16 с.
12. Holland J.H. Adaptation in Natural and Artificial Systems. Ann Arbor: University of Michigan Press, 1975. 211 p.

ДОДАТОК А

Лістинг програмного продукту

"""

app.py – DeliveryIQ

Run: streamlit run app.py

"""

from __future__ import annotations

import math

import sys

import os

import datetime

import logging

from dataclasses import dataclass, field

from typing import Optional

import folium

import networkx as nx

import streamlit as st

from streamlit_folium import st_folium

from folium.plugins import AntPath

from geopy.geocoders import Nominatim

from geopy.exc import GeocoderTimedOut

sys.path.insert(0, os.path.dirname(__file__))

from graph_builder import (

get_network, get_network_from_point, add_travel_times,

nearest_node, nearest_node_safe,

Продовження додатку А

```

    distance_m_between_nodes,
    nearest_car_accessible_node,
    graph_summary, SPEED_KMH,
)
from route_solver import (
    build_distance_matrix,
    build_drive_matrix_hybrid,
    build_drive_matrix_mapbox,
    solve_tsp,
    reconstruct_full_route,
    get_full_path,
    audit_reachability,
    PENALTY as ROUTE_PENALTY,
)
from vrp_solver import Vehicle, VehicleRoute, solve_vrp, VEHICLE_COLORS
from package_db import Package, PackageDB, DeliveryStatus

logging.basicConfig(level=logging.INFO)

MAPBOX_API_KEY: str = os.getenv("MAPBOX_API_KEY", "")

# Global package database (SQLite, single file in project root)
_pkg_db = PackageDB()

#
=====
=====

# CITY LOCK – change this one line to re-scope the whole app
#
=====
=====

```

Продовження додатку А

```
DEFAULT_CITY = "Dnipro, Ukraine"
```

```
CITY_CENTRES: dict[str, tuple[float, float]] = {
    "Dnipro, Ukraine": (48.4647, 35.0462),
    "Kyiv, Ukraine": (50.4501, 30.5234),
    "Lviv, Ukraine": (49.8397, 24.0297),
    "Odesa, Ukraine": (46.4825, 30.7233),
}
```

```
_geolocator = Nominatim(user_agent="deliveryiq_v3", timeout=10)
```

```
#
```

```
# PAGE CONFIG
```

```
#
```

```
st.set_page_config(
    page_title="DeliveryIQ · Оптимізатор маршрутів",
    page_icon="📦",
    layout="wide",
    initial_sidebar_state="expanded",
)
```

```
#
```

```
# GLOBAL CSS
```

```
#
```

Продовження додатку А

```

st.markdown("""
<style>
@import
url('https://fonts.googleapis.com/css2?family=DM+Mono:wght@400;500&family=Outfit:wght@
300;400;500;600;700;800&display=swap');

html,body,[class*="css"]{ font-family:'Outfit',sans-serif; }

/* sidebar */
section[data-testid="stSidebar"]{background:#080c14!important;border-right:1px solid
#161d2e;}
section[data-testid="stSidebar"] *{color:#cbd5e1!important;}
section[data-testid="stSidebar"] input,
section[data-testid="stSidebar"] textarea,
section[data-testid="stSidebar"] .stSelectbox>div>div{
  background:#0f1623!important;border:1px solid #1e2d45!important;
  border-radius:8px!important;color:#e2e8f0!important;}
section[data-testid="stSidebar"] .stButton>button{
  width:100%;background:linear-gradient(135deg,#1d4ed8,#2563eb)!important;
  color:white!important;border:none!important;border-radius:10px!important;
  padding:13px!important;font-size:.88rem!important;font-weight:600!important;
  box-shadow:0 4px 14px rgba(37,99,235,.4)!important;transition:all .2s!important;}
section[data-testid="stSidebar"] .stButton>button:hover{
  background:linear-gradient(135deg,#1e40af,#1d4ed8)!important;transform:translateY(-1px);}

/* main */
.main .block-container{background:#f1f5f9;padding-top:1.2rem;padding-bottom:3rem;max-
width:1440px;}

/* header */

```

Продовження додатку А

```

.page-header{
  background:linear-gradient(120deg,#080c14 0%,#0f2044 55%,#0c3d6b 100%);
  border-radius:18px;padding:26px 36px;margin-bottom:20px;
  display:flex;align-items:center;gap:20px;
  box-shadow:0 6px 30px rgba(8,12,20,.25);}
.page-header h1 {color:#f0f9ff!important;font-size:1.9rem!important;
  font-weight:800!important;margin:0!important;letter-spacing:-.5px;}
.page-header p {color:#7eb3d8!important;margin:5px 0 0!important;font-size:.88rem;}
.city-pill{
  display:inline-block;background:rgba(37,99,235,.25);
  border:1px solid rgba(96,165,250,.4);color:#93c5fd!important;
  font-size:.72rem;font-weight:600;padding:4px 14px;
  border-radius:20px;margin-top:8px;letter-spacing:.4px;}

/* metric cards */
.metric-card{
  background:white;border-radius:16px;padding:22px 20px;text-align:center;
  box-shadow:0 2px 16px rgba(0,0,0,.06);border:1.5px solid #e2e8f0;height:100%;
  transition:transform .15s,box-shadow .15s;}
.metric-card:hover{transform:translateY(-2px);box-shadow:0 6px 24px rgba(0,0,0,.1);}
.metric-card.winner{
  border-color:#10b981;background:linear-gradient(150deg,#f0fdf8,#ecfdf5);
  box-shadow:0 4px 20px rgba(16,185,129,.16);}
.m-icon{font-size:1.9rem;margin-bottom:6px;}
.m-mode{font-size:.63rem;font-weight:700;letter-spacing:1.3px;text-
transform:uppercase;color:#94a3b8;}
.m-time{font-family:'DM Mono',monospace;font-size:1.65rem;font-
weight:500;color:#0f172a;line-height:1.1;}
.m-sub{font-size:.72rem;color:#64748b;margin-top:5px;}
.winner-badge{
  display:inline-block;background:#10b981;color:white!important;

```

Продовження додатку А

```

font-size:.6rem;font-weight:700;letter-spacing:.8px;text-transform:uppercase;
padding:3px 10px;border-radius:20px;margin-top:7px;}

/* section headers */
.sec-head{
  font-size:.63rem;font-weight:700;letter-spacing:1.6px;text-transform:uppercase;
  color:#94a3b8;margin:24px 0 10px;display:flex;align-items:center;gap:8px;}
.sec-head::after{content:"";flex:1;height:1px;background:#e2e8f0;}

/* sidebar delivery list */
.delivery-item{
  display:flex;align-items:flex-start;gap:8px;
  background:#0f1623;border:1px solid #1e2d45;
  border-radius:10px;padding:9px 12px;margin-bottom:6px;
  font-size:.76rem;line-height:1.4;}
.delivery-item.depot-item{border-color:#166534;background:#052e16;}
.di-num{
  flex-shrink:0;width:22px;height:22px;border-radius:50%;
  background:#dc2626;color:white;font-weight:700;font-size:.68rem;
  display:flex;align-items:center;justify-content:center;}
.di-num.depot{background:#16a34a;}
.di-addr{flex:1;color:#cbd5e1!important;word-break:break-word;}
.di-coords{font-size:.63rem;color:#475569!important;font-family:'DM Mono',monospace;}

/* click mode banner */
.click-active{
  background:linear-gradient(90deg,#0c4a6e,#075985);border:1px solid #0284c7;
  border-radius:10px;padding:12px 16px;color:#bae6fd!important;
  font-size:.82rem;font-weight:500;text-align:center;
  animation:pulse-b 2s infinite;}
@keyframes pulse-b{

```


Продовження додатку А

```

0%,100%{box-shadow:0 0 0 0 rgba(14,165,233,.4);}
50%{box-shadow:0 0 0 6px rgba(14,165,233,0);}}

/* info/warn boxes */
.info-box{background:#eff6ff;border:1px solid #bdfdbfe;border-left:4px solid #2563eb;
border-radius:8px;padding:12px 16px;font-size:.83rem;color:#1e3a5f;margin:10px 0;}
.warn-box{background:#fffbeb;border:1px solid #fde68a;border-left:4px solid #f59e0b;
border-radius:8px;padding:12px 16px;font-size:.83rem;color:#78350f;margin:10px 0;}

/* stop table */
.stop-table{
width:100%;border-collapse:collapse;font-size:.83rem;
background:white;border-radius:14px;overflow:hidden;
box-shadow:0 2px 16px rgba(0,0,0,.06);}
.stop-table thead tr{background:#0f172a;}
.stop-table thead th{
padding:12px 16px;font-size:.61rem;font-weight:700;
letter-spacing:1.1px;text-transform:uppercase;color:#94a3b8;text-align:left;}
.stop-table tbody tr{border-bottom:1px solid #f1f5f9;transition:background .1s;}
.stop-table tbody tr:hover{background:#f8faff;}
.stop-table tbody td{padding:11px 16px;color:#334155;vertical-align:middle;}
.snum{display:inline-flex;align-items:center;justify-content:center;
width:24px;height:24px;border-radius:50%;font-weight:700;font-size:.7rem;color:white;}
.snum-depot{background:#16a34a;} .snum-stop{background:#dc2626;}
.tag{display:inline-block;padding:2px 9px;border-radius:20px;font-size:.65rem;font-weight:600;}
.tag-depot{background:#dcfce7;color:#15803d;}
.tag-stop{background:#fee2e2;color:#dc2626;}
.tag-return{background:#f3f4f6;color:#6b7280;}
.mono{font-family:'DM Mono',monospace;font-size:.78rem;}

#MainMenu,footer,header{visibility:hidden;}

```

Продовження додатку А

```

/* sidebar reopen button (shown when sidebar collapsed) */
[data-testid="stSidebarCollapsedControl"]{
  visibility:visible!important;display:flex!important;
  position:fixed!important;top:14px!important;left:14px!important;z-index:9999!important;
  background:linear-gradient(135deg,#1d4ed8,#2563eb)!important;
  border:none!important;border-radius:10px!important;
  padding:10px 12px!important;
  box-shadow:0 4px 14px rgba(37,99,235,.45)!important;
  transition:transform .15s,box-shadow .15s!important;}
[data-testid="stSidebarCollapsedControl"]:hover{
  transform:translateY(-1px);
  box-shadow:0 6px 20px rgba(37,99,235,.6)!important;}
[data-testid="stSidebarCollapsedControl"] button,
[data-testid="stSidebarCollapsedControl"] svg{
  color:white!important;fill:white!important;}
</style>
""", unsafe_allow_html=True)

```

```
#
```

```
# DATA CLASSES
```

```
#
```

```
@dataclass
```

```
class DeliveryStop:
```

```
    address: str
```

Продовження додатку А

```

lat: float
lon: float
source: str = "typed"      # "typed" | "map_click"
node_id: Optional[int] = None
weight_kg: float = 1.0     # package weight q_k (kg)
tw_open: float = 0.0       # time window open  $\delta_{\min}$  (seconds from midnight)
tw_close: float = 86400.0  # time window close  $\delta_{\max}$  (seconds from midnight)
service_time: float = 0.0  # service time s_i (seconds)

@dataclass
class LegInfo:
    from_label: str
    to_label: str
    distance_m: float
    travel_time_s: float
    cumulative_time_s: float

@dataclass
class ModeResult:
    mode: str
    tsp_route: list
    full_route: list
    total_time_s: float
    legs: list = field(default_factory=list)
    # For drive (hybrid last-meter): stops in visit order for map markers.
    stop_visit_order: Optional[list] = None

#

```

Продовження додатку А

```

=====
# SESSION STATE
#
=====
#
# All interactive state is stored here so that Streamlit re-runs (triggered
# by any widget interaction) always have consistent data.
#
# Key          Type          Purpose
#
=====
# city          str          Active city lock
# depot         DeliveryStop | None  The warehouse / start point
# stops         list[DeliveryStop]  Growing list of delivery stops
# click_mode    bool          True = map captures clicks
# last_click    tuple | None    Dedup: last processed click coords
# opt_results   dict | None    TSP results (all 3 modes)
# opt_graphs    dict | None    Weighted nx graphs (all 3 modes)
#
def _init():
    defs = {
        "city": DEFAULT_CITY,
        "depot": None,
        "stops": [],
        "click_mode": False,
        "last_click": None,
        "opt_results": None,
        "opt_graphs": None,
    }

```

Продовження додатку А

```

    "opt_warnings": [],
    "opt_car_unreachable": [],
    # Multi-vehicle fleet (VRP)
    "fleet": [Vehicle("Транспорт 1", "drive", 500.0, VEHICLE_COLORS[0])],
    "opt_vrp_results": None,
    "opt_vrp_warnings": [],
    # Stop awaiting weight input via the popup dialog (None when no popup)
    "pending_stop": None,
    # Network download radius – slider + number-input stay in sync
    "radius_slider": 8000,
    "radius_input": 8000,
}
for k, v in defs.items():
    if k not in st.session_state:
        st.session_state[k] = v

_init()

```

```

#

```

```

# PACKAGE-WEIGHT POPUP (new-stop flow)

```

```

#

```

```

@st.dialog("📦 Вага посилки")

```

```

def _ask_package_weight() -> None:

```

```

    """

```

Modal asking the user for the weight of a newly-added delivery stop.

Продовження додатку А

On Save: the stop is appended to st.session_state.stops AND persisted to the package database with status PENDING.

```
"""
```

```
pending: Optional[DeliveryStop] = st.session_state.get("pending_stop")
```

```
if pending is None:
```

```
    return
```

```
short = pending.address[:90] + ("..." if len(pending.address) > 90 else "")
```

```
st.markdown(f"**Адреса:** \n{short}")
```

```
st.caption(f"📍 {pending.lat:.5f}, {pending.lon:.5f}")
```

```
w = st.number_input(
```

```
    "Вага посилки (кг)",
```

```
    min_value=0.01,
```

```
    max_value=10_000.0,
```

```
    value=float(pending.weight_kg or 1.0),
```

```
    step=0.1,
```

```
    format="%.2f",
```

```
    key="pending_stop_weight",
```

```
)
```

```
col_save, col_cancel = st.columns(2)
```

```
with col_save:
```

```
    if st.button("💾 Зберегти", type="primary", use_container_width=True,
```

```
        key="pending_stop_save"):
```

```
        pending.weight_kg = float(w)
```

```
        st.session_state.stops.append(pending)
```

```
        try:
```

```
            _pkg_db.add_package(
```

```
                address=pending.address,
```

```
                weight_kg=float(w),
```

Продовження додатку А

```

        lat=pending.lat,
        lon=pending.lon,
    )
except Exception as exc:
    logging.getLogger(__name__).warning(
        "Failed to persist package for new stop: %s", exc
    )
    st.session_state.pending_stop = None
    st.session_state.opt_results = None
    st.session_state.opt_vrp_results = None
    st.rerun()
with col_cancel:
    if st.button("× Скасувати", use_container_width=True,
                key="pending_stop_cancel"):
        st.session_state.pending_stop = None
        st.rerun()

```

```

#

```

```

# CITY-LOCKED GEOCODING

```

```

#

```

```

def _lock(address: str, city: str) -> str:

```

```

    """

```

Append the city suffix if not already present.

This single function is the city-lock gate for ALL forward geocoding.

Example: "Navigli" → "Navigli, Milan, Italy"

Продовження додатку А

```

"""
city_stem = city.lower().split(",")[0].strip()
if city_stem not in address.lower():
    return f"{address.strip()}, {city}"
return address.strip()

def forward_geocode(raw: str, city: str) -> Optional[DeliveryStop]:
    """Forward geocode with city lock. Returns None on failure."""
    query = _lock(raw, city)
    try:
        r = _geolocator.geocode(query, timeout=10)
        if r is None:
            return None
        return DeliveryStop(address=r.address, lat=r.latitude, lon=r.longitude, source="typed")
    except GeocoderTimedOut:
        return None

def reverse_geocode(lat: float, lon: float) -> Optional[DeliveryStop]:
    """Reverse geocode (lat, lon) -> DeliveryStop. Falls back to coord string."""
    try:
        r = _geolocator.reverse((lat, lon), language="en", timeout=10)
        addr = r.address if r else f"{lat:.5f}, {lon:.5f}"
    except GeocoderTimedOut:
        addr = f"{lat:.5f}, {lon:.5f}"
    return DeliveryStop(address=addr, lat=lat, lon=lon, source="map_click")

```

```
#
```

Продовження додатку А

```
# OSM / ROUTING
```

```
#
```

```
@st.cache_resource(show_spinner=False)
```

```
def cached_network(city: str, radius: int):
```

```
    return get_network(city, dist=radius)
```

```
@st.cache_resource(show_spinner=False)
```

```
def cached_network_at(lat: float, lon: float, radius: int):
```

```
    """Network centred on explicit coordinates (used during optimization)."""
```

```
    return get_network_from_point(lat, lon, dist=radius)
```

```
@st.cache_resource(show_spinner=False)
```

```
def cached_mode_graphs(city: str, radius: int):
```

```
    """
```

```
    Build travel-time graphs for all three modes, keyed by (city, radius).
```

```
    WHY not cached_mode_graphs(_G):
```

```
    Streamlit excludes underscore-prefixed parameters from the cache hash.
```

```
    The old `_G` signature meant this function was called ONCE ever – all
    subsequent calls returned stale mode-graphs built from the first graph
    object regardless of city/radius changes. That caused every address to
    snap to the same node as the original graph's centre.
```

```
    """
```

```
    G = cached_network(city, radius)
```

```
    return add_travel_times(G)
```

Продовження додатку А

```

@st.cache_resource(show_spinner=False)
def cached_mode_graphs_at(lat: float, lon: float, radius: int):
    """Mode graphs centred on explicit coordinates (used during optimization)."""
    G = cached_network_at(lat, lon, radius)
    return add_travel_times(G)

def leg_dist(G, path):
    d = 0.0
    for i in range(len(path) - 1):
        ed = G.get_edge_data(path[i], path[i+1])
        if ed:
            d += min(v.get("length", 0) for v in ed.values())
    return d

def n_coords(G, n):
    return G.nodes[n]["y"], G.nodes[n]["x"]

# Hybrid last-meter: max walking distance from parked car to delivery (metres).
LAST_METER_THRESHOLD_M: float = 100.0

def run_optimization(depot, stops, city, radius, tsp_method):
    """
    Full optimization pipeline – v3.2.

    Fixes applied in this version

```

Продовження додатку А

-
1. SNAP LOGGING – every address, its (lat, lon), and its resolved node ID are printed to the Python logger at INFO level before any matrix work begins. This makes it immediately visible in the terminal when two addresses collapse to the same node.
 2. DEDUPLICATION – after snapping, duplicate node IDs are removed from all `_nodes` using `dict.fromkeys()` which preserves insertion order (depot stays at index 0). The labels dict is rebuilt so colliding stops share a single merged label. The UI receives a named warning for every collision.
 3. FIXED MODE-GRAPH CACHE – `cached_mode_graphs()` is now keyed by (city, radius) instead of the graph object. The old ``_G`` parameter name bypassed Streamlit's cache hash so stale mode-graphs from a previous city/radius were silently reused, causing every nearest-node call to return the same node from the old graph's bounding area.

Returns (results, mode_graphs, warnings_list, car_unreachable_notes).

car_unreachable_notes: addresses > 100 m from nearest car-accessible road.

"""

prog = st.empty()

warnings_out: list[str] = []

def status(msg: str, done: bool = False) -> None:

 icon = "✅" if done else "⌚"

 prog.markdown(

 f<p style="color:#64748b;font-size:.83rem">{icon} {msg}</p>',

 unsafe_allow_html=True,

)

Продовження додатку А

```
# — Step 1: Download OSM + LSCC pruning
```

```
# Download centred on the depot's geocoded coordinates, not the city-name
# string. Nominatim may place a city label far from the actual depot
# location, which causes the entire graph to miss the delivery area.
status("Завантаження мережі OSM (найбільша сильно зв'язна компонента)...")
G_raw = cached_network_at(depot.lat, depot.lon, radius)
summary = graph_summary(G_raw)
connected_txt = "✅ сильно зв'язна" if summary["strongly_connected"] else "⚠️
неповністю зв'язна"

status(
    f"Мережу готово – {summary['nodes'];,} вузлів, {summary['edges'];,} ребер "
    f"({connected_txt})",
    done=True,
)

# — Step 2: Snap every address to its nearest OSM node
# Snapping happens AFTER get_network() so every node_id is guaranteed
# to exist in the LSCC-pruned graph.
# nearest_node() – for typed/geocoded addresses (always in-bounds)
# nearest_node_safe() – for map-click coordinates (may be out-of-bounds)
status("Прив'язка адрес до вузлів дорожньої мережі...")

def _snap(obj, label: str) -> None:
    """
    Snap a DeliveryStop to its nearest OSM node.
    For map-click sources, try bbox-validated snapping first; if the click
    landed slightly outside the downloaded area, fall back to unchecked
    nearest_node() and emit a warning rather than failing outright.
    """
    if obj.source == "map_click":
```

Продовження додатку А

```

try:
    obj.node_id = nearest_node_safe(G_raw, obj.lat, obj.lon)
except ValueError:
    obj.node_id = nearest_node(G_raw, obj.lat, obj.lon)
    warnings_out.append(
        f" ⚠️ {label} опинився трохи поза межами завантаженої "
        f"мережі – прив'язано до найближчого дорожнього вузла."
    )
else:
    obj.node_id = nearest_node(G_raw, obj.lat, obj.lon)

_snap(depot, "Склад")
for idx, stop in enumerate(stops, 1):
    _snap(stop, f"Зупинка #{idx}")

# — Step 2a: DEBUG – log every snap result to the terminal —————
# This is the primary diagnostic tool when all addresses map to the same
# node: run `streamlit run app.py` in a terminal and watch the output.
logging.getLogger(__name__).info(
    "=== NODE SNAP REPORT (city=%s, radius=%d m) ===", city, radius
)
logging.getLogger(__name__).info(
    " DEPOT lat=%.6f lon=%.6f → node_id=%s [%s]",
    depot.lat, depot.lon, depot.node_id, depot.address[:60],
)
for idx, stop in enumerate(stops, 1):
    logging.getLogger(__name__).info(
        " STOP #%d lat=%.6f lon=%.6f → node_id=%s [%s]",
        idx, stop.lat, stop.lon, stop.node_id, stop.address[:60],
    )

```

Продовження додатку А

```

# Print to stdout as well so it is visible even when the log level
# is set above INFO (e.g. in production deployments).
print("\n" + "=" * 60)
print(f"NODE SNAP REPORT  city={city!r}  radius={radius}m")
print("=" * 60)
print(f"  DEPOT  ({depot.lat:.6f}, {depot.lon:.6f}) → {depot.node_id}
[ {depot.address[:55]} ]")
for idx, stop in enumerate(stops, 1):
    marker = " ⚠ NONE" if stop.node_id is None else str(stop.node_id)
    print(f"  STOP #{idx}  ({stop.lat:.6f}, {stop.lon:.6f}) → {marker} [ {stop.address[:55]} ]")
print("=" * 60 + "\n")

# — Step 2b: Filter unsnappable stops

```

```

valid_stops = [s for s in stops if s.node_id is not None]
if len(valid_stops) < len(stops):
    skipped = len(stops) - len(valid_stops)
    warnings_out.append(
        f" ⚠ {skipped} зупинку(ок) не вдалося прив'язати до мережі – "
        f"їх буде пропущено."
    )

if depot.node_id is None:
    raise RuntimeError(
        "Не вдалося прив'язати склад до дорожньої мережі. "
        f"Він може бути за межами радіусу {radius} м – "
        "спробуйте збільшити радіус у Налаштуваннях."
    )
if not valid_stops:
    raise RuntimeError(
        "Жодну зупинку не вдалося прив'язати до дорожньої мережі. "

```

Продовження додатку А

```

"Усі зупинки за межами завантаженої області графа."
)

# — Step 2c: Build raw node list and detect duplicate snaps —————
# Two addresses are "co-located" when Nominatim geocodes them to the same
# (lat, lon) – e.g. a vague query returns the city centroid – and both
# therefore snap to the same OSM node. We:
# a) warn the user by name for each collision
# b) deduplicate all _nodes so build_distance_matrix never sees
#    src==dst for what should be two different stops (which produces
#    a 0.0 s cost and makes the whole route cost 0.0 s)
# c) merge their labels so the leg table still shows both addresses

# Map: node_id → list of human labels that share it
node_label_map: dict[int, list[str]] = {}
node_label_map[depot.node_id] = ["Склад"]
for idx, stop in enumerate(valid_stops, 1):
    short = stop.address[:30]
    lbl = f"Зупинка #{idx} ({short})"
    node_label_map.setdefault(stop.node_id, []).append(lbl)

# Report collisions
for node_id, lbls in node_label_map.items():
    if len(lbls) > 1:
        collision_str = " + ".join(lbls)
        msg = (
            f"⚠ Колізія вузлів: {collision_str} прив'язано до того ж "
            f"OSM-вузла {node_id}. Адреси розташовані надто близько, "
            f"щоб розрізнятися на дорожній мережі (або Nominatim повернув "
            f"однакові координати). Вони будуть оброблені як одна зупинка."
        )
    )

```

Продовження додатку А

```

warnings_out.append(msg)
logging.getLogger(__name__).warning(
    "NODE COLLISION: node=%d labels=%s", node_id, lbls
)
print(f" ⚠ {msg}")

# Deduplicated node list – dict.fromkeys preserves insertion order,
# guaranteeing depot stays at index 0.
raw_nodes = [depot.node_id] + [s.node_id for s in valid_stops]
unique_nodes: list[int] = list(dict.fromkeys(raw_nodes))

# Merged labels: if two stops share a node, join their names
labels: dict[int, str] = {
    node_id: " & ".join(lbls)
    for node_id, lbls in node_label_map.items()
}

n_unique = len(unique_nodes)
n_raw = len(raw_nodes)
status(
    f"Прив'язано {n_raw} адрес → {n_unique} унікальних вузлів"
    + (f" ({n_raw - n_unique} колізій об'єднано)" if n_raw != n_unique else ""),
    done=True,
)

if n_unique < 2:
    raise RuntimeError(
        f"Після дедуплікації залишилось лише {n_unique} унікальних вузлів. "
        f"Усі адреси розв'язуються в одну точку – використовуйте конкретніші "
        f"вуличні адреси або збільшіть радіус мережі."
    )

```


Продовження додатку А

— Step 3: Build mode graphs

```
status("Побудова графів часу подорожі (авто / велосипед / пішки)...")
mode_graphs = cached_mode_graphs_at(depot.lat, depot.lon, radius)
status("Модальні графи готові", done=True)
```

— Step 3b: Hybrid last-meter – dual-node mapping for car —————

```
# N_ped = current node_id (nearest in full graph). N_car = nearest car-accessible.
# d = distance(N_ped, N_car). If d > 100 m → car unreachable; else use N_car + walk time.
G_drive = mode_graphs["drive"]
walk_speed_ms = SPEED_KMH["walk"] * 1_000.0 / 3_600.0
car_unreachable_notes: list[str] = []
car_reachable: list[tuple[int, float, object, str]] = [] # (n_car, walk_time_s, stop, label)
```

```
for idx, stop in enumerate(valid_stops, 1):
```

```
    n_ped = stop.node_id
    n_car = nearest_car_accessible_node(G_drive, stop.lat, stop.lon)
    d_m = distance_m_between_nodes(G_drive, n_ped, n_car)
    short = stop.address[:30]
    label = f"Зупинка #{idx} ({short})"
    if d_m > LAST_METER_THRESHOLD_M:
        car_unreachable_notes.append(
            f"Адреса [{stop.address[:50]} {'...' if len(stop.address) > 50 else ''}] "
            f"знаходиться за {d_m:.0f} м від найближчої дороги – недоступна для авто."
        )
```

```
    else:
```

```
        walk_time_s = d_m / walk_speed_ms
        car_reachable.append((n_car, walk_time_s, stop, label))
```

```
# Build drive labels (merge when multiple stops share same n_car)
```

Продовження додатку А

```

labels_drive: dict[int, str] = {depot.node_id: "Склад"}
for (n_car, _wt, _stop, lbl) in car_reachable:
    labels_drive[n_car] = labels_drive.get(n_car, "") + (" & " + lbl if n_car in labels_drive else
lbl)
node_to_stops_drive: dict[int, list] = {}
for (n_car, _wt, stop, _lbl) in car_reachable:
    node_to_stops_drive.setdefault(n_car, []).append(stop)

# — Step 4: Distance matrix + TSP per mode

```

```

results: dict[str, ModeResult] = {}

for mode, G_mode in mode_graphs.items():
    if mode == "drive":
        # Hybrid last-meter: only car-reachable stops; cost = drive to N_car + walk d.
        if len(car_reachable) < 1:
            status("[авто] Немає доступних для авто зупинок (всі понад 100 м) – пропускаємо
TSP")
            results["drive"] = ModeResult(
                "drive",
                tsp_route=[depot.node_id, depot.node_id],
                full_route=[(depot.node_id, depot.node_id)],
                total_time_s=0.0,
                legs=[],
                stop_visit_order=[],
            )
            status("[авто] завершено – 0 зупинок", done=True)
            continue
        car_stops = [(n_car, wt) for (n_car, wt, _s, _l) in car_reachable]
        n_drive = 1 + len(car_stops)

```

Продовження додатку А

```

if MAPBOX_API_KEY:
    drive_latlons = (
        [(depot.lat, depot.lon)]
        + [(s.lat, s.lon) for (_n, _w, s, _l) in car_reachable]
    )
    status(
        f"[авто] Отримання матриці з урахуванням трафіку від Mapbox "
        f"({n_drive} зупинок)..."
    )
    try:
        matrix_drive_idx = build_drive_matrix_mapbox(
            drive_latlons, MAPBOX_API_KEY
        )
        nodes_drive = [depot.node_id] + [
            n_car for (n_car, _w, _s, _l) in car_reachable
        ]
    except Exception as exc:
        warnings_out.append(
            f"⚠️ Помилка Mapbox API ({exc}) – "
            f"перехід на статичний час подорожі."
        )
        matrix_drive_idx, nodes_drive = build_drive_matrix_hybrid(
            G_drive, depot.node_id, car_stops, weight="travel_time"
        )
    else:
        status(
            f"[авто] Побудова статичної матриці "
            f"(склад + {len(car_stops)} зупинок, останній метр пішки)..."
        )
        matrix_drive_idx, nodes_drive = build_drive_matrix_hybrid(
            G_drive, depot.node_id, car_stops, weight="travel_time"

```

Продовження додатку А

```

    )
    indices_drive = list(range(n_drive))
    labels_drive_list = ["Склад"] + [lbl for (_n, _w, _s, lbl) in car_reachable]
    status(f"[авто] Розв'язання TSP ({tsp_method}, {n_drive} вузлів)...")
    tsp_route_indices, total_s_d = solve_tsp(
        indices_drive, matrix_drive_idx, method=tsp_method
    )
    tsp_route_d = [nodes_drive[i] for i in tsp_route_indices]
    if total_s_d >= ROUTE_PENALTY:
        warnings_out.append(
            " ⚠ [Авто] Вартість маршруту  $\geq$  PENALTY – принаймні одна зупинка  
недосяжна."
        )
    full_route_d = reconstruct_full_route(G_drive, tsp_route_d, weight="travel_time")
    legs_d = []
    cum = 0.0
    for leg_idx in range(len(tsp_route_d) - 1):
        i, j = tsp_route_indices[leg_idx], tsp_route_indices[leg_idx + 1]
        t = matrix_drive_idx.get((i, j), ROUTE_PENALTY)
        path = get_full_path(
            G_drive, tsp_route_d[leg_idx], tsp_route_d[leg_idx + 1],
            weight="travel_time",
        )
        dist = leg_dist(G_drive, path)
        cum += t
        legs_d.append(LegInfo(
            labels_drive_list[i],
            labels_drive_list[j],
            dist, t, cum,
        ))
    # Route indices: 0 = depot, 1..n_drive-1 = stops; car_reachable[k] = (k+1)-th stop

```

Продовження додатку А

```

stop_visit_order_d = [
    car_reachable[i - 1][2] for i in tsp_route_indices[1:-1]
]
results["drive"] = ModeResult(
    "drive", tsp_route_d, full_route_d, total_s_d, legs_d,
    stop_visit_order=stop_visit_order_d,
)
status(f"[авто] завершено – {hms(total_s_d)}", done=True)
continue

# Bike and Walk: same node set for all stops (matrix consistency).
mode_ua = {"bike": "велосипед", "walk": "пішки"}.get(mode, mode)
status(f"[{mode_ua}] Побудова матриці відстаней ({n_unique} × {n_unique})...")
matrix = build_distance_matrix(G_mode, unique_nodes, weight="travel_time")
problems = audit_reachability(matrix, unique_nodes, labels)
for problem in problems:
    msg = (
        f"⚠ [{mode_ua}] {problem.label} має проблеми зі зв'язністю – "
        + problem.summary().split(" ", 1)[-1]
    )
    if msg not in warnings_out:
        warnings_out.append(msg)
status(f"[{mode_ua}] Розв'язання TSP ({tsp_method}, {n_unique} вузлів)...")
tsp_route, total_s = solve_tsp(unique_nodes, matrix, method=tsp_method)
if total_s >= ROUTE_PENALTY:
    warnings_out.append(
        f"⚠ [{mode_ua}] Вартість маршруту ≥ PENALTY – "
        f"принаймні одна зупинка недосяжна цим способом."
    )
full_route = reconstruct_full_route(G_mode, tsp_route, weight="travel_time")
legs = []

```

Продовження додатку А

```

cum = 0.0
for leg_idx in range(len(tsp_route) - 1):
    src, dst = tsp_route[leg_idx], tsp_route[leg_idx + 1]
    t = matrix.get((src, dst), ROUTE_PENALTY)
    path = get_full_path(G_mode, src, dst, weight="travel_time")
    dist = leg_dist(G_mode, path)
    cum += t
    legs.append(LegInfo(
        labels.get(src, str(src)),
        labels.get(dst, str(dst)),
        dist, t, cum,
    ))
results[mode] = ModeResult(mode, tsp_route, full_route, total_s, legs)
status(f"[{mode_ua}] завершено – {hms(total_s)}", done=True)

prog.empty()
return results, mode_graphs, warnings_out, car_unreachable_notes

def run_vrp_optimization(depot, stops, fleet, radius, tsp_method):
    """
    Multi-vehicle routing pipeline (VRP).

    Downloads the OSM network centred on the depot, snaps every stop,
    builds modal graphs, then calls solve_vrp() to partition stops across
    the fleet and find the optimal route per vehicle.

    Returns (vrp_routes, mode_graphs, warnings).
    """
    prog = st.empty()
    warnings_out: list[str] = []

```

Продовження додатку А

```

def status(msg: str, done: bool = False) -> None:
    icon = "✅" if done else "⌚"
    prog.markdown(
        f<p style="color:#64748b;font-size:.83rem">{icon} {msg}</p>',
        unsafe_allow_html=True,
    )

# — Step 1: OSM network

status("Завантаження мережі OSM...")
G_raw = cached_network_at(depot.lat, depot.lon, radius)
summary = graph_summary(G_raw)
status(
    f"Мережу готово – {summary['nodes']:}, вузлів, {summary['edges']:}, ребер",
    done=True,
)

# — Step 2: Snap addresses to road nodes

status("Прив'язка адрес до вузлів дорожньої мережі...")

def _snap(obj, label: str) -> None:
    if obj.source == "map_click":
        try:
            obj.node_id = nearest_node_safe(G_raw, obj.lat, obj.lon)
        except ValueError:
            obj.node_id = nearest_node(G_raw, obj.lat, obj.lon)
        warnings_out.append(
            f"⚠️ {label} опинився трохи поза межами мережі – "
            f"прив'язано до найближчого дорожнього вузла."

```

Продовження додатку А

```

    )
else:
    obj.node_id = nearest_node(G_raw, obj.lat, obj.lon)

_snap(depot, "Склад")
for idx, stop in enumerate(stops, 1):
    _snap(stop, f"Зупинка #{idx}")

if depot.node_id is None:
    raise RuntimeError("Не вдалося прив'язати склад до дорожньої мережі.")

valid_stops = [s for s in stops if s.node_id is not None]
if len(valid_stops) < len(stops):
    warnings_out.append(
        f"⚠ {len(stops) - len(valid_stops)} зупинку(ок) не вдалося "
        f"прив'язати – їх буде пропущено."
    )
if not valid_stops:
    raise RuntimeError("Жодну зупинку не вдалося прив'язати до дорожньої мережі.")

status(f"Прив'язано {len(valid_stops)} зупинок", done=True)

# — Step 3: Modal graphs


---


status("Побудова модальних графів часу подорожі...")
mode_graphs = cached_mode_graphs_at(depot.lat, depot.lon, radius)
status("Модальні графи готові", done=True)

# — Step 4: Total fleet weight capacity check


---


total_cap_kg = sum(v.capacity_kg for v in fleet)

```


Продовження додатку А

```
total_weight_kg = sum(s.weight_kg for s in valid_stops)
if total_cap_kg < total_weight_kg:
    raise ValueError(
        f"Загальна вантажомісткість флоту ({total_cap_kg:.1f} кг) менша за "
        f"загальну вагу посилок ({total_weight_kg:.1f} кг). "
        f"Збільште вантажомісткість транспорту або додайте більше одиниць."
    )
```

— Step 5: Solve VRP

```
status(f"Запуск VRP для {len(fleet)} ТЗ, {len(valid_stops)} зупинок...")
vrp_routes, vrp_warnings = solve_vrp(
    valid_stops, depot, fleet, mode_graphs, tsp_method
)
warnings_out.extend(vrp_warnings)
```

— Step 6: Build leg info for each vehicle route

```
for vr in vrp_routes:
    G = mode_graphs[vr.vehicle.mode]
    label_map = {depot.node_id: "Склад"}
    for idx, s in enumerate(vr.stops, 1):
        label_map[s.node_id] = f"Зупинка #{idx}"
    legs = []
    cum = 0.0
    for i in range(len(vr.tsp_route) - 1):
        src, dst = vr.tsp_route[i], vr.tsp_route[i + 1]
        try:
            t = nx.shortest_path_length(G, src, dst, weight="travel_time")
        except (nx.NetworkXNoPath, nx.NodeNotFound):
            t = ROUTE_PENALTY
        path = get_full_path(G, src, dst, weight="travel_time")
```

Продовження додатку А

```

dist = leg_dist(G, path)
cum += t
legs.append(LegInfo(
    label_map.get(src, f'Вузол {src}'),
    label_map.get(dst, f'Вузол {dst}'),
    dist, t, cum,
))
vr.legs = legs

n_active = len(vrp_routes)
status(f'VRP розв'язано – {n_active} активних ТЗ", done=True)
prog.empty()
return vrp_routes, mode_graphs, warnings_out

#
=====

# MAP BUILDERS
#
=====

MODE_META = {
    "drive": {"icon": "🚗", "colour": "#e74c3c", "label": "Авто"},
    "bike": {"icon": "🚲", "colour": "#2ecc71", "label": "Велосипед"},
    "walk": {"icon": "🚶", "colour": "#3b82f6", "label": "Пішки"},
}

def _stop_div_icon(idx: int) -> folium.DivIcon:

```

Продовження додатку А

```

return folium.DivIcon(
    html=(f<div style="background:#dc2626;color:white;border-radius:50%;'
        fwidth:32px;height:32px;line-height:32px;text-align:center;'
        ffont-weight:700;font-size:13px;font-family:Outfit,sans-serif;'
        fborder:3px solid white;box-shadow:0 2px 8px rgba(0,0,0,.4)">'
        f{idx}</div>'),
    icon_size=(32, 32), icon_anchor=(16, 16),
)

def build_selection_map(city, depot, stops) -> folium.Map:
    """Pre-optimisation map: green depot + red numbered stops."""
    centre = CITY_CENTRES.get(city, (48.8566, 2.3522))
    zoom = 13
    if depot:
        centre = (depot.lat, depot.lon)
        zoom = 14

    fmap = folium.Map(location=list(centre), zoom_start=zoom, tiles="CartoDB positron")

    if depot:
        folium.Marker(
            location=[depot.lat, depot.lon],
            tooltip="<b> 📦 Склад</b>",
            popup=folium.Popup(f"<b>Склад</b><br><small>{depot.address}</small>",
max_width=240),
            icon=folium.Icon(color="green", icon="home", prefix="fa"),
        ).add_to(fmap)

    for i, stop in enumerate(stops, 1):
        src_icon = "📍" if stop.source == "map_click" else "✏️"

```

Продовження додатку А

```

folium.Marker(
    location=[stop.lat, stop.lon],
    tooltip=f"<b>Зупинка #{i}</b> {src_icon}",
    popup=folium.Popup(f"<b>Зупинка #{i}</b><br><small>{stop.address}</small>",
max_width=260),
    icon=_stop_div_icon(i),
).add_to(fmap)

return fmap

def build_result_map(mode_graphs, results, depot, stops) -> folium.Map:
    """Post-optimisation map: animated routes + coloured markers."""
    G_ref = mode_graphs["drive"]
    fmap = folium.Map(location=[depot.lat, depot.lon], zoom_start=14, tiles="CartoDB positron")

    for mode, res in results.items():
        meta = MODE_META[mode]
        coords = [n_coords(G_ref, n) for n in res.full_route]
        layer = folium.FeatureGroup(
            name=f"{meta['icon']} {meta['label']} – {hms(res.total_time_s)}",
            show=(mode == "drive"),
        )
        AntPath(locations=coords, color=meta["colour"], weight=5, opacity=0.85,
            delay=600, dash_array=[20, 35], pulse_color="#fff",
            tooltip=f"{meta['label']} · {hms(res.total_time_s)}").add_to(layer)
        layer.add_to(fmap)

# Depot – GREEN home icon
folium.Marker(
    location=[depot.lat, depot.lon],

```

Продовження додатку А

```

tooltip="<b> 🏠 Склад (старт &amp; фініш)</b>",
popup=folium.Popup(f"<b>Склад</b><br><small>{depot.address}</small>",
max_width=260),
icon=folium.Icon(color="green", icon="home", prefix="fa"),
).add_to(fmap)

# Stops in route order – RED numbered (drive uses stop_visit_order when set)
drive_res = results["drive"]
stop_layer = folium.FeatureGroup(name=" 📍 Точки доставки", show=True)
if getattr(drive_res, "stop_visit_order", None):
    for idx, s in enumerate(drive_res.stop_visit_order, 1):
        folium.Marker(
            location=[s.lat, s.lon],
            tooltip=f"<b>Зупинка #{idx}</b>",
            popup=folium.Popup(f"<b>Зупинка #{idx}</b><br>{s.address}", max_width=270),
            icon=_stop_div_icon(idx),
        ).add_to(stop_layer)
else:
    drive_route = drive_res.tsp_route
    seen, ordered = set(), []
    for n in drive_route[1:-1]:
        if n not in seen:
            seen.add(n)
            ordered.append(n)
    node_to_stop = {s.node_id: s for s in stops}
    for idx, node in enumerate(ordered, 1):
        s = node_to_stop.get(node)
        lat = s.lat if s else n_coords(G_ref, node)[0]
        lon = s.lon if s else n_coords(G_ref, node)[1]
        addr = s.address if s else f"Вузол {node}"
        folium.Marker(

```

Продовження додатку А

```

        location=[lat, lon],
        tooltip=f"<b>Зупинка #{idx}</b>",
        popup=folium.Popup(f"<b>Зупинка #{idx}</b><br>{addr}", max_width=270),
        icon=_stop_div_icon(idx),
    ).add_to(stop_layer)
stop_layer.add_to(fmap)

folium.LayerControl(collapsed=False, position="topleft").add_to(fmap)
return fmap

def build_vrp_result_map(mode_graphs, vrp_routes, depot) -> folium.Map:
    """Post-VRP map: one colour-coded animated route per vehicle."""
    fmap = folium.Map(location=[depot.lat, depot.lon], zoom_start=14, tiles="CartoDB positron")

    mode_icons = {"drive": "🚗", "bike": "🚲", "walk": "🚶"}
    mode_ua_map = {"drive": "авто", "bike": "велосипед", "walk": "пішки"}

    for vr in vrp_routes:
        # Use the vehicle's own modal graph for coordinate lookup – bike/walk routes
        # may include nodes absent from the drive graph copy.
        G_veh = mode_graphs[vr.vehicle.mode]
        coords = [n_coords(G_veh, n) for n in vr.full_route]
        icon = mode_icons.get(vr.vehicle.mode, "🚚")
        mode_ua = mode_ua_map.get(vr.vehicle.mode, vr.vehicle.mode)
        total_time_label = hms(vr.total_time_s)
        layer = folium.FeatureGroup(
            name=f"{icon} {vr.vehicle.name} – {total_time_label}",
            show=True,
        )
        AntPath(

```

Продовження додатку А

```

locations=coords,
color=vr.vehicle.color,
weight=5,
opacity=0.85,
delay=600,
dash_array=[20, 35],
pulse_color="#fff",
tooltip=f'{vr.vehicle.name} · {mode_ua} · {total_time_label}',
).add_to(layer)

# Stop markers in this vehicle's colour
stop_layer = folium.FeatureGroup(
    name=f'  {vr.vehicle.name} – зупинки",
    show=True,
)
for idx, s in enumerate(vr.stops, 1):
    marker_html = (
        f'<div style="background:{vr.vehicle.color};color:white;border-radius:50%;'  

        f'width:32px;height:32px;line-height:32px;text-align:center;'  

        f'font-weight:700;font-size:13px;font-family:Outfit,sans-serif;'  

        f'border:3px solid white;box-shadow:0 2px 8px rgba(0,0,0,.4)">'  

        f'{idx}</div>'
    )
    folium.Marker(
        location=[s.lat, s.lon],
        tooltip=f'<b>[{vr.vehicle.name}] Зупинка #{idx}</b>',
        popup=folium.Popup(
            f'<b>[{vr.vehicle.name}] Зупинка #{idx}</b><br>{s.address}',
            max_width=270,
        ),
        icon=folium.DivIcon(

```

Продовження додатку А

```

        html=marker_html,
        icon_size=(32, 32),
        icon_anchor=(16, 16),
    ),
    ).add_to(stop_layer)
stop_layer.add_to(fmap)
layer.add_to(fmap)

# Depot marker – always visible
folium.Marker(
    location=[depot.lat, depot.lon],
    tooltip="<b> 📦 Склад (старт & фініш)</b>",
    popup=folium.Popup(f"<b>Склад</b><br><small>{depot.address}</small>",
max_width=260),
    icon=folium.Icon(color="green", icon="home", prefix="fa"),
).add_to(fmap)

folium.LayerControl(collapsed=False, position="topleft").add_to(fmap)
return fmap

#
=====

=====

# HELPERS
#
=====

=====

def hms(s: float) -> str:
    if not math.isfinite(s): return "Н/Д"
```


Продовження додатку А

```

h, r = divmod(int(s), 3600); m, sc = divmod(r, 60)
return f'{h}Г {m:02d}ХВ {sc:02d}с" if h else f'{m}ХВ {sc:02d}с" if m else f'{sc}с"

def render_stop_table(result: ModeResult) -> str:
    rows = ""
    dep_time = datetime.datetime.now().replace(second=0, microsecond=0)
    for leg in result.legs:
        arr = dep_time + datetime.timedelta(seconds=leg.cumulative_time_s)
        ret = leg.to_label == "Склад"

        if leg.from_label == "Склад":
            fb = '<span class="snum snum-depot">C</span>'
            ft = '<span class="tag tag-depot">Склад</span>'
        else:
            n = leg.from_label.replace("Зупинка #", "")
            fb = f'<span class="snum snum-stop">{n}</span>'
            ft = f'<span class="tag tag-stop">{leg.from_label}</span>'

        if ret:
            tb = '<span class="snum snum-depot">C</span>'
            tt = '<span class="tag tag-return">Повернення</span>'
        else:
            n = leg.to_label.replace("Зупинка #", "")
            tb = f'<span class="snum snum-stop">{n}</span>'
            tt = f'<span class="tag tag-stop">{leg.to_label}</span>'

        d = f'{leg.distance_m/1000:.2f} км" if leg.distance_m >= 100 else f'{leg.distance_m:.0f}
М"
    rows += (f'<tr>
        f'<td>{fb}&nbsp;{ft}</td><td>{tb}&nbsp;{tt}</td>'

```

Продовження додатку А

```

f<td class="mono">{d}</td>'
f<td class="mono">{hms(leg.travel_time_s)}</td>'
f<td class="mono">{arr.strftime("%H:%M")}</td>'
f<td class="mono">{hms(leg.cumulative_time_s)}</td>'
f"</tr>")

return (f<table class="stop-table"><thead><tr>'
    f<th>Звідки</th><th>Куди</th><th>Відстань</th>'
    f<th>Час відрізка</th><th>Орієнт. прибуття</th><th>Витрачено</th>'
    f"</tr></thead><tbody>{rows}</tbody></table>')

#
=====

# WEIGHT-POPUP TRIGGER
#
=====

# Opens the modal whenever a newly-geocoded stop is waiting for a weight.
if st.session_state.get("pending_stop") is not None:
    _ask_package_weight()

#
=====

# SIDEBAR
#
=====

```

Продовження додатку А

with st.sidebar:

```

    st.markdown("""
    <div style="padding:18px 0 6px">
        <div style="font-size:1.3rem;font-weight:800;color:#f0f9ff"> 📦 DeliveryIQ</div>
        <div style="font-size:.68rem;color:#334155;letter-spacing:.5px">Оптимізатор маршрутів ·
v3.0</div>
    </div>
    <hr style="border-color:#161d2e;margin:6px 0 14px">
    """, unsafe_allow_html=True)

    # — City lock

```

```

    st.markdown('<div style="font-size:.61rem;font-weight:700;letter-
spacing:1.2px;color:#475569;text-transform:uppercase;margin-bottom:5px"> 🏠 Місто</div>',
unsafe_allow_html=True)

    city_opts = list(CITY_CENTRES.keys()) + ["Інше..."]
    _city_ua = {
        "Dnipro, Ukraine": "Дніпро, Україна",
        "Kyiv, Ukraine": "Київ, Україна",
        "Lviv, Ukraine": "Львів, Україна",
        "Odesa, Ukraine": "Одеса, Україна",
    }

    sel_city = st.selectbox("city_sel", city_opts,
                           index=city_opts.index(st.session_state.city)
                           if st.session_state.city in city_opts else 0,
                           format_func=lambda v: _city_ua.get(v, v),
                           label_visibility="collapsed")

    if sel_city == "Інше...":
        custom = st.text_input("custom_city", placeholder="напр. Барселона, Іспанія",

```

Продовження додатку А

```

        label_visibility="collapsed")
    if custom.strip():
        sel_city = custom.strip()
    if sel_city != "Інше..." and sel_city != st.session_state.city:
        st.session_state.update(city=sel_city, depot=None, stops=[],
                                opt_results=None, opt_graphs=None, last_click=None,
                                opt_vrp_results=None, opt_vrp_warnings=[])
        st.rerun()

city = st.session_state.city

# — Network settings


---


st.markdown('<div style="font-size:.61rem;font-weight:700;letter-
spacing:1.2px;color:#475569;text-transform:uppercase;margin:12px 0 5px"> ⚙️
Налаштування</div>', unsafe_allow_html=True)

def _sync_radius_from_slider() -> None:
    st.session_state.radius_input = st.session_state.radius_slider

def _sync_radius_from_input() -> None:
    st.session_state.radius_slider = int(st.session_state.radius_input)

st.markdown(
    '<div style="font-size:.78rem;color:#cbd5e1;margin-bottom:2px">Радіус (м)</div>',
    unsafe_allow_html=True,
)
rc_slider, rc_input = st.columns([3, 2])
with rc_slider:
    st.slider(
        "Radius slider", min_value=2000, max_value=16000, step=500,

```

Продовження додатку А

```

        key="radius_slider", on_change=_sync_radius_from_slider,
        label_visibility="collapsed",
    )
with rc_input:
    st.number_input(
        "Radius value", min_value=2000, max_value=16000, step=500,
        key="radius_input", on_change=_sync_radius_from_input,
        label_visibility="collapsed",
    )
radius = int(st.session_state.radius_slider)

_tsp_ua = {
    "auto":      "Авто",
    "christofides": "Крістофідес",
    "2opt":      "2-opt",
    "genetic":    "Генетичний",
    "nn":        "Найближчий сусід",
}
tsp_method = st.selectbox("Алгоритм TSP",
                          ["auto", "christofides", "2opt", "genetic", "nn"],
                          format_func=lambda v: _tsp_ua.get(v, v))

st.markdown('<hr style="border-color:#161d2e;margin:14px 0">', unsafe_allow_html=True)

st.markdown('<div style="font-size:.61rem;font-weight:700;letter-
spacing:1.2px;color:#475569;text-transform:uppercase;margin-bottom:5px"> 🏠 Адреса
складу</div>', unsafe_allow_html=True)
dc1, dc2 = st.columns([4, 1])
with dc1:
    depot_txt = st.text_input("dep_txt",
                              placeholder=f"Вулиця в {city.split(',')[0]}...",

```

Продовження додатку А

```

        label_visibility="collapsed")

with dc2:
    dep_go = st.button("➡", key="dep_go")
    if dep_go and depot_txt.strip():
        with st.spinner("Геокодування..."):
            r = forward_geocode(depot_txt.strip(), city)
            if r:
                st.session_state.depot = r
                st.session_state.opt_results = None
                st.session_state.opt_vrp_results = None
                st.rerun()
            else:
                st.error(f"Не знайдено у {city}")

if st.session_state.depot:
    dep = st.session_state.depot
    st.markdown(f"""
    <div class="delivery-item depot-item">
    <div class="di-num depot">C</div>
    <div>
    <div class="di-addr">{dep.address[:65]} {"..." if len(dep.address)>65 else ""}</div>
    <div class="di-coords">{dep.lat:.5f}, {dep.lon:.5f}</div>
    </div>
    </div>""", unsafe_allow_html=True)
    if st.button("✕ Видалити склад", key="clr_dep"):
        st.session_state.depot = None
        st.session_state.opt_results = None
        st.session_state.opt_vrp_results = None
        st.rerun()

st.markdown('<hr style="border-color:#161d2e;margin:12px 0">', unsafe_allow_html=True)

```

Продовження додатку А

```

st.markdown('<div style="font-size:.61rem;font-weight:700;letter-
spacing:1.2px;color:#475569;text-transform:uppercase;margin-bottom:5px"> 📌 Додати точку
доставки</div>', unsafe_allow_html=True)
sc1, sc2 = st.columns([4, 1])
with sc1:
    stop_txt = st.text_input("stop_txt",
                             placeholder=f"Адреса у {city.split(',')[0]}...",
                             label_visibility="collapsed")
with sc2:
    stop_go = st.button("➡", key="stop_go")
if stop_go and stop_txt.strip():
    with st.spinner("Геокодування..."):
        r = forward_geocode(stop_txt.strip(), city)
    if r:
        st.session_state.pending_stop = r
        st.rerun()
    else:
        st.error(f"Не знайдено у {city}")

st.markdown('<hr style="border-color:#161d2e;margin:12px 0">', unsafe_allow_html=True)

n_stops = len(st.session_state.stops)
st.markdown(
    f"<div style="font-size:.61rem;font-weight:700;letter-spacing:1.2px;"
    f"color:#475569;text-transform:uppercase;margin-bottom:7px">'
    f" 📄 Обрані точки ({n_stops})</div>",
    unsafe_allow_html=True)

if n_stops == 0:
    st.markdown('<div style="font-size:.74rem;color:#374151;padding:4px 0">Поки що немає

```

Продовження додатку А

```

зупинок.</div>',
        unsafe_allow_html=True)
else:
    for i, stop in enumerate(st.session_state.stops):
        src = "📍" if stop.source == "map_click" else "✏️"
        short = stop.address[:52] + ("..." if len(stop.address) > 52 else "")
        st.markdown(f"""
<div class="delivery-item">
    <div class="di-num">{i+1}</div>
    <div style="flex:1">
        <div class="di-addr">{src} {short}</div>
        <div class="di-coords">{stop.lat:.5f}, {stop.lon:.5f} · ⚖️ {stop.weight_kg:.2f}
кг</div>
    </div>
</div>""", unsafe_allow_html=True)
    if st.button("✕", key=f"rm_{i}", help=f"Видалити зупинку #{i+1}"):
        st.session_state.stops.pop(i)
        st.session_state.opt_results = None
        st.session_state.opt_vrp_results = None
        st.rerun()

st.markdown('<hr style="border-color:#161d2e;margin:12px 0">', unsafe_allow_html=True)

# — Run button

```

```

can_run = st.session_state.depot is not None and len(st.session_state.stops) >= 1
run_btn = st.button(
    "🚀 Оптимізувати маршрути",
    disabled=not can_run,
    use_container_width=True,

```


Продовження додатку А

```

        help="Спочатку додайте склад + хоча б 1 зупинку" if not can_run else "Запустити
оптимізацію маршруту",
    )

```

```

st.markdown("""
<div style="font-size:.63rem;color:#1e293b;line-height:1.8;margin-top:8px">
    <b style="color:#334155">Алгоритми</b><br>Dijkstra · Christofides · 2-opt ·
Генетичний<br>
    <b style="color:#334155">Геокодер</b><br>Nominatim (прив'язано до міста)<br>
    <b style="color:#334155">Дані</b><br>OpenStreetMap / OSMnx
</div>""", unsafe_allow_html=True)

```

```

#

```

```

# MAIN CONTENT

```

```

#

```

```

city = st.session_state.city
depot = st.session_state.depot
stops = st.session_state.stops
fleet = st.session_state.fleet

```

```

st.markdown(f"""
<div class="page-header">
    <div style="font-size:2.5rem"> 📦 </div>
    <div>
        <h1>DeliveryIQ · Оптимізатор маршрутів</h1>
    </div>
</div>

```

Продовження додатку А

<p>Маршрутизація з прив'язкою до міста · Маркери по кліку · Реальні OSM-дані · 3
режими пересування</p>

```
<div class="city-pill"> 📍 Прив'язано до: {city}</div>
</div>
</div>""", unsafe_allow_html=True)
```

```
map_tab, fleet_tab, pkg_tab = st.tabs(["🗺️ Карта та оптимізація", "🚚 Налаштування флоту",
"📦 Посилки"])
```

```
#
```

```
# MAP & OPTIMIZE TAB
```

```
#
```

```
with map_tab:
```

```
no_results = (
    st.session_state.opt_results is None
    and st.session_state.opt_vrp_results is None
)
if no_results:
    st.markdown('<div class="sec-head"> 🗺️ Карта координатора – Сформуйте список  
доставки</div>',
                unsafe_allow_html=True)
```

```
tc_1, tc_r = st.columns([1.2, 3])
```

```
with tc_1:
```

```
    click_on = st.toggle(
```

Продовження додатку А

```

    "📍 Режим кліку по карті",
    value=st.session_state.click_mode,
    key="click_toggle_main",
    help="Коли УВІМКНЕНО, клік по карті додає склад або точку доставки.",
)
if click_on != st.session_state.click_mode:
    st.session_state.click_mode = click_on
    st.rerun()
with tc_r:
    if st.session_state.click_mode:
        target_hint = "розташування складу" if depot is None else "точку доставки"
        st.markdown(f"""
        <div class="click-active" style="text-align:left;padding:10px 18px;margin:4px 0">
            ● <b>УВІМКНЕНО</b> – наступний клік по карті додасть
        <b>{target_hint}</b>.
            Координати автоматично перетворюються на адресу.
        </div>""", unsafe_allow_html=True)
    else:
        st.markdown("""
        <div style="background:#f1f5f9;border:1px solid #e2e8f0;
            border-left:4px solid #94a3b8;border-radius:8px;
            padding:10px 18px;color:#475569;font-size:.83rem;
            margin:4px 0;text-align:left">
            ○ <b>ВИМКНЕНО</b> – увімкніть перемикач, щоб клацанням розміщувати
маркери на карті.
        </div>""", unsafe_allow_html=True)

if not st.session_state.click_mode and depot is None:
    st.markdown("""
    <div class="info-box">
        📍 Задайте <b>склад</b> у бічній панелі або увімкніть <b>Режим кліку по

```

Продовження додатку А

карті

вище, щоб розмістити його на карті, потім додайте точки доставки.

```
</div>""", unsafe_allow_html=True)
```

```
sel_map = build_selection_map(city, depot, stops)
```

```
map_output = st_folium(
    sel_map,
    width="100%",
    height=510,
    returned_objects=["last_clicked"],
    key="coord_map",
)
```

```
if (
    st.session_state.click_mode
    and map_output
    and map_output.get("last_clicked")
):
    raw = map_output["last_clicked"]
    ck = (round(raw["lat"], 5), round(raw["lng"], 5))
    if ck != st.session_state.last_click:
        st.session_state.last_click = ck
        with st.spinner("Зворотнє геокодування..."):
            new = reverse_geocode(raw["lat"], raw["lng"])
            if new:
                if st.session_state.depot is None:
                    new.source = "map_click"
                    st.session_state.depot = new
                    st.session_state.opt_results = None
                    st.session_state.opt_vrp_results = None
                    st.rerun()
```

Продовження додатку А

```

else:
    st.session_state.pending_stop = new
    st.rerun()

# — Run optimisation

```

```

if run_btn and can_run:
    st.markdown('<div class="sec-head"> ⌚ Виконання оптимізації</div>',
unsafe_allow_html=True)
    if len(st.session_state.fleet) >= 2:
        # VRP mode – multi-vehicle fleet
        try:
            vrp_routes, mg, vrp_warnings = run_vrp_optimization(
                depot, stops, st.session_state.fleet, radius, tsp_method
            )
            st.session_state.opt_vrp_results = vrp_routes
            st.session_state.opt_graphs = mg
            st.session_state.opt_vrp_warnings = vrp_warnings
            st.session_state.opt_results = None
            st.rerun()
        except ValueError as e:
            st.error(f"Помилка конфігурації флоту: {e}")
        except Exception as e:
            st.error(f"VRP-оптимізація не вдалася: {e}")
            st.exception(e)
    else:
        # Single-vehicle mode – existing 3-mode comparison
        try:
            res, mg, warnings, car_unreachable = run_optimization(
                depot, stops, city, radius, tsp_method
            )

```

Продовження додатку А

```

    st.session_state.opt_results      = res
    st.session_state.opt_graphs      = mg
    st.session_state.opt_warnings    = warnings
    st.session_state.opt_car_unreachable = car_unreachable
    st.session_state.opt_vrp_results  = None
    st.rerun()
except Exception as e:
    st.error(f"Оптимізація не вдалася: {e}")
    st.exception(e)

# — Single-vehicle results dashboard


---


if st.session_state.opt_results is not None:
    results      = st.session_state.opt_results
    mode_graphs  = st.session_state.opt_graphs
    warnings     = st.session_state.get("opt_warnings", [])
    car_unreachable_notes = st.session_state.get("opt_car_unreachable", [])
    best_mode    = min(results, key=lambda m: results[m].total_time_s)

    if warnings:
        st.markdown('<div class="sec-head"> ⚠ Попередження зв'язності</div>',
                     unsafe_allow_html=True)
        for w in warnings:
            st.markdown(f'<div class="warn-box">{w}</div>', unsafe_allow_html=True)

    st.markdown('<div class="sec-head"> 🇮🇹 Порівняння режимів</div>',
                unsafe_allow_html=True)
    cols = st.columns(3)
    for col, mode in zip(cols, ["drive", "bike", "walk"]):
        res = results[mode]
        meta = MODE_META[mode]

```

Продовження додатку А

```

win = mode == best_mode
tdist = sum(l.distance_m for l in res.legs)
badge = '<div class="winner-badge"> ⚡ Найефективніший</div>' if win else ""
col.markdown(f"""
<div class="metric-card {'winner' if win else ''}>
  <div class="m-icon">{meta['icon']}</div>
  <div class="m-mode">{meta['label']}</div>
  <div class="m-time">{hms(res.total_time_s)}</div>
  <div class="m-sub">{tdist/1000:.1f} км · {SPEED_КМН[mode]:.0f} км/год ·
{len(stops)} зупинок</div>
  {badge}
</div>""", unsafe_allow_html=True)

st.markdown('<div class="sec-head"> 🗺 Оптимізовані маршрути</div>',
unsafe_allow_html=True)
st.markdown("""
<div class="info-box">
  <b>Керування шарами</b> (зверху-зліва) перемикає маршрути Авто / Велосипед /
Пішки.
  ● Зелений = Склад · Червоні номери = Точки доставки в
оптимальному порядку.
</div>""", unsafe_allow_html=True)
rmap = build_result_map(mode_graphs, results, depot, stops)
st_folium(rmap, width="100%", height=530, returned_objects=[], key="result_map")

st.markdown('<div class="sec-head"> 📋 Детальний розклад зупинок</div>',
unsafe_allow_html=True)
t1, t2, t3 = st.tabs(["🚗 Авто", "🚲 Велосипед", "🚶 Пішки"])
for tab, mode in zip([t1, t2, t3], ["drive", "bike", "walk"]):
  with tab:
    res = results[mode]
```

Продовження додатку А

```

if mode == "drive" and car_unreachable_notes:
    st.markdown(
        '<div class="warn-box"><strong> ⚠ Недоступно для авто (останній
метр)</strong><br>'
        + "<br>".join(car_unreachable_notes)
        + "</div>",
        unsafe_allow_html=True,
    )
tdist = sum(l.distance_m for l in res.legs)
c1, c2, c3 = st.columns(3)
c1.metric("Загальний час", hms(res.total_time_s))
c2.metric("Загальна відстань", f'{tdist/1000:.2f} км')
c3.metric("Середня швидкість", f'{SPEED_KMH[mode]:.0f} км/год')
st.markdown(render_stop_table(res), unsafe_allow_html=True)

st.markdown('<div style="height:14px"></div>', unsafe_allow_html=True)
ca, cb = st.columns([3, 1])
with ca:
    st.markdown("""
<div class="warn-box">
 Додайте ще зупинки через бічну панель або клік по карті, потім натисніть
<b>Оптимізувати маршрути</b> знову. OSM-мережа кешується автоматично.
</div>""", unsafe_allow_html=True)
with cb:
    if st.button("🧼 Очистити та почати знову", key="clear_single",
use_container_width=True):
        st.session_state.update(
            opt_results=None, opt_graphs=None, opt_warnings=[],
            opt_car_unreachable=[], opt_vrp_results=None, opt_vrp_warnings=[],
        )
        st.rerun()

```


Продовження додатку А

```
# — VRP results dashboard

elif st.session_state.opt_vrp_results is not None:
    vrp_routes = st.session_state.opt_vrp_results
    mode_graphs = st.session_state.opt_graphs
    vrp_warnings = st.session_state.get("opt_vrp_warnings", [])

    if vrp_warnings:
        st.markdown('<div class="sec-head"> ⚠ Попередження маршрутизації</div>',
                    unsafe_allow_html=True)
        for w in vrp_warnings:
            st.markdown(f'<div class="warn-box">{w}</div>', unsafe_allow_html=True)

    # Per-vehicle summary cards
    st.markdown('<div class="sec-head"> 🚚 Підсумок флоту</div>',
                unsafe_allow_html=True)

    mode_icons = {"drive": " 🚗 ", "bike": " 🚲 ", "walk": " 🚶 "}
    mode_ua_map = {"drive": "авто", "bike": "велосипед", "walk": "пішки"}
    n_cols = min(len(vrp_routes), 4)
    vcols = st.columns(n_cols)
    for col, vr in zip(vcols, vrp_routes):
        icon = mode_icons.get(vr.vehicle.mode, " 🚚 ")
        n_stops = len(vr.stops)
        mode_ua = mode_ua_map.get(vr.vehicle.mode, vr.vehicle.mode)
        col.markdown(f"""
<div class="metric-card" style="border-top:4px solid {vr.vehicle.color}">
  <div class="m-icon">{icon}</div>
  <div class="m-mode">{vr.vehicle.name}</div>
  <div class="m-time">{hms(vr.total_time_s)}</div>
  <div class="m-sub">
```


Продовження додатку А

```

c2.metric("Загальна відстань", f"{tdist/1000:.2f} км")
c3.metric("Режим", mode_ua_map.get(vr.vehicle.mode,
vr.vehicle.mode).capitalize())
if vr.legs:
    # Reuse existing render_stop_table by wrapping in a ModeResult-like object
    pseudo = ModeResult(vr.vehicle.mode, vr.tsp_route, vr.full_route,
                        vr.total_time_s, vr.legs)
    st.markdown(render_stop_table(pseudo), unsafe_allow_html=True)

st.markdown('<div style="height:14px"></div>', unsafe_allow_html=True)
ca, cb = st.columns([3, 1])
with ca:
    st.markdown("""
<div class="warn-box">
 Додайте ще зупинки через бічну панель або налаштуйте флот у
<b>Налаштування флоту</b>, потім натисніть <b>Оптимізувати маршрути</b>
знову.
</div>""", unsafe_allow_html=True)
with cb:
    if st.button("🗑 Очистити та почати знову", key="clear_vrp",
use_container_width=True):
        st.session_state.update(
            opt_results=None, opt_graphs=None, opt_warnings=[],
            opt_car_unreachable=[], opt_vrp_results=None, opt_vrp_warnings=[],
        )
        st.rerun()

#

```

```

# FLEET SETTINGS TAB

```

Продовження додатку А

#

```

with fleet_tab:
    st.markdown('<div class="sec-head">🚚 Конфігурація флоту</div>',
unsafe_allow_html=True)

    n_vehicles = len(fleet)
    mode_badge = {"drive": "🚗 Авто", "bike": "🚲 Велосипед", "walk": "🚶 Пішки"}
    if n_vehicles >= 2:
        st.markdown(
            f'<div class="info-box">У флоті <b>{n_vehicles}</b> ТЗ</b> – '
            f'<b>Оптимізація маршрутів</b> використає <b>режим багатьох ТЗ</b> (VRP)</b>.</div>',
            unsafe_allow_html=True,
        )
    else:
        st.markdown(
            '<div class="info-box">У флоті <b>1</b> ТЗ</b> – '
            '<b>Оптимізація маршрутів</b> використовує одиничний режим (порівняння 3 способів). '
            'Додайте другий ТЗ, щоб увімкнути VRP.</div>',
            unsafe_allow_html=True,
        )

# Edit existing vehicles
st.markdown("***Редагувати транспорт.**")
updated_fleet = list(fleet)
changed = False
for i, v in enumerate(fleet):

```

Продовження додатку А

```

c1, c2, c3, c4 = st.columns([2, 2, 1, 0.4])
with c1:
    new_name = st.text_input("Назва", value=v.name, key=f"v_name_{i}",
                              label_visibility="collapsed")
with c2:
    mode_opts = ["drive", "bike", "walk"]
    _mode_ua = {"drive": " 🚗 Авто", "bike": " 🚲 Велосипед", "walk": " 🚶 Пішки"}
    new_mode = st.selectbox("Режим", mode_opts,
                             index=mode_opts.index(v.mode),
                             format_func=lambda v: _mode_ua.get(v, v),
                             key=f"v_mode_{i}",
                             label_visibility="collapsed")
with c3:
    new_cap = st.number_input("Вантаж. (кг)", min_value=1.0, max_value=10000.0,
                              value=v.capacity_kg, step=10.0,
                              key=f"v_cap_{i}",
                              label_visibility="collapsed")
with c4:
    rm = st.button("X", key=f"v_rm_{i}",
                   disabled=(len(fleet) <= 1),
                   help="Видалити цей транспорт")

if rm:
    updated_fleet.pop(i)
    # Reassign colours in order
    for j, u in enumerate(updated_fleet):
        updated_fleet[j] = Vehicle(u.name, u.mode, u.capacity_kg,
                                    VEHICLE_COLORS[j % len(VEHICLE_COLORS)])
    st.session_state.fleet = updated_fleet
    st.session_state.opt_vrp_results = None
    st.session_state.opt_results = None

```

Продовження додатку А

```

st.rerun()

if new_name != v.name or new_mode != v.mode or float(new_cap) != v.capacity_kg:
    updated_fleet[i] = Vehicle(new_name, new_mode, float(new_cap),
                               VEHICLE_COLORS[i % len(VEHICLE_COLORS)])
    changed = True

if changed:
    st.session_state.fleet = updated_fleet
    st.session_state.opt_vrp_results = None
    st.session_state.opt_results = None

st.markdown('<div style="height:8px"></div>', unsafe_allow_html=True)
col_add, _ = st.columns([1, 3])
with col_add:
    if st.button("+ Додати транспорт", use_container_width=True):
        new_idx = len(st.session_state.fleet)
        st.session_state.fleet.append(
            Vehicle(f"Транспорт {new_idx + 1}", "drive", 500.0,
                   VEHICLE_COLORS[new_idx % len(VEHICLE_COLORS)])
        )
        st.session_state.opt_vrp_results = None
        st.rerun()

# Fleet summary table
if fleet:
    st.markdown('<div style="height:16px"></div>', unsafe_allow_html=True)
    st.markdown("***Поточний флот:**")
    header = "| # | Транспорт | Режим | Вантажомісткість (кг) | Колір |"
    sep = "|---|---|---|---|---|"
    rows = [header, sep]

```

Продовження додатку А

```

for i, v in enumerate(st.session_state.fleet, 1):
    swatch = f'<span style="color:{v.color}">■</span>'
    rows.append(f'| {i} | {v.name} | {mode_badge.get(v.mode, v.mode)} | {v.capacity_kg:.0f} | {swatch} |')
st.markdown("\n".join(rows), unsafe_allow_html=True)

#
=====

# PACKAGES TAB
#
=====

with pkg_tab:
    st.markdown('<div class="sec-head"> 📦 Менеджер посилок</div>',
unsafe_allow_html=True)

    counts = _pkg_db.count_by_status()
    pc1, pc2, pc3 = st.columns(3)
    pc1.metric(" ⌚ Очікує", counts[DeliveryStatus.PENDING])
    pc2.metric(" 🚚 В дорозі", counts[DeliveryStatus.IN_TRANSIT])
    pc3.metric(" ✅ Доставлено", counts[DeliveryStatus.DELIVERED])

    st.markdown('<div style="height:8px"></div>', unsafe_allow_html=True)

# — Add new package form
=====

with st.expander(" + Додати нову посилку", expanded=True):
    with st.form("add_package_form", clear_on_submit=True):

```

Продовження додатку А

```

fa1, fa2 = st.columns([3, 1])
with fa1:
    new_addr = st.text_input(
        "Адреса доставки",
        placeholder=f"Вулиця у {st.session_state.city.split(',')[0]}...",
    )
with fa2:
    new_weight = st.number_input(
        "Вага (кг)",
        min_value=0.01,
        max_value=10_000.0,
        value=1.0,
        step=0.1,
        format="%.2f",
    )
submitted = st.form_submit_button("Зберегти посилку", use_container_width=True)

if submitted:
    if not new_addr.strip():
        st.error("Адреса не може бути порожньою.")
    else:
        # Attempt geocoding so coordinates are stored immediately
        with st.spinner("Геокодування адреси..."):
            geo = forward_geocode(new_addr.strip(), st.session_state.city)
        if geo:
            pkg = _pkg_db.add_package(
                address=geo.address,
                weight_kg=float(new_weight),
                lat=geo.lat,
                lon=geo.lon,
            )

```


Продовження додатку А

```

        st.success(f"Посилку збережено – {geo.address[:60]} ({new_weight:.2f} кг)")
    else:
        # Save with raw address even if geocoding failed
        pkg = _pkg_db.add_package(
            address=new_addr.strip(),
            weight_kg=float(new_weight),
        )
        st.warning(
            f"Адресу не знайдено у {st.session_state.city} – "
            "посилку збережено без координат."
        )
    st.rerun()

st.markdown('<div style="height:4px"></div>', unsafe_allow_html=True)

# — Filter controls

```

```

st.markdown("***Фільтр за статусом:***")
_status_labels = {
    "Yci":      None,
    "🕒 Очікує":  DeliveryStatus.PENDING,
    "🚚 В дорозі": DeliveryStatus.IN_TRANSIT,
    "✅ Доставлено": DeliveryStatus.DELIVERED,
}
filter_choice = st.radio(
    "pkg_filter",
    list(_status_labels.keys()),
    horizontal=True,
    label_visibility="collapsed",
)
selected_status = _status_labels[filter_choice]

```

Продовження додатку А

— Package list

```

packages = (
    _pkg_db.get_all()
    if selected_status is None
    else _pkg_db.get_by_status(selected_status)
)

if not packages:
    st.markdown(
        '<div style="font-size:.82rem;color:#64748b;padding:12px 0">Посилонк не
знайдено.</div>',
        unsafe_allow_html=True,
    )
else:
    for pkg in packages:
        col_info, col_weight, col_status, col_actions = st.columns([4, 1.6, 2, 2])

        with col_info:
            short_addr = pkg.address[:55] + ("..." if len(pkg.address) > 55 else "")
            coords_txt = (
                f'{pkg.lat:.5f}, {pkg.lon:.5f}'
                if pkg.lat is not None
                else "координати невідомі"
            )
            st.markdown(
                f'""<div style="padding:6px 0">
                    <div style="font-weight:600;font-size:.88rem">{short_addr}</div>
                    <div style="font-size:.75rem;color:#64748b">

```

Продовження додатку А

```

     {coords_txt} &nbsp;   ·&nbsp;   

     {pkg.created_at.strftime('%d.%m.%Y %H:%M')} UTC
</div>
</div>""",
unsafe_allow_html=True,
)

```

with col_weight:

```

new_weight = st.number_input(
    "Bara (кг)",
    min_value=0.01,
    max_value=10_000.0,
    value=float(pkg.weight_kg),
    step=0.1,
    format="%.2f",
    key=f"pkg_weight_{pkg.id}",
    label_visibility="collapsed",
)

if abs(float(new_weight) - float(pkg.weight_kg)) > 1e-6:
    if st.button(
        " Зберегти вагу",
        key=f"pkg_save_weight_{pkg.id}",
        use_container_width=True,
    ):
        _pkg_db.set_weight(pkg.id, float(new_weight))
        st.rerun()

```

with col_status:

```

st.markdown(
    f<div style="margin-top:8px;font-size:.82rem;font-weight:600;'
    fcolor:{pkg.status_color}">{pkg.status_label}</div>',

```

Продовження додатку А

```

        unsafe_allow_html=True,
    )
    # Status change selector
    _opts = [s.value for s in DeliveryStatus]
    _opts_labels_map = {
        "pending": " ⏰ Очікує",
        "in_transit": " 🚚 В дорозі",
        "delivered": " ✅ Доставлено",
    }
    new_status_val = st.selectbox(
        "Змінити статус",
        _opts,
        index=_opts.index(pkg.status.value),
        format_func=lambda v: _opts_labels_map.get(v, v),
        key=f"pkg_status_{pkg.id}",
        label_visibility="collapsed",
    )
    if new_status_val != pkg.status.value:
        _pkg_db.set_status(pkg.id, DeliveryStatus(new_status_val))
        st.rerun()

with col_actions:
    st.markdown('<div style="height:8px"></div>', unsafe_allow_html=True)
    # Add to delivery stops (only if geocoded and not already delivered)
    can_dispatch = (
        pkg.lat is not None
        and pkg.status != DeliveryStatus.DELIVERED
    )
    if st.button(
        "➔ Додати у маршрут",
        key=f"pkg_dispatch_{pkg.id}",

```

Продовження додатку А

```

disabled=not can_dispatch,
help=(
    "Додати цю адресу як точку доставки"
    if can_dispatch
    else "Немає координат – геокодування не вдалось, або вже доставлено"
),
use_container_width=True,
):
    new_stop = DeliveryStop(
        address=pkg.address,
        lat=pkg.lat,
        lon=pkg.lon,
        source="typed",
        weight_kg=pkg.weight_kg,
    )
    # Avoid duplicate addresses
    existing_addrs = [s.address for s in st.session_state.stops]
    if pkg.address not in existing_addrs:
        st.session_state.stops.append(new_stop)
        st.session_state.opt_results = None
        st.session_state.opt_vrp_results = None
        _pkg_db.set_status(pkg.id, DeliveryStatus.IN_TRANSIT)
        st.success(f"Додано у маршрут: {pkg.address[:50]}")
        st.rerun()
    else:
        st.warning("Ця адреса вже є у списку доставки.")

if st.button(
    "🗑 Видалити",
    key=f"pkg_delete_{pkg.id}",
    use_container_width=True,

```

Продовження додатку А

```

        type="secondary",
    ):
        _pkg_db.delete_package(pkg.id)
        st.rerun()

    st.markdown(
        '<hr style="border:none;border-top:1px solid #1e293b;margin:4px 0">',
        unsafe_allow_html=True,
    )
    """Address-to-(lat, lon) geocoding via Nominatim (geopy)."""

import logging
from dataclasses import dataclass

from geopy.geocoders import Nominatim
from geopy.exc import GeocoderTimedOut, GeocoderServiceError

logger = logging.getLogger(__name__)

_geolocator = Nominatim(user_agent="delivery_optimizer_v1")

@dataclass
class Location:
    address: str
    lat: float
    lon: float
    node_id: int | None = None      # set after snapping to OSM graph

```

Продовження додатку А

```

def geocode(address: str, retries: int = 3) -> tuple[float, float]:
    """Geocode address to (lat, lon). Retries on timeout."""
    for attempt in range(1, retries + 1):
        try:
            result = _geolocator.geocode(address, timeout=10)
            if result is None:
                raise ValueError(f'Nominatim returned no result for: '{address}''')
            logger.info("  Geocoded '%s' → (%.5f, %.5f)", address, result.latitude,
result.longitude)
            return result.latitude, result.longitude
        except GeocoderTimedOut:
            logger.warning("  Geocoder timed-out (attempt %d/%d) for '%s'", attempt,
retries, address)
            raise GeocoderServiceError(f'Geocoder failed after {retries} retries for
'{address}''')

def geocode_all(addresses: list[str]) -> list[Location]:
    """Geocode a list of addresses into Location objects."""
    locations: list[Location] = []
    for addr in addresses:
        lat, lon = geocode(addr)
        locations.append(Location(address=addr, lat=lat, lon=lon))
    return locations
"""

```

Продовження додатку А

OSM street-network download and pre-processing. Requires OSMnx ≥ 2.1 .

Each edge in the 'all' graph carries OSM tags describing which modes may use it: highway (road/path type), access, motor_vehicle, bicycle, foot, oneway, oneway:bicycle, cycleway, and OSMnx's synthetic `reversed` flag.

PENALTY (1e9 s) marks impassable edges. It matches route_solver.PENALTY so audit_reachability() flags any pair with cost $\geq$ PENALTY. A smaller sentinel would let Dijkstra route through "impassable" edges without warning.

"""

```
from __future__ import annotations
```

```
import logging
```

```
import math
```

```
from typing import Optional
```

```
import networkx as nx
```

```
import osmnx as ox
```

```
logger = logging.getLogger(__name__)
```

```
logger.info("graph_builder loaded – OSMnx version: %s", ox.__version__)
```

```
# Mode speeds (km/h)
```

```
SPEED_KMH: dict[str, float] = {
```

```
    "drive": 30.0,
```


Продовження додатку А

```
"bike": 15.0,  
"walk": 5.0,  
}
```

Impassable edges and unreachable pairs. Must match route_solver.PENALTY.

PENALTY: float = 1e9

OSM tags to retain on every edge

_REQUIRED_TAGS: list[str] = [

"access",

"highway",

"junction",

"lanes",

"maxspeed",

"name",

"oneway",

"service",

"width",

"bicycle",

"cycleway",

"cycleway:left",

"cycleway:right",

"cycleway:both",

"foot",

"motor_vehicle",

"oneway:bicycle",

Продовження додатку А

```

    "bicycle:oneway",
    "surface",
]

# --- OSM tag reading ---

def _osm_tag(data: dict, key: str, default: str = "") -> str:
    """Read one OSM tag as a normalised lowercase string (first item if a list)."""
    val = data.get(key, default)
    if val is None or val == "":
        return default
    if isinstance(val, list):
        items = [str(v).strip().lower() for v in val if v is not None and v != ""]
        return items[0] if items else default
    return str(val).strip().lower()

def _osm_tag_in(data: dict, key: str, values: frozenset) -> bool:
    """True if ANY value of an OSM tag (scalar or list) is a member of *values*."""
    raw = data.get(key)
    if raw is None:
        return False
    if isinstance(raw, list):
        return any(
            str(v).strip().lower() in values
            for v in raw

```

Продовження додатку А

```

        if v is not None
    )
    return str(raw).strip().lower() in values

def _bike_contraflow_allowed(data: dict) -> bool:
    """True if a bicycle may travel against a one-way car restriction here."""
    _OPPOSITE = frozenset({"opposite", "opposite_lane", "opposite_track"})
    return (
        _osm_tag(data, "oneway:bicycle") == "no" or
        _osm_tag(data, "bicycle") == "two_way" or
        _osm_tag(data, "bicycle:oneway") == "no" or
        _osm_tag_in(data, "cycleway", _OPPOSITE) or
        _osm_tag_in(data, "cycleway:left", _OPPOSITE) or
        _osm_tag_in(data, "cycleway:right", _OPPOSITE) or
        _osm_tag_in(data, "cycleway:both", _OPPOSITE)
    )

# --- Per-mode travel time ---

_DRIVE_BLOCKED_HW: frozenset = frozenset({
    "footway", "path", "pedestrian", "steps", "corridor", "elevator", "escalator",
    "bridleway",
})

_MOTOR_NO: frozenset = frozenset({"no", "private", "destination"})
_BIKE_ALLOWED: frozenset = frozenset({"yes", "designated", "permissive",

```

Продовження додатку А

```

"official", "use_sidepath"})

_BIKE_FORBIDDEN: frozenset = frozenset({"no", "dismount"})
_FOOT_NO: frozenset = frozenset({"no", "private"})
_STEPS_WALK_MULTIPLIER: float = 0.5

def _compute_travel_time(data: dict, mode: str, speed_ms: float) -> float:
    """Travel time (s) for one directed edge under mode; PENALTY if blocked."""
    raw_len = data.get("length", None)
    length_m = max(float(raw_len) if raw_len is not None else 1.0, 1.0)

    hw = _osm_tag(data, "highway")
    access = _osm_tag(data, "access")
    bicycle = _osm_tag(data, "bicycle")
    mv = _osm_tag(data, "motor_vehicle")
    foot = _osm_tag(data, "foot")
    oneway = _osm_tag(data, "oneway") in {"yes", "true", "1", "-1"}
    is_reversed = bool(data.get("reversed", False))

    if mode == "drive":
        if _osm_tag_in(data, "highway", _DRIVE_BLOCKED_HW):
            return PENALTY
        if mv in _MOTOR_NO:
            return PENALTY
        if access in _MOTOR_NO and mv not in {"yes", "permissive", "designated"}:
            return PENALTY
        return length_m / speed_ms

```

Продовження додатку А

```
elif mode == "bike":
    if bicycle in _BIKE_FORBIDDEN:
        return PENALTY
    if hw == "cycleway" or bicycle in _BIKE_ALLOWED:
        return length_m / speed_ms
    if hw in {"path", "bridleway"}:
        return length_m / speed_ms
    if hw == "steps":
        return PENALTY
    if oneway and is_reversed and not _bike_contraflow_allowed(data):
        return PENALTY
    return length_m / speed_ms

elif mode == "walk":
    if foot in _FOOT_NO:
        return PENALTY
    if access in _FOOT_NO and foot not in {"yes", "designated", "permissive"}:
        return PENALTY
    if hw == "steps":
        return length_m / (speed_ms * _STEPS_WALK_MULTIPLIER)
    return length_m / speed_ms

raise ValueError(f'Unknown mode {mode!r}. Expected one of: 'drive', 'bike',
'walk'.')
```

Продовження додатку А

```
# --- OSMnx 2.x wrappers ---
```

```
def _configure_osmnx_tags() -> None:
```

```
    """Retain the tags the modal filter needs. Call before any ox.graph_from_*."""
```

```
    current: list[str] = list(ox.settings.useful_tags_way or [])
```

```
    merged = list(dict.fromkeys(current + _REQUIRED_TAGS))
```

```
    ox.settings.useful_tags_way = merged
```

```
    logger.debug("Tag config: %d tags retained in useful_tags_way.", len(merged))
```

```
def _largest_strongly_connected_component(G: nx.MultiDiGraph) ->
```

```
nx.MultiDiGraph:
```

```
    """Return the subgraph of the largest strongly connected component."""
```

```
    raw_n = G.number_of_nodes()
```

```
    raw_e = G.number_of_edges()
```

```
    G_lsc = ox.truncate.largest_component(G, strongly=True)
```

```
    lsc_n = G_lsc.number_of_nodes()
```

```
    lsc_e = G_lsc.number_of_edges()
```

```
    dropped_n = raw_n - lsc_n
```

```
    dropped_e = raw_e - lsc_e
```

```
    if dropped_n > 0:
```

```
        logger.warning(
```

```
            "SCC pruning: removed %d node(s) and %d edge(s) (%d->%d nodes, %d->%d edges).",
```

Продовження додатку А

```

        dropped_n, dropped_e, raw_n, lscn_n, raw_e, lscn_e,
    )
else:
    logger.info(
        "SCC pruning: graph already strongly connected (%d nodes, %d edges).",
        lscn_n, lscn_e,
    )
return G_lscn

# --- Public API ---

def get_network_from_point(lat: float, lon: float, dist: int = 3_000) ->
nx.MultiDiGraph:
    """
    Download an OSM 'all' network centred on (lat, lon) and return its LSCC.

    Prefer this over get_network() when you have explicit coordinates, so the
    download is centred on the actual location rather than a Nominatim label.
    dist is the download radius in metres.
    """
    _configure_osmnx_tags()
    logger.info(
        "get_network_from_point: downloading OSM 'all' graph – "
        "lat=%.6f, lon=%.6f, dist=%d m", lat, lon, dist,
    )

```

Продовження додатку А

```

G_raw = ox.graph_from_point((lat, lon), dist=dist, network_type="all",
simplify=True)

logger.info(" Raw graph: %d nodes, %d edges", G_raw.number_of_nodes(),
G_raw.number_of_edges())

if G_raw.number_of_nodes() == 0:
    raise RuntimeError(
        f"OSMnx returned an empty graph for ({lat}, {lon}) at dist={dist} m."
    )

G = _largest_strongly_connected_component(G_raw)

if G.number_of_nodes() == 0:
    raise RuntimeError(
        f"After LSCC pruning the graph for ({lat}, {lon}) is empty. "
        "Try a larger radius."
    )

logger.info(
    " Final graph: %d nodes, %d edges (strongly_connected=%s)",
    G.number_of_nodes(), G.number_of_edges(), nx.is_strongly_connected(G),
)

return G

def get_network(location: str, dist: int = 3_000) -> nx.MultiDiGraph:

```


Продовження додатку А

```

"""
Download an OSM 'all' network centred on location and return its LSCC.

location is a Nominatim-compatible address; dist is the radius in metres.
"""
_configure_osmnx_tags()
logger.info("get_network: downloading OSM 'all' graph – location='%s', dist=%d
m", location, dist)

G_raw = ox.graph_from_address(location, dist=dist, network_type="all",
simplify=True)

logger.info(" Raw graph: %d nodes, %d edges", G_raw.number_of_nodes(),
G_raw.number_of_edges())

if G_raw.number_of_nodes() == 0:
    raise RuntimeError(
        f"OSMnx returned an empty graph for '{location}' at dist={dist} m. "
        "The address may be outside OSM coverage, or the radius is too small."
    )

G = _largest_strongly_connected_component(G_raw)

if G.number_of_nodes() == 0:
    raise RuntimeError(
        f"After LSCC pruning the graph for '{location}' is empty. "
        "Try a different depot address or a larger radius."
    )

```

Продовження додатку А

```

    )

logger.info(
    " Final graph: %d nodes, %d edges (strongly_connected=%s)",
    G.number_of_nodes(), G.number_of_edges(), nx.is_strongly_connected(G),
)
return G

def get_network_from_place(
    place: str,
    which_result: Optional[int] = None,
) -> nx.MultiDiGraph:
    """
    Download an OSM 'all' network for a named place and return its LSCC.

    which_result picks which Nominatim result to use (1-based).
    """
    _configure_osmnx_tags()
    logger.info("get_network_from_place: downloading OSM 'all' graph –
place='%s'", place)

    kwargs: dict = {"network_type": "all", "simplify": True}
    if which_result is not None:
        kwargs["which_result"] = which_result

    G_raw = ox.graph_from_place(place, **kwargs)

```

Продовження додатку А

```

    logger.info(" Raw graph: %d nodes, %d edges", G_raw.number_of_nodes(),
G_raw.number_of_edges())

if G_raw.number_of_nodes() == 0:
    raise RuntimeError(
        f"OSMnx returned an empty graph for '{place}'. "
        "Try a different place name or check Nominatim coverage."
    )

G = _largest_strongly_connected_component(G_raw)

if G.number_of_nodes() == 0:
    raise RuntimeError(f"After LSCC pruning the graph for '{place}' is empty.")

logger.info(
    " Final graph: %d nodes, %d edges (strongly_connected=%s)",
    G.number_of_nodes(), G.number_of_edges(), nx.is_strongly_connected(G),
)
return G

def add_travel_times(G: nx.MultiDiGraph) -> dict[str, nx.MultiDiGraph]:
    """
    Build three mode-specific copies of G, each edge stamped with a
    travel_time attribute (seconds). Returns {"drive", "bike", "walk"}.
    """

```

Продовження додатку А

```

mode_graphs: dict[str, nx.MultiDiGraph] = {}

for mode, speed_kmh in SPEED_KMH.items():
    speed_ms = speed_kmh * 1_000.0 / 3_600.0
    H = G.copy()

    n_passable = 0
    n_impassable = 0
    n_penalised = 0

    for u, v, key, data in H.edges(keys=True, data=True):
        tt = _compute_travel_time(data, mode, speed_ms)

        if tt >= PENALTY:
            n_impassable += 1
        elif mode == "walk" and _osm_tag(data, "highway") == "steps":
            n_penalised += 1
        elif mode == "bike" and bool(data.get("reversed", False)):
            n_penalised += 1
        else:
            n_passable += 1

    H[u][v][key]["travel_time"] = tt

logger.info(
    " [%s] %.1f km/h – edges: %d passable, %d penalised, %d impassable",
    mode, speed_kmh, n_passable, n_penalised, n_impassable,

```

Продовження додатку А

```

    )

    total = H.number_of_edges()
    if n_impassable == total and total > 0:
        logger.warning(
            " [%s] ALL %d edges are impassable – check that
_configure_osmnx_tags() "
            "ran before get_network().", mode, total,
        )

    mode_graphs[mode] = H

return mode_graphs

def add_travel_times_to_single_graph(G: nx.MultiDiGraph) -> nx.MultiDiGraph:
    """
    Add travel_time_drive/bike/walk attributes to every edge of G.
    """
    H = G.copy()
    speed_ms = {
        "drive": SPEED_KMH["drive"] * 1_000.0 / 3_600.0,
        "bike": SPEED_KMH["bike"] * 1_000.0 / 3_600.0,
        "walk": SPEED_KMH["walk"] * 1_000.0 / 3_600.0,
    }
    for u, v, key, data in H.edges(keys=True, data=True):
        H[u][v][key]["travel_time_drive"] = _compute_travel_time(data, "drive",

```

Продовження додатку А

```

speed_ms["drive"])
    H[u][v][key]["travel_time_bike"] = _compute_travel_time(data, "bike",
speed_ms["bike"])
    H[u][v][key]["travel_time_walk"] = _compute_travel_time(data, "walk",
speed_ms["walk"])
    logger.info(
        " Single graph: added travel_time_drive/bike/walk to %d edges.",
H.number_of_edges()
    )
    return H

```

```

def distance_m_between_nodes(
    G: nx.MultiDiGraph,
    node_a: int,
    node_b: int,
) -> float:
    """Return the great-circle distance in metres between two graph nodes."""
    try:
        y1, x1 = G.nodes[node_a]["y"], G.nodes[node_a]["x"]
        y2, x2 = G.nodes[node_b]["y"], G.nodes[node_b]["x"]
    except KeyError as e:
        raise ValueError(f"Node missing x/y: {e}") from e

    try:
        return float(ox.distance.great_circle(y1, x1, y2, x2))
    except (AttributeError, TypeError, ValueError):

```

Продовження додатку А

```

pass

# Haversine fallback
R = 6_371_000.0
phi1, phi2 = math.radians(y1), math.radians(y2)
dphi = math.radians(y2 - y1)
dlam = math.radians(x2 - x1)
a = math.sin(dphi / 2) ** 2 + math.cos(phi1) * math.cos(phi2) * math.sin(dlam /
2) ** 2
return 2 * R * math.asin(math.sqrt(min(1.0, a)))

def _node_car_accessible(
    G: nx.MultiDiGraph,
    node: int,
    *,
    weight: str = "travel_time",
    penalty: float = PENALTY,
) -> bool:
    """True if node has at least one incident edge with weight < penalty."""
    for u, v, key, data in G.edges(node, keys=True, data=True):
        if u != node and v != node:
            continue
        w = data.get(weight)
        if w is not None and isinstance(w, (int, float)) and float(w) < penalty:
            return True
    for u, v, key, data in G.in_edges(node, keys=True, data=True):

```

Продовження додатку А

```

    w = data.get(weight)
    if w is not None and isinstance(w, (int, float)) and float(w) < penalty:
        return True
    return False

def nearest_car_accessible_node(
    G_drive: nx.MultiDiGraph,
    lat: float,
    lon: float,
    *,
    weight: str = "travel_time",
    penalty: float = PENALTY,
) -> int:
    """Nearest car-accessible node to (lat, lon) (incident edge < penalty)."""
    nn = ox.nearest_nodes(G_drive, X=lon, Y=lat)
    if _node_car_accessible(G_drive, nn, weight=weight, penalty=penalty):
        return nn

    # Fallback: scan car-accessible nodes and minimise haversine distance
    best_node: Optional[int] = None
    best_dist: float = float("inf")

    for node in G_drive.nodes():
        if not _node_car_accessible(G_drive, node, weight=weight, penalty=penalty):
            continue
        try:

```


Продовження додатку А

```

    ny, nx_ = G_drive.nodes[node]["y"], G_drive.nodes[node]["x"]
except KeyError:
    continue
try:
    d = float(ox.distance.great_circle(lat, lon, ny, nx_))
except (AttributeError, TypeError, ValueError):
    R = 6_371_000.0
    phi1, phi2 = math.radians(lat), math.radians(ny)
    dphi = math.radians(ny - lat)
    dlam = math.radians(lon - nx_)
    a = math.sin(dphi / 2) ** 2 + math.cos(phi1) * math.cos(phi2) *
math.sin(dlam / 2) ** 2
    d = 2 * R * math.asin(math.sqrt(min(1.0, a)))
    if d < best_dist:
        best_dist = d
        best_node = node

if best_node is None:
    return ox.nearest_nodes(G_drive, X=lon, Y=lat)
return best_node

def nearest_node(G: nx.MultiDiGraph, lat: float, lon: float) -> int:
    """Snap a geocoded (lat, lon) to the nearest OSM node in *G*."""
    return ox.nearest_nodes(G, X=lon, Y=lat)

```

Продовження додатку А

```
def nearest_node_safe(
    G: nx.MultiDiGraph,
    lat: float,
    lon: float,
    *,
    tolerance_m: float = 500.0,
) -> int:
    """
```

Bbox-validated node snapping for map-click coordinates.

Raises ValueError if (lat, lon) is more than tolerance_m outside the graph bbox, so the UI can show a clear warning.

```
    """
```

```
    node_lats = [d["y"] for _, d in G.nodes(data=True)]
```

```
    node_lons = [d["x"] for _, d in G.nodes(data=True)]
```

```
    min_lat, max_lat = min(node_lats), max(node_lats)
```

```
    min_lon, max_lon = min(node_lons), max(node_lons)
```

```
    tol_lat = tolerance_m / 111_000.0
```

```
    tol_lon = tolerance_m / (111_000.0 * math.cos(math.radians(lat)))
```

```
    if not ((min_lat - tol_lat) <= lat <= (max_lat + tol_lat)):
```

```
        raise ValueError(
```

```
            f'Clicked latitude {lat:.5f} is outside the graph bounds "
```

```
            f'({min_lat:.4f} - {max_lat:.4f}). "
```

```
            "Try increasing the network radius or clicking closer to the depot."
```

Продовження додатку А

```

    )
    if not ((min_lon - tol_lon) <= lon <= (max_lon + tol_lon)):
        raise ValueError(
            f"Clicked longitude {lon:.5f} is outside the graph bounds "
            f"({min_lon:.4f} - {max_lon:.4f}). "
            "Try increasing the network radius or clicking closer to the depot."
        )

    return ox.nearest_nodes(G, X=lon, Y=lat)

def graph_summary(G: nx.MultiDiGraph) -> dict:
    """Return key graph statistics for debugging / sidebar display."""
    node_lats = [d["y"] for _, d in G.nodes(data=True)]
    node_lons = [d["x"] for _, d in G.nodes(data=True)]
    return {
        "nodes":      G.number_of_nodes(),
        "edges":      G.number_of_edges(),
        "min_lat":    min(node_lats),
        "max_lat":    max(node_lats),
        "min_lon":    min(node_lons),
        "max_lon":    max(node_lons),
        "strongly_connected": nx.is_strongly_connected(G),
        "osmnx_version":  ox.__version__,
    }
"""

package_db.py

```

Продовження додатку А

Provides the Package dataclass and a SQLite-backed PackageDB for persistent package management. Packages represent physical parcels to be delivered; they carry a weight, a delivery address, and a delivery status.

Usage:

```
from package_db import Package, PackageDB, DeliveryStatus

db = PackageDB()                # opens/creates packages.db
pkg = db.add_package("Shevchenka 1", 2.5) # returns Package
db.set_status(pkg.id, DeliveryStatus.IN_TRANSIT)
pending = db.get_by_status(DeliveryStatus.PENDING)
"""
```

```
from __future__ import annotations
```

```
import sqlite3
```

```
import uuid
```

```
import datetime
```

```
from dataclasses import dataclass, field
```

```
from enum import Enum
```

```
from pathlib import Path
```

```
from typing import Optional
```

```
#
```

Продовження додатку А

```
# STATUS ENUM
```

```
#
```

```
class DeliveryStatus(str, Enum):
```

```
    PENDING    = "pending"    # Waiting to be dispatched
```

```
    IN_TRANSIT = "in_transit" # Currently being delivered
```

```
    DELIVERED  = "delivered"  # Successfully delivered
```

```
#
```

```
# PACKAGE DATACLASS
```

```
#
```

```
@dataclass
```

```
class Package:
```

```
    """A parcel to be delivered."""
```

```
    id: str          # UUID4 hex string
```

```
    address: str      # Delivery address (human-readable)
```

```
    weight_kg: float  # Weight in kilograms
```

```
    status: DeliveryStatus # Current delivery status
```

```
    created_at: datetime.datetime # UTC creation timestamp
```

Продовження додатку А

```
lat:    Optional[float] = None # Geocoded latitude (set on dispatch)
lon:    Optional[float] = None # Geocoded longitude (set on dispatch)
```

```
@property
```

```
def status_label(self) -> str:
```

```
    labels = {
```

```
        DeliveryStatus.PENDING: " ⌚ Очікує",
```

```
        DeliveryStatus.IN_TRANSIT: " 🚚 В дорозі",
```

```
        DeliveryStatus.DELIVERED: " ✅ Доставлено",
```

```
    }
```

```
    return labels.get(self.status, self.status.value)
```

```
@property
```

```
def status_color(self) -> str:
```

```
    colors = {
```

```
        DeliveryStatus.PENDING: "#f59e0b",
```

```
        DeliveryStatus.IN_TRANSIT: "#3b82f6",
```

```
        DeliveryStatus.DELIVERED: "#10b981",
```

```
    }
```

```
    return colors.get(self.status, "#6b7280")
```

```
#
```

```
# DATABASE
```

Продовження додатку А

#

```
_DB_PATH = Path(__file__).parent / "packages.db"
```

```
_CREATE_TABLE = """
```

```
CREATE TABLE IF NOT EXISTS packages (
    id      TEXT  PRIMARY KEY,
    address TEXT  NOT NULL,
    weight_kg REAL NOT NULL,
    status   TEXT  NOT NULL DEFAULT 'pending',
    created_at TEXT NOT NULL,
    lat      REAL,
    lon      REAL
);
"""
```

```
class PackageDB:
```

```
    """Thin wrapper around a SQLite database for Package persistence."""
```

```
    def __init__(self, db_path: Path = _DB_PATH) -> None:
```

```
        self._path = db_path
```

```
        self._bootstrap()
```

```
    # — internal helpers
```

```
def _connect(self) -> sqlite3.Connection:
```

```
    conn = sqlite3.connect(self._path)
```

```
    conn.row_factory = sqlite3.Row
```

```
    return conn
```

```
def _bootstrap(self) -> None:
```

```
    with self._connect() as conn:
```

```
        conn.execute(_CREATE_TABLE)
```

```
@staticmethod
```

```
def _row_to_package(row: sqlite3.Row) -> Package:
```

```
    return Package(
```

```
        id=row["id"],
```

```
        address=row["address"],
```

```
        weight_kg=float(row["weight_kg"]),
```

```
        status=DeliveryStatus(row["status"]),
```

```
        created_at=datetime.datetime.fromisoformat(row["created_at"]),
```

```
        lat=float(row["lat"]) if row["lat"] is not None else None,
```

```
        lon=float(row["lon"]) if row["lon"] is not None else None,
```

```
    )
```

```
# — public API
```

Продовження додатку А

```

def add_package(
    self,
    address: str,
    weight_kg: float,
    lat: Optional[float] = None,
    lon: Optional[float] = None,
) -> Package:
    """Create and persist a new package. Returns the saved Package."""
    pkg = Package(
        id=uuid.uuid4().hex,
        address=address,
        weight_kg=weight_kg,
        status=DeliveryStatus.PENDING,
        created_at=datetime.datetime.utcnow(),
        lat=lat,
        lon=lon,
    )
    with self._connect() as conn:
        conn.execute(
            "INSERT INTO packages (id, address, weight_kg, status, created_at, lat,
lon) "
            "VALUES (?, ?, ?, ?, ?, ?, ?)",
            (pkg.id, pkg.address, pkg.weight_kg, pkg.status.value,
            pkg.created_at.isoformat(), pkg.lat, pkg.lon),
        )
    return pkg

```

Продовження додатку А

```

def get_all(self) -> list[Package]:
    """Return all packages ordered by creation time (newest first)."""
    with self._connect() as conn:
        rows = conn.execute(
            "SELECT * FROM packages ORDER BY created_at DESC"
        ).fetchall()
    return [self._row_to_package(r) for r in rows]

def get_by_status(self, status: DeliveryStatus) -> list[Package]:
    """Return packages filtered by status."""
    with self._connect() as conn:
        rows = conn.execute(
            "SELECT * FROM packages WHERE status = ? ORDER BY created_at
DESC",
            (status.value,),
        ).fetchall()
    return [self._row_to_package(r) for r in rows]

def get_by_id(self, package_id: str) -> Optional[Package]:
    """Return a single package by ID, or None if not found."""
    with self._connect() as conn:
        row = conn.execute(
            "SELECT * FROM packages WHERE id = ?", (package_id,)
        ).fetchone()
    return self._row_to_package(row) if row else None

```

Продовження додатку А

```
def set_status(self, package_id: str, status: DeliveryStatus) -> None:
    """Update the delivery status of a package."""
    with self._connect() as conn:
        conn.execute(
            "UPDATE packages SET status = ? WHERE id = ?",
            (status.value, package_id),
        )

def set_coordinates(self, package_id: str, lat: float, lon: float) -> None:
    """Attach geocoded coordinates to a package."""
    with self._connect() as conn:
        conn.execute(
            "UPDATE packages SET lat = ?, lon = ? WHERE id = ?",
            (lat, lon, package_id),
        )

def set_weight(self, package_id: str, weight_kg: float) -> None:
    """Update the weight of a package."""
    with self._connect() as conn:
        conn.execute(
            "UPDATE packages SET weight_kg = ? WHERE id = ?",
            (float(weight_kg), package_id),
        )

def delete_package(self, package_id: str) -> None:
    """Permanently remove a package from the database."""
    with self._connect() as conn:
```

Продовження додатку А

```

conn.execute("DELETE FROM packages WHERE id = ?", (package_id,))

def count_by_status(self) -> dict[DeliveryStatus, int]:
    """Return a status → count mapping for all packages."""
    with self._connect() as conn:
        rows = conn.execute(
            "SELECT status, COUNT(*) AS n FROM packages GROUP BY status"
        ).fetchall()
        result = {s: 0 for s in DeliveryStatus}
        for row in rows:
            result[DeliveryStatus(row["status"])] = row["n"]
        return result
    """

```

TSP solver for the DeliveryIQ route optimizer.

Notes:

- build_distance_matrix uses nx.shortest_path_length per pair, inserting PENALTY (1e9 s) for missing paths rather than a silent 0.0.
- audit_reachability inspects the matrix and reports unreachable stops by name instead of producing a silent 0-cost route.
- _christofides_tsp pre-checks that the helper graph is connected and has ≥ 3 nodes, falling back to 2-opt otherwise.
- "auto" prefers 2-opt over Christofides for small instances: Christofides needs a complete undirected graph, which fails when stops are unreachable; 2-opt handles PENALTY weights gracefully.

"""

Продовження додатку А

```

from __future__ import annotations

import logging
import math
import random
from dataclasses import dataclass, field

import networkx as nx
import requests
from networkx.algorithms.approximation import christofides

logger = logging.getLogger(__name__)

# Unreachable-pair sentinel: large enough to dominate any real path, but
# finite so TSP solvers can still form a valid (if bad) tour.
PENALTY: float = 1e9

# --- Reachability audit ---

@dataclass
class UnreachableStop:
    """One stop that could not be reached from / to some other stop."""
    node_id: int
    label: str          # "Depot", "Stop #2", ...
    unreachable_from: list[str] = field(default_factory=list)
    unreachable_to: list[str] = field(default_factory=list)

```

Продовження додатку А

```

@property
def is_isolated(self) -> bool:
    """True if the node cannot reach OR be reached from ANY other node."""
    return bool(self.unreachable_from) or bool(self.unreachable_to)

def summary(self) -> str:
    parts = []
    if self.unreachable_to:
        parts.append(f'cannot reach: {', '.join(self.unreachable_to)}')
    if self.unreachable_from:
        parts.append(f'unreachable from: {', '.join(self.unreachable_from)}')
    return f'[{self.label}] " + "; ".join(parts)

def audit_reachability(
    matrix: dict[tuple[int, int], float],
    nodes: list[int],
    labels: dict[int, str],
) -> list[UnreachableStop]:
    """
    Scan the matrix for PENALTY-weight entries and return one UnreachableStop
    per affected node. Empty if every pair is reachable.

    nodes is the ordered list [depot_node, stop1_node, ...]; labels maps each
    node_id to a human-readable name.
    """

```

Продовження додатку А

```

report: dict[int, UnreachableStop] = {}

for src in nodes:
    for dst in nodes:
        if src == dst:
            continue
        cost = matrix.get((src, dst), PENALTY)
        if cost >= PENALTY:
            # src cannot reach dst
            if src not in report:
                report[src] = UnreachableStop(
                    node_id=src, label=labels.get(src, str(src)))
            report[src].unreachable_to.append(labels.get(dst, str(dst)))

            # dst is unreachable from src – record on dst side too
            if dst not in report:
                report[dst] = UnreachableStop(
                    node_id=dst, label=labels.get(dst, str(dst)))
            report[dst].unreachable_from.append(labels.get(src, str(src)))

# Deduplicate the lists
for us in report.values():
    us.unreachable_to = sorted(set(us.unreachable_to))
    us.unreachable_from = sorted(set(us.unreachable_from))

problems = list(report.values())
if problems:

```

Продовження додатку А

```

logger.warning(
    "Reachability audit found %d problematic node(s):\n %s",
    len(problems),
    "\n ".join(p.summary() for p in problems),
)
return problems

# --- Distance matrix ---

def build_distance_matrix(
    G: nx.MultiDiGraph,
    nodes: list[int],
    weight: str = "travel_time",
) -> dict[tuple[int, int], float]:
    """
    All-pairs shortest-path travel times (seconds) via Dijkstra.

    nodes is deduplicated first (dict.fromkeys preserves order, so the depot
    stays at index 0). This is critical: if two addresses snap to the same
    OSM node, the src==dst diagonal writes 0.0 and the TSP total collapses to
    0.0 s. Deduplication ensures every off-diagonal pair gets a real path.

    Returns {(src, dst): travel_time_s}. Diagonal is 0.0; unreachable pairs
    carry PENALTY (1e9 s).

    Raises RuntimeError if a node is absent from G (snapping happened before

```


Продовження додатку А

LSCC pruning – always snap after `get_network()`, or `ValueError` if fewer than 2 unique nodes remain.

```
"""
```

```
# Deduplicate while preserving order
```

```
n_raw = len(nodes)
```

```
unique_nodes: list[int] = list(dict.fromkeys(nodes))
```

```
n = len(unique_nodes)
```

```
n_dupes = n_raw - n
```

```
if n_dupes > 0:
```

```
    # Log every duplicate so the terminal output reveals which addresses
```

```
    # are collapsing to the same node.
```

```
    seen: set[int] = set()
```

```
    for pos, node_id in enumerate(nodes):
```

```
        if node_id in seen:
```

```
            logger.warning(
```

```
                " build_distance_matrix: duplicate node %d at position %d "
```

```
                "removed. Two or more addresses snapped to the same OSM "
```

```
                "node – check the NODE SNAP REPORT printed above for the "
```

```
                "specific lat/lon values.",
```

```
                node_id, pos,
```

```
            )
```

```
            print(
```

```
                f" ⚠ Duplicate node {node_id} at position {pos} removed "
```

```
                f"from distance matrix input."
```

```
            )
```

Продовження додатку А

```

        seen.add(node_id)

if n < 2:
    raise ValueError(
        f"build_distance_matrix requires at least 2 unique nodes; "
        f"got {n_raw} input node(s) that deduplicate to {n} unique node(s). "
        f"All delivery addresses have resolved to the same road node. "
        f"Use more specific street addresses (include building numbers), "
        f"verify the city lock is correct, or increase the network radius."
    )

logger.info(
    "Building distance matrix: %d input node(s) → %d unique (weight='%s')...",
    n_raw, n, weight,
)

print(
    f" Building {n}×{n} distance matrix"
    + (f" ({n_dupes} duplicate(s) removed)" if n_dupes else "")
    + f" weight='{weight}'"
)

# Validate every node exists in the graph up front, so the error fires
# immediately rather than mid-way through the O(n^2) Dijkstra loop.
for node_id in unique_nodes:
    if node_id not in G:
        raise RuntimeError(
            f"Node {node_id} is not present in the travel-time graph. "

```

Продовження додатку А

```

f"This means the address was snapped to an OSM node BEFORE "
f"the graph was pruned to its Largest Strongly Connected "
f"Component. Always call nearest_node() / nearest_node_safe() "
f"using the graph returned by get_network(), not the raw download."
)

# All-pairs Dijkstra
matrix: dict[tuple[int, int], float] = {}

for src in unique_nodes:
    for dst in unique_nodes:
        if src == dst:
            matrix[(src, dst)] = 0.0
            continue

    try:
        cost = nx.shortest_path_length(G, src, dst, weight=weight)

        # 0.0 cost between two different nodes means travel_time was
        # not stamped on the intermediate edges; treat as PENALTY.
        if cost == 0.0:
            logger.warning(
                " Dijkstra: %s → %s returned 0.0 s – "
                ""'%s' attribute missing on one or more edges. "
                "Inserting PENALTY.",
                src, dst, weight,
            )

```

Продовження додатку А

```

print(
    f" ⚠️ Dijkstra returned 0.0 s for {src}→{dst}. "
    f"The '{weight}' edge attribute may be missing. "
    f"PENALTY inserted."
)
cost = PENALTY

matrix[(src, dst)] = float(cost)
logger.debug("  %s → %s : %.2f s", src, dst, cost)

except nx.NetworkXNoPath:
    # No directed path; should not happen inside the LSCC.
    logger.warning(
        " No directed path %s → %s – inserting PENALTY.", src, dst
    )
    matrix[(src, dst)] = PENALTY

except nx.NodeNotFound as exc:
    # Unreachable after the validation loop above; re-raise with context.
    raise RuntimeError(
        f"Node lookup failed during Dijkstra ({src} → {dst}): {exc}"
    ) from exc

# Summary
reachable = sum(1 for (s, d), v in matrix.items() if s != d and v < PENALTY)
penalised = sum(1 for (s, d), v in matrix.items() if s != d and v >= PENALTY)
diagonal = n

```

Продовження додатку А

```

logger.info(
    " Matrix complete – %d×%d – reachable=%d penalty=%d diagonal=%d",
    n, n, reachable, penalised, diagonal,
)
print(
    f" Matrix complete: {n}×{n} | "
    f"reachable={reachable} penalty={penalised} diagonal={diagonal}"
)

if penalised > 0:
    logger.warning(
        " %d off-diagonal pair(s) are unreachable (PENALTY weight). "
        "The reachability audit will identify the specific stop names.",
        penalised,
    )

return matrix

def build_drive_matrix_hybrid(
    G_drive: nx.MultiDiGraph,
    depot_node: int,
    car_stops: list[tuple[int, float]],
    *,
    weight: str = "travel_time",
) -> tuple[dict[tuple[int, int], float], list[int]]:

```

Продовження додатку А

```
"""
```

Drive-mode distance matrix for hybrid last-meter routing.

Each car-reachable stop is represented by its nearest car-accessible node (N_car). Cost to reach stop j = drive time to N_car_j + walk_time_ j . Keyed by (i, j) indices so stops sharing one N_car still get distinct columns.

car_stops is a list of $(n_car, walk_time_s)$. Returns $(matrix, nodes_drive)$ where $nodes_drive$ is $[depot_node, n_car_1, \dots]$.

```
"""
```

```
nodes_drive: list[int] = [depot_node] + [n_car for (n_car, _) in car_stops]
```

```
walk_times: list[float] = [0.0] + [wt for (_, wt) in car_stops]
```

```
n = len(nodes_drive)
```

```
for node_id in nodes_drive:
```

```
    if node_id not in G_drive:
```

```
        raise RuntimeError(
```

```
            f'Node {node_id} is not in the drive graph. "
```

```
            "Snap addresses after get_network() and use car-accessible nodes."
```

```
        )
```

```
matrix: dict[tuple[int, int], float] = {}
```

```
for i in range(n):
```

```
    for j in range(n):
```

```
        if i == j:
```

```
            matrix[(i, j)] = 0.0
```

```
            continue
```

Продовження додатку А

```

src_id, dst_id = nodes_drive[i], nodes_drive[j]
try:
    drive_s = nx.shortest_path_length(
        G_drive, src_id, dst_id, weight=weight
    )
except (nx.NetworkXNoPath, nx.NodeNotFound):
    drive_s = PENALTY
if isinstance(drive_s, (int, float)) and drive_s == 0.0:
    drive_s = PENALTY
total = float(drive_s) + walk_times[j]
matrix[(i, j)] = min(total, PENALTY)
logger.info(
    " Drive hybrid matrix: %d×%d (depot + %d stops, last-meter walk added)",
    n, n, len(car_stops),
)
return matrix, nodes_drive

_MAPBOX_MATRIX_URL = (
    "https://api.mapbox.com/directions-matrix/v1/mapbox/driving-traffic/{coords}"
)
# Mapbox Matrix allows at most 25 coordinates per request. Chunks of 12 keep
# each request to at most 24 unique coordinates (12 sources + 12 dests).
_MAPBOX_CHUNK = 12

def build_drive_matrix_mapbox(

```

Продовження додатку А

```

stops: list[tuple[float, float]],
api_key: str,
) -> dict[tuple[int, int], float]:
    """
    n x n drive-time matrix (seconds) via the Mapbox Directions Matrix API
    (driving-traffic profile), which uses historical traffic patterns for more
    realistic times than the static OSMnx Dijkstra approach.

    stops is [(lat, lon), ...] with the depot at index 0. Returns {(i, j): s};
    diagonal is 0.0, unreachable pairs carry PENALTY.

    Raises requests.HTTPError on a non-2xx response, ValueError if < 2 stops.
    """
    n = len(stops)
    if n < 2:
        raise ValueError("build_drive_matrix_mapbox requires at least 2 stops.")

    matrix: dict[tuple[int, int], float] = {(i, i): 0.0 for i in range(n)}

    def _request_chunk(
        src_indices: list[int],
        dst_indices: list[int],
    ) -> None:
        """Make one Mapbox Matrix API call for a subset of source/dest pairs."""
        # Build a deduplicated coordinate list for this request.
        # Mapbox sources/destinations are expressed as indices into this list.
        coord_pos: dict[int, int] = {} # global stop index → position in coord_list

```


Продовження додатку А

```

coord_list: list[tuple[float, float]] = []

for idx in dict.fromkeys(src_indices + dst_indices): # stable-dedup order
    coord_pos[idx] = len(coord_list)
    coord_list.append(stops[idx])

coords_str = ";".join(f'{{lon}},{{lat}}' for lat, lon in coord_list)
url = _MAPBOX_MATRIX_URL.format(coords=coords_str)

resp = requests.get(
    url,
    params={
        "access_token": api_key,
        "annotations": "duration",
        "sources": ";".join(str(coord_pos[i]) for i in src_indices),
        "destinations": ";".join(str(coord_pos[j]) for j in dst_indices),
    },
    timeout=15,
)
resp.raise_for_status()
data = resp.json()

durations = data.get("durations") or []
for r, row in enumerate(durations):
    for c, val in enumerate(row):
        gi, gj = src_indices[r], dst_indices[c]
        if gi == gj:

```

Продовження додатку А

```

        matrix[(gi, gj)] = 0.0
    elif val is None:
        matrix[(gi, gj)] = PENALTY
    else:
        matrix[(gi, gj)] = float(val)

# Iterate over (source_chunk x destination_chunk) pairs; each chunk is at
# most _MAPBOX_CHUNK indices, keeping unique coords per request <= 24.
all_indices = list(range(n))
for src_start in range(0, n, _MAPBOX_CHUNK):
    src_chunk = all_indices[src_start : src_start + _MAPBOX_CHUNK]
    for dst_start in range(0, n, _MAPBOX_CHUNK):
        dst_chunk = all_indices[dst_start : dst_start + _MAPBOX_CHUNK]
        _request_chunk(src_chunk, dst_chunk)

reachable = sum(1 for (i, j), v in matrix.items() if i != j and v < PENALTY)
penalised = sum(1 for (i, j), v in matrix.items() if i != j and v >= PENALTY)
logger.info(
    " Mapbox drive matrix: %d×%d – reachable=%d penalty=%d",
    n, n, reachable, penalised,
)
return matrix

# --- TSP solvers ---

def _route_cost(route: list[int], matrix: dict) -> float:

```

Продовження додатку А

```
"""
```

Total travel cost of a closed TSP tour [depot, s1, ..., sn, depot].

The return-to-depot leg is already the last consecutive pair, so the cost is just the sum of `matrix[route[i] -> route[i+1]]` – no extra wrap-around term (that would double-count depot->depot).

Any pathological matrix value (missing key, None, inf, nan) is treated as PENALTY, so solvers always work with finite floats. Never raises.

```
"""
```

```
if len(route) < 2:
```

```
    # A 0- or 1-node route has no edges to traverse.
```

```
    return 0.0
```

```
total = 0.0
```

```
for i in range(len(route) - 1):
```

```
    src = route[i]
```

```
    dst = route[i + 1]
```

```
    raw = matrix.get((src, dst), None)
```

```
    # Normalise every pathological value to PENALTY
```

```
    if raw is None:
```

```
        # Key missing – pair never computed; should not happen.
```

```
        logger.debug(
```

```
            "_route_cost: matrix key (%s, %s) missing – using PENALTY.", src, dst
        )
```

Продовження додатку А

```

cost = PENALTY

elif not isinstance(raw, (int, float)):
    # Non-numeric value in matrix (e.g. a string from bad serialisation)
    logger.warning(
        "_route_cost: non-numeric matrix value %r for (%s, %s) – using
PENALTY.",
        raw, src, dst,
    )
    cost = PENALTY

else:
    f = float(raw)
    if math.isnan(f) or math.isinf(f):
        logger.debug(
            "_route_cost: matrix[(%s, %s)] = %s – replacing with PENALTY.",
            src, dst, f,
        )
        cost = PENALTY
    else:
        cost = f # normal finite value

total += cost

# Early exit: once total hits PENALTY the tour is non-viable.
if total >= PENALTY:
    return PENALTY

```

Продовження додатку А

```

return total

def solve_tsp(
    nodes: list[int],
    matrix: dict[tuple[int, int], float],
    method: str = "auto",
    seed: int = 42,
) -> tuple[list[int], float]:
    """
    Find a near-optimal visit order for nodes (nodes[0] is the depot).

    Methods:
    "auto"      pick by stop count: nn (1-2), 2opt (3-20), genetic (21+)
    "nn"        nearest-neighbour greedy
    "2opt"      NN + 2-opt refinement
    "christofides" NetworkX Christofides (needs a complete graph)
    "genetic"   order-crossover genetic algorithm

    auto uses 2opt rather than Christofides for small graphs because
    Christofides needs a complete connected graph and fails when any stop is
    unreachable; 2opt handles PENALTY weights gracefully.

    Returns (ordered_node_ids, total_travel_time_s); the order starts and ends
    at the depot.
    """

```

Продовження додатку А

```
n = len(nodes)
if n == 0:
    raise ValueError("Cannot solve TSP: node list is empty.")
if n == 1:
    return [nodes[0], nodes[0]], 0.0

if method == "auto":
    if n <= 2:
        method = "nn"
    elif n <= 20:
        method = "2opt"
    else:
        method = "genetic"

logger.info("Solving TSP for %d stop(s) with method='%s'...", n - 1, method)

if method == "nn":
    route = _nearest_neighbour(nodes, matrix)
elif method == "2opt":
    route = _two_opt(_nearest_neighbour(nodes, matrix), matrix)
elif method == "christofides":
    route = _christofides_tsp(nodes, matrix)
elif method == "genetic":
    route = _genetic_algorithm(nodes, matrix, seed=seed)
else:
    raise ValueError(f"Unknown TSP method: {method!r}")
```

Продовження додатку А

```

total = _route_cost(route, matrix)
logger.info(" Route cost: %.1f s (%.2f min)", total, total / 60.0)

if total >= PENALTY:
    logger.error(
        " Route cost is  $\geq$  PENALTY – at least one stop is unreachable. "
        "Check the unreachability audit output above for the specific address."
    )

return route, total

# --- Nearest-neighbour ---

def _nearest_neighbour(nodes: list[int], matrix: dict) -> list[int]:
    """Greedy nearest-neighbour from the depot; always returns a full tour."""
    depot = nodes[0]
    unvisited = set(nodes[1:])
    route = [depot]
    current = depot

    while unvisited:
        nearest = min(unvisited, key=lambda n: matrix.get((current, n), PENALTY))
        route.append(nearest)
        unvisited.remove(nearest)
        current = nearest

```

Продовження додатку А

```

route.append(depot)
return route

# --- 2-opt ---

def _two_opt(
    route: list[int],
    matrix: dict,
    max_iter: int = 2_000,
) -> list[int]:
    """
    2-opt local search. Depot is pinned at index 0 / len-1 and never swapped.
    A swap is accepted only if it lowers total cost, so PENALTY edges are
    avoided whenever a finite alternative exists.
    """
    best = route[:]
    best_cost = _route_cost(best, matrix)
    improved = True
    iterations = 0

    while improved and iterations < max_iter:
        improved = False
        iterations += 1
        for i in range(1, len(best) - 2):
            for j in range(i + 1, len(best) - 1):
                candidate = best[:i] + best[i:j + 1][::-1] + best[j + 1:]

```


Продовження додатку А

```

        candidate_cost = _route_cost(candidate, matrix)
        if candidate_cost < best_cost:
            best = candidate
            best_cost = candidate_cost
            improved = True

    logger.debug(" 2-opt: %d iteration(s), final cost=%.1f s", iterations, best_cost)
    return best

# --- Christofides ---

def _christofides_tsp(nodes: list[int], matrix: dict) -> list[int]:
    """
    NetworkX Christofides approximation, with fallbacks to 2-opt when the
    helper graph H (PENALTY edges excluded) has isolated nodes, fewer than 3
    nodes, or is disconnected. Falls back on any exception, so a result is
    always returned.
    """
    try:
        # Build an undirected helper graph with only finite edges.
        H = nx.Graph()
        H.add_nodes_from(nodes)
        for (u, v), w in matrix.items():
            if u != v and w < PENALTY:
                # Keep the cheaper of the two directions.
                existing = H.get_edge_data(u, v)

```

Продовження додатку А

```

    if existing is None or w < existing["weight"]:
        H.add_edge(u, v, weight=w)

# Christofides needs at least 3 nodes.
if H.number_of_nodes() < 3:
    logger.info(
        " Christofides: graph has < 3 nodes – falling back to 2-opt."
    )
    return _two_opt(_nearest_neighbour(nodes, matrix), matrix)

# Every node must have at least one finite-weight neighbour.
isolated = [n for n in nodes if H.degree(n) == 0]
if isolated:
    logger.warning(
        " Christofides: %d isolated node(s) found (no finite-cost "
        "path to any other stop): %s. Falling back to 2-opt.",
        len(isolated), isolated,
    )
    return _two_opt(_nearest_neighbour(nodes, matrix), matrix)

# The helper graph must be connected.
if not nx.is_connected(H):
    components = nx.number_connected_components(H)
    logger.warning(
        " Christofides: helper graph has %d disconnected component(s) "
        "– this means some stops cannot be linked with finite-cost edges. "
        "Falling back to 2-opt.",

```

Продовження додатку А

```

        components,
    )
    return _two_opt(_nearest_neighbour(nodes, matrix), matrix)

# All checks passed – run Christofides.
cycle = christofides(H, weight="weight")

# Rotate so the depot is first.
depot = nodes[0]
if depot in cycle:
    idx = cycle.index(depot)
    cycle = cycle[idx:] + cycle[1:idx + 1]
else:
    # Cycle missing the depot; should not happen.
    cycle = cycle + [cycle[0]]

logger.debug(" Christofides succeeded – %d-node cycle.", len(cycle))
return cycle

except Exception as exc:
    logger.warning(
        " Christofides raised an unexpected exception (%s: %s). "
        "Falling back to 2-opt.",
        type(exc).__name__, exc,
    )
    return _two_opt(_nearest_neighbour(nodes, matrix), matrix)

```

Продовження додатку А

```
# --- Genetic algorithm ---
```

```
def _genetic_algorithm(
```

```
    nodes: list[int],
```

```
    matrix: dict,
```

```
    population_size: int = 120,
```

```
    generations: int = 400,
```

```
    mutation_rate: float = 0.02,
```

```
    seed: int = 42,
```

```
) -> list[int]:
```

```
    """
```

```
    Order-crossover (OX) genetic algorithm. The depot sits at position 0 / -1
```

```
    and is excluded from the chromosome. PENALTY weights are handled by the
    fitness function, which penalises any unreachable leg.
```

```
    """
```

```
    random.seed(seed)
```

```
    depot = nodes[0]
```

```
    stops = nodes[1:]
```

```
    n = len(stops)
```

```
    if n == 0:
```

```
        return [depot, depot]
```

```
    if n == 1:
```

```
        return [depot, stops[0], depot]
```

```
def fitness(chrom: list[int]) -> float:
```

Продовження додатку А

```

route = [depot] + chrom + [depot]
cost = _route_cost(route, matrix)
# Reciprocal fitness – lower cost = higher fitness
# Add small epsilon so PENALTY routes don't divide by zero
return 1.0 / (cost + 1.0)

def ox_crossover(p1: list[int], p2: list[int]) -> list[int]:
    """Order crossover: preserve a slice of p1, fill rest from p2."""
    a, b = sorted(random.sample(range(n), 2))
    child: list[int | None] = [None] * n
    child[a:b] = p1[a:b]
    fill = [g for g in p2 if g not in child]
    fill_idx = 0
    for i in range(n):
        if child[i] is None:
            child[i] = fill[fill_idx]
            fill_idx += 1
    return child # type: ignore[return-value]

def mutate(chrom: list[int]) -> list[int]:
    """Swap mutation: exchange two random positions."""
    if random.random() < mutation_rate and n >= 2:
        i, j = random.sample(range(n), 2)
        chrom[i], chrom[j] = chrom[j], chrom[i]
    return chrom

# Initialise with random permutations

```

Продовження додатку А

```

population = [random.sample(stops, n) for _ in range(population_size)]

for _ in range(generations):
    population.sort(key=fitness, reverse=True)
    elites = population[:max(2, population_size // 10)]
    children: list[list[int]] = elites[:]
    while len(children) < population_size:
        p1, p2 = random.choices(elites, k=2)
        children.append(mutate(ox_crossover(p1, p2)))
    population = children

best_chrom = max(population, key=fitness)
return [depot] + best_chrom + [depot]

# --- Path reconstruction ---

def get_full_path(
    G: nx.MultiDiGraph,
    src: int,
    dst: int,
    weight: str = "travel_time",
) -> list[int]:
    """
    OSM node ids for the shortest path src -> dst.

    Returns [src] if src == dst, or [src, dst] with a warning if no path

```

Продовження додатку А

```

exists (the polyline degrades to a straight line on the map).
"""

if src == dst:
    return [src]
try:
    return nx.shortest_path(G, src, dst, weight=weight)
except nx.NetworkXNoPath:
    logger.warning(
        "get_full_path: no path from %s to %s – route segment will be "
        "a straight line. This stop may need to be removed.",
        src, dst,
    )
    return [src, dst]
except nx.NodeNotFound as exc:
    logger.error("get_full_path: node not found – %s", exc)
    return [src, dst]

def reconstruct_full_route(
    G: nx.MultiDiGraph,
    tsp_route: list[int],
    weight: str = "travel_time",
) -> list[int]:
    """
    Expand a TSP node sequence into the full OSM node sequence, including all
    intermediate road nodes between each pair of stops.
    """

```

Продовження додатку А

```

full: list[int] = []
for i in range(len(tsp_route) - 1):
    leg = get_full_path(G, tsp_route[i], tsp_route[i + 1], weight)
    if full:
        leg = leg[1:] # drop the shared boundary node
    full.extend(leg)
return full

"""Renders an interactive Folium map for all three travel modes."""
from __future__ import annotations
import logging, math
from pathlib import Path
import folium
from folium.plugins import AntPath
import networkx as nx
from graph_builder import SPEED_KMH

logger = logging.getLogger(__name__)
MODE_COLOURS = {"drive": "#E74C3C", "bike": "#2ECC71", "walk":
"#3498DB"}

def _nc(G, n):
    d = G.nodes[n]; return d["y"], d["x"]

def _hms(s):
    if not math.isfinite(s): return "N/A"
    h, r = divmod(int(s), 3600); m, sc = divmod(r, 60)

```


Продовження додатку А

```

return (f'{h}h {m:02d}m {sc:02d}s' if h else f'{m}m {sc:02d}s' if m else
f'{sc}s')

def build_map(mode_graphs, mode_routes, mode_times, depot_node,
              stop_nodes, output_path="delivery_map.html", default_mode="drive"):
    op = Path(output_path)
    G = mode_graphs[default_mode]
    dlat, dlon = _nc(G, depot_node)
    fm = folium.Map(location=[dlat, dlon], zoom_start=14, tiles="CartoDB
positron")

    for mode, route in mode_routes.items():
        coords = [_nc(G, n) for n in route]
        lg = folium.FeatureGroup(name=f'{mode.capitalize()} route', show=(mode
== default_mode))
        AntPath(locations=coords, color=MODE_COLOURS[mode], weight=5,
opacity=0.85,
                delay=800, dash_array=[20, 30], pulse_color="#FFFFFF",
                tooltip=f'{mode} - {_hms(mode_times[mode])}').add_to(lg)
        lg.add_to(fm)

    folium.Marker([dlat, dlon], tooltip="Depot",
                  icon=folium.Icon(color="darkblue", icon="home",
prefix="fa")).add_to(fm)

    sl = folium.FeatureGroup(name="Stops", show=True)

```

Продовження додатку А

```

for i, node in enumerate(stop_nodes, 1):
    lat, lon = _nc(G, node)
    folium.Marker([lat, lon], tooltip=f"Stop #{i}",
        icon=folium.DivIcon(
            html=f"<div style='background:#E74C3C;color:white;border-
radius:50%;'
                f'width:28px;height:28px;line-height:28px;text-align:center;'
                f'font-weight:bold;border:2px solid white'>{i}</div>",
            icon_size=(28, 28), icon_anchor=(14, 14))).add_to(sl)
sl.add_to(fm)

rows = "".join(
    f'<tr><td style="padding:4px 8px"><span
style="color: {MODE_COLOURS[m]}">#{1044}</span>'
    f'<b>{m.capitalize()}</b></td>'
    f'<td style="padding:4px 8px;text-align:right">{SPEED_KMH[m]:.0f}
km/h</td>'
    f'<td style="padding:4px 8px;text-
align:right"><b>{_hms(mode_times.get(m,float("inf")))}</b></td></tr>'
    for m in ("drive", "bike", "walk"))
panel = (
    '<div
style="position:fixed;top:16px;right:16px;background:rgba(255,255,255,0.95);'
    'border-radius:10px;padding:14px 18px;box-shadow:0 2px 12px
rgba(0,0,0,.25);z-index:9999;'
    'font-family:sans-serif;font-size:14px;min-width:240px">'
    f'<b style="font-size:16px">Delivery Summary</b><br>'

```

Продовження додатку А

```

        f<span style="font-size:12px;color:#666">{len(stop_nodes)} stop(s) - round-
trip from depot</span>'
        '<table style="width:100%;border-collapse:collapse;margin-top:8px">'
        '<tr style="font-
size:12px;color:#888"><th>Mode</th><th>Speed</th><th>Time</th></tr>'
        f'{rows}</table></div>')
    fm.get_root().html.add_child(folium.Element(panel))
    folium.LayerControl(collapsed=False).add_to(fm)
    fm.save(str(op))
    logger.info("Map saved -> %s", op.resolve())
    return op
from __future__ import annotations

import logging
import math
import random
from dataclasses import dataclass, field

import networkx as nx

from route_solver import (
    build_distance_matrix,
    reconstruct_full_route,
    PENALTY,
)
from graph_builder import nearest_car_accessible_node

```

Продовження додатку А

```

logger = logging.getLogger(__name__)

# 10-colour qualitative palette (ColorBrewer Set1 + extensions).
VEHICLE_COLORS: list[str] = [
    "#e41a1c", "#377eb8", "#4daf4a", "#984ea3",
    "#ff7f00", "#a65628", "#f781bf", "#999999",
    "#66c2a5", "#fc8d62",
]

# — Genetic-algorithm default parameters (§2.7 of thesis)


---


POP_SIZE: int = 50
N_GENERATIONS: int = 150
CROSSOVER_RATE: float = 0.85
MUTATION_RATE: float = 0.15
ELITE_SIZE: int = 2
TOURNAMENT_SIZE: int = 3
UNROUTED_PENALTY: float = 1.0e6 # fitness penalty per unrouted customer
GA_SEED: int | None = 42 # None => non-deterministic runs

# Objective-function weights:  $F = A \cdot T_{\text{total}} + B \cdot |V_{\text{act}}|$ 
OBJ_A: float = 1.0
OBJ_B: float = 0.0

#

```

Продовження додатку А

```
# DATA CLASSES
```

```
#
```

```
@dataclass
```

```
class Vehicle:
```

```
    name: str
```

```
    mode: str      # "drive" | "bike" | "walk"
```

```
    capacity_kg: float # maximum payload weight  $Q_{\theta}$  (kg)
```

```
    color: str = ""
```

```
@dataclass
```

```
class VehicleRoute:
```

```
    vehicle: Vehicle
```

```
    stops: list      # ordered list of DeliveryStop in visit order (depot excluded)
```

```
    tsp_route: list   # closed OSM node tour: [depot_node, n1, ..., depot_node]
```

```
    full_route: list  # expanded OSM node sequence for map polyline rendering
```

```
    total_time_s: float
```

```
    total_dist_m: float
```

```
    legs: list = field(default_factory=list)
```

```
    skipped_stops: list = field(default_factory=list)
```

```
#
```

Продовження додатку А

```
# GENETIC ALGORITHM – DECODER (Split)
```

```
#
```

```
def _decode_chromosome(
```

```
    chromosome: list[int],
```

```
    stops: list,
```

```
    vehicles: list,
```

```
    depot,
```

```
    matrix: dict,
```

```
    depot_node: int,
```

```
    stop_nodes: dict,
```

```
) -> tuple[list[tuple[Vehicle, list[int]]], list[int]]:
```

```
    """
```

Split decoder – converts a permutation into a list of feasible vehicle routes.

Walks through the chromosome in order, adding each customer to the current vehicle's route as long as capacity and time-window constraints remain satisfied. When an insertion would violate a constraint, the current route is closed and the customer is tried on the next vehicle in the fleet.

Returns

Продовження додатку А

```

-----
routes    : list of (Vehicle, [customer_indices]) for routes with  $\geq 1$  stop
unassigned : list of customer indices that fit no vehicle
"""

def t(i: int, j: int) -> float:
    ni = depot_node if i == -1 else stop_nodes[i]
    nj = depot_node if j == -1 else stop_nodes[j]
    return matrix.get((ni, nj), PENALTY)

def tw_open(i: int) -> float:
    return depot.tw_open if i == -1 else stops[i].tw_open

def tw_close(i: int) -> float:
    return depot.tw_close if i == -1 else stops[i].tw_close

def service(i: int) -> float:
    return depot.service_time if i == -1 else stops[i].service_time

def weight(i: int) -> float:
    return stops[i].weight_kg

def route_feasible(route: list[int], vehicle: Vehicle) -> bool:
    """Capacity + time-window feasibility for a closed route
    (depot→...→depot)."""
    if not route:
        return True
    if sum(weight(i) for i in route) > vehicle.capacity_kg:

```

Продовження додатку А

```

    return False
prev = -1
prev_begin = tw_open(-1)
for cust in route:
    tt = t(prev, cust)
    if tt >= PENALTY:
        return False
    arr = prev_begin + service(prev) + tt
    if arr > tw_close(cust):
        return False
    prev_begin = max(arr, tw_open(cust))
    prev = cust
tt_back = t(prev, -1)
if tt_back >= PENALTY:
    return False
if prev_begin + service(prev) + tt_back > tw_close(-1):
    return False
return True

```

```

routes: list[tuple[Vehicle, list[int]]] = []
unassigned: list[int] = []
veh_idx = 0
current_route: list[int] = []

```

```

for cust in chromosome:
    placed = False
    while veh_idx < len(vehicles) and not placed:

```


Продовження додатку А

```

    vehicle = vehicles[veh_idx]
    candidate = current_route + [cust]
    if route_feasible(candidate, vehicle):
        current_route = candidate
        placed = True
    else:
        if current_route:
            routes.append((vehicle, current_route))
            current_route = []
        veh_idx += 1
    if not placed:
        unassigned.append(cust)

if current_route and veh_idx < len(vehicles):
    routes.append((vehicles[veh_idx], current_route))

return routes, unassigned

def _compute_route_time(
    route: list[int],
    matrix: dict,
    depot_node: int,
    stop_nodes: dict,
) -> float:
    """Total travel time along a closed route: depot → customers → depot."""
    if not route:

```

Продовження додатку А

```

    return 0.0

total = matrix.get((depot_node, stop_nodes[route[0]]), PENALTY)
for k in range(len(route) - 1):
    total += matrix.get(
        (stop_nodes[route[k]], stop_nodes[route[k + 1]]),
        PENALTY,
    )
total += matrix.get((stop_nodes[route[-1]], depot_node), PENALTY)
return total

def _evaluate(
    chromosome: list[int],
    stops: list,
    vehicles: list,
    depot,
    matrix: dict,
    depot_node: int,
    stop_nodes: dict,
) -> tuple[float, list[tuple[Vehicle, list[int]]], list[int]]:
    """
    Compute fitness  $F = A \cdot T_{\text{total}} + B \cdot K + P \cdot |U|$  (lower is better).

    Returns (fitness, routes, unassigned).
    """
    routes, unassigned = _decode_chromosome(
        chromosome, stops, vehicles, depot, matrix, depot_node, stop_nodes,

```

Продовження додатку А

```

)
T_total = sum(
    _compute_route_time(r, matrix, depot_node, stop_nodes)
    for _, r in routes
)
K = len(routes)
fitness = OBJ_A * T_total + OBJ_B * K + UNROUTED_PENALTY *
len(unassigned)
return fitness, routes, unassigned

```

```

#

```

```

# GENETIC OPERATORS

```

```

#

```

```

def _order_crossover(
    p1: list[int],
    p2: list[int],
    rng: random.Random,

```

```

) -> list[int]:

```

```

    """

```

Order Crossover (OX). Copies a contiguous slice of p1 into the child,
then fills the remaining positions in the order they appear in p2

Продовження додатку А

(starting after the slice, wrapping around).

```

"""
n = len(p1)
if n < 2:
    return p1[:]
i, j = sorted(rng.sample(range(n), 2))
child: list[int] = [-1] * n
used = [False] * n
for k in range(i, j + 1):
    child[k] = p1[k]
    used[p1[k]] = True
fill_pos = (j + 1) % n
scan_pos = (j + 1) % n
for _ in range(n):
    gene = p2[scan_pos]
    scan_pos = (scan_pos + 1) % n
    if not used[gene]:
        child[fill_pos] = gene
        fill_pos = (fill_pos + 1) % n
return child

```

```

def _swap_mutation(chromo: list[int], rng: random.Random) -> list[int]:
    """Swap mutation: exchange the values at two random positions."""
    n = len(chromo)
    if n < 2:
        return chromo

```

Продовження додатку А

```

i, j = rng.sample(range(n), 2)
chromo[i], chromo[j] = chromo[j], chromo[i]
return chromo

def _tournament_select(
    population: list[list[int]],
    fitnesses: list[float],
    k: int,
    rng: random.Random,
) -> list[int]:
    """Tournament selection: draw k at random, return the fittest."""
    size = min(k, len(population))
    candidates = rng.sample(range(len(population)), size)
    best = min(candidates, key=lambda idx: fitnesses[idx])
    return population[best]

def _nearest_neighbour_seed(
    n_stops: int,
    matrix: dict,
    depot_node: int,
    stop_nodes: dict,
) -> list[int]:
    """Greedy nearest-neighbour permutation – used as one high-quality seed."""
    if n_stops == 0:
        return []

```

Продовження додатку А

```

unvisited = set(range(n_stops))
current = depot_node
chromo: list[int] = []
while unvisited:
    nxt = min(
        unvisited,
        key=lambda c: matrix.get((current, stop_nodes[c]), PENALTY),
    )
    chromo.append(nxt)
    current = stop_nodes[nxt]
    unvisited.discard(nxt)
return chromo

#
=====

=====

# MAIN GA LOOP
#
=====

=====

def _genetic_algorithm(
    stops: list,
    vehicles: list,
    depot,
    matrix: dict,

```

Продовження додатку А

```

depot_node: int,
stop_nodes: dict,
*,
pop_size: int = POP_SIZE,
n_generations: int = N_GENERATIONS,
crossover_rate: float = CROSSOVER_RATE,
mutation_rate: float = MUTATION_RATE,
elite_size: int = ELITE_SIZE,
tournament_size: int = TOURNAMENT_SIZE,
seed: int | None = GA_SEED,
) -> list[tuple[Vehicle, list[int]]]:
    """

```

Genetic algorithm for CVRPTW.

Parameters

stops : list[DeliveryStop] – unrouted customers
vehicles : list[Vehicle] – available fleet for this mode
depot : DeliveryStop – depot (tw_open/close, service_time)
matrix : dict[(node_i, node_j) -> float] travel-time matrix
depot_node : OSM node id of the depot
stop_nodes : dict[customer_index -> OSM node id]
pop_size, n_generations, crossover_rate, mutation_rate,
elite_size, tournament_size : GA control parameters.
seed : RNG seed for reproducibility (None = non-deterministic).

Returns

Продовження додатку А

```

-----
list of (Vehicle, [customer_indices_in_visit_order])
"""

n = len(stops)
if n == 0:
    return []

rng = random.Random(seed)

# — Initial population: 1 NN seed + random permutations

```

```

population: list[list[int]] = []
nn_chromo = _nearest_neighbour_seed(n, matrix, depot_node, stop_nodes)
if nn_chromo:
    population.append(nn_chromo)
while len(population) < pop_size:
    perm = list(range(n))
    rng.shuffle(perm)
    population.append(perm)

evaluations = [
    _evaluate(c, stops, vehicles, depot, matrix, depot_node, stop_nodes)
    for c in population
]
fitnesses = [e[0] for e in evaluations]

best_idx = min(range(pop_size), key=lambda i: fitnesses[i])

```


Продовження додатку А

```

best_fitness = fitnesses[best_idx]
best_eval = evaluations[best_idx]

# — Evolutionary loop

```

```

for _ in range(n_generations):
    order = sorted(range(pop_size), key=lambda i: fitnesses[i])
    new_population: list[list[int]] = [
        population[i][:] for i in order[:elite_size]
    ]

    while len(new_population) < pop_size:
        p1 = _tournament_select(population, fitnesses, tournament_size, rng)
        p2 = _tournament_select(population, fitnesses, tournament_size, rng)

        child = _order_crossover(p1, p2, rng) if rng.random() < crossover_rate else
p1[:]
        if rng.random() < mutation_rate:
            child = _swap_mutation(child, rng)

        new_population.append(child)

population = new_population
evaluations = [
    _evaluate(c, stops, vehicles, depot, matrix, depot_node, stop_nodes)
    for c in population
]

```

Продовження додатку А

```

fitnesses = [e[0] for e in evaluations]

gen_best = min(range(pop_size), key=lambda i: fitnesses[i])
if fitnesses[gen_best] < best_fitness:
    best_fitness = fitnesses[gen_best]
    best_eval = evaluations[gen_best]

_, best_routes, unassigned = best_eval
if unassigned:
    logger.warning(
        "GA: %d customer(s) could not be routed (fleet exhausted or "
        "infeasible): indices %s",
        len(unassigned), sorted(unassigned),
    )
logger.info(
    "GA: best fitness %.1f after %d generation(s), %d active route(s)",
    best_fitness, n_generations, len(best_routes),
)
return best_routes

#

```

```

# INTERNAL HELPERS
#

```

Продовження додатку А

```

def _bearing(lat1: float, lon1: float, lat2: float, lon2: float) -> float:
    """Compass bearing in degrees (0-360) from point 1 to point 2."""
    lat1r, lat2r = math.radians(lat1), math.radians(lat2)
    dlon = math.radians(lon2 - lon1)
    x = math.sin(dlon) * math.cos(lat2r)
    y = (math.cos(lat1r) * math.sin(lat2r)
        - math.sin(lat1r) * math.cos(lat2r) * math.cos(dlon))
    return (math.degrees(math.atan2(x, y)) + 360) % 360

def _check_reachable(stop, depot, graphs: dict) -> set:
    """
    Return the set of mode names that can reach *stop* from *depot*.

    A mode is considered reachable when the Dijkstra shortest-path cost
    (travel_time weight) is strictly below PENALTY.
    """
    if stop.node_id is None or stop.node_id == depot.node_id:
        return set()

    reachable = set()
    for mode, G in graphs.items():
        if mode == "drive":
            check_node = nearest_car_accessible_node(G, stop.lat, stop.lon)
            if check_node is None:

```

Продовження додатку А

```

        check_node = stop.node_id
        depot_node = nearest_car_accessible_node(G, depot.lat, depot.lon)
        if depot_node is None:
            depot_node = depot.node_id
        else:
            check_node = stop.node_id
            depot_node = depot.node_id
        try:
            cost = nx.shortest_path_length(G, depot_node, check_node,
weight="travel_time")
            if cost < PENALTY:
                reachable.add(mode)
        except (nx.NetworkXNoPath, nx.NodeNotFound):
            pass
    return reachable

```

```
def _assign_stops_to_modes(
```

```
    stops: list,
```

```
    fleet: list,
```

```
    depot,
```

```
    graphs: dict,
```

```
) -> tuple[dict, list]:
```

```
    """
```

Phase 1 – assign each stop to a vehicle-type pool.

For stops compatible with multiple fleet modes, the mode whose vehicles

Продовження додатку А

have the most remaining total capacity is chosen.

Returns

mode_stop_pool : dict[mode -> list[stop]]

unreachable : list[stop]

"""

mode_remaining_kg: dict[str, float] = {}

for v in fleet:

 mode_remaining_kg[v.mode] = mode_remaining_kg.get(v.mode, 0.0) +
v.capacity_kg

fleet_modes = set(v.mode for v in fleet)

mode_stop_pool: dict[str, list] = {m: [] for m in fleet_modes}

unreachable: list = []

for stop in stops:

 stop_w = getattr(stop, "weight_kg", 1.0)

 compatible = _check_reachable(stop, depot, graphs)

 available = [m for m in compatible if mode_remaining_kg.get(m, 0.0) >=
stop_w]

 if not available:

 unreachable.append(stop)

 continue

 chosen = max(available, key=lambda m: mode_remaining_kg[m])

Продовження додатку А

```

mode_stop_pool[chosen].append(stop)
mode_remaining_kg[chosen] -= stop_w

return mode_stop_pool, unreachable

def _build_ga_routes(
    pool_stops: list,
    vehicles: list,
    depot,
    graphs: dict,
) -> list[VehicleRoute]:
    """
    Phase 2 – build routes for one mode's stop pool using the GA.

    Steps:
    1. Snap depot + stops to OSM nodes (car-accessible for drive mode).
    2. Build the travel-time matrix over depot + stop nodes.
    3. Run the genetic algorithm to partition stops across vehicles.
    4. For each vehicle route, reconstruct the full OSM path and compute
       the total road distance.
    """
    mode = vehicles[0].mode
    G = graphs[mode]

    if mode == "drive":
        node_ids = [

```

Продовження додатку А

```

        nearest_car_accessible_node(G, s.lat, s.lon) or s.node_id
    for s in pool_stops
]
    depot_node = nearest_car_accessible_node(G, depot.lat, depot.lon) or
depot.node_id
else:
    node_ids = [s.node_id for s in pool_stops]
    depot_node = depot.node_id

all_nodes = [depot_node] + node_ids
unique_nodes = list(dict.fromkeys(all_nodes))
matrix = build_distance_matrix(G, unique_nodes)

stop_node_map: dict[int, int] = {i: node_ids[i] for i in range(len(pool_stops))}

ga_routes = _genetic_algorithm(
    pool_stops, vehicles, depot, matrix, depot_node, stop_node_map,
)

results: list[VehicleRoute] = []
for vehicle, stop_indices in ga_routes:
    if not stop_indices:
        continue

    tsp_route = [depot_node] + [stop_node_map[i] for i in stop_indices] +
[depot_node]
    full_route = reconstruct_full_route(G, tsp_route)

```

Продовження додатку А

```

total_time_s = 0.0
for k in range(len(tsp_route) - 1):
    total_time_s += matrix.get((tsp_route[k], tsp_route[k + 1]), PENALTY)

total_dist_m = 0.0
for k in range(len(full_route) - 1):
    ed = G.get_edge_data(full_route[k], full_route[k + 1])
    if ed:
        total_dist_m += min(
            (v.get("length", 0.0) for v in ed.values()),
            default=0.0,
        )

ordered_stops = [pool_stops[i] for i in stop_indices]

results.append(VehicleRoute(
    vehicle=vehicle,
    stops=ordered_stops,
    tsp_route=tsp_route,
    full_route=full_route,
    total_time_s=total_time_s,
    total_dist_m=total_dist_m,
))

return results

```


Продовження додатку А

#

OBJECTIVE FUNCTION (§1.4 of thesis)

#

```
def compute_objective(routes: list[VehicleRoute], A: float = OBJ_A, B: float =
OBJ_B) -> float:
```

```
    """
```

```
    Objective function value:
```

$$F = A \cdot \Sigma T_{\text{total}} + B \cdot K$$

```
    where T_total is the total travel time across all routes and K is the
    number of active vehicles.
```

```
    """
```

```
    T_total = sum(vr.total_time_s for vr in routes)
```

```
    K = len(routes)
```

```
    return A * T_total + B * K
```

#

PUBLIC API

#

```
def solve_vrp(
    stops: list,
    depot,
    fleet: list,
    graphs: dict,
    tsp_method: str = "auto",
) -> tuple[list, list]:
    """
    Solve a Capacitated VRP with Time Windows using a genetic algorithm.

    Parameters
    -----
    stops      : list[DeliveryStop]    – all delivery stops (node_id set)
    depot      : DeliveryStop          – start/end point (node_id set)
    fleet      : list[Vehicle]         – vehicles with mode and capacity
    graphs     : dict[str, MultiDiGraph] – {"drive": G, "bike": G, "walk": G}
    tsp_method : str                   – kept for API compatibility (unused by GA)

    Returns
    -----
    (routes, warnings)
    routes : list[VehicleRoute] – one entry per active vehicle
    warnings : list[str]       – user-facing warning messages
    """
```

Продовження додатку А

```

if not fleet:
    raise ValueError("Fleet is empty. Add at least one vehicle.")
if not stops:
    raise ValueError("No stops to route.")
if depot.node_id is None:
    raise ValueError("Depot node_id is None – snap depot to the OSM graph
before calling solve_vrp().")

warnings: list[str] = []

# — Phase 1: assign each stop to a vehicle-type pool

```

```

mode_stop_pool, unreachable = _assign_stops_to_modes(stops, fleet, depot,
graphs)

if unreachable:
    names = "; ".join(s.address[:35] for s in unreachable)
    warnings.append(
        f" ⚠ {len(unreachable)} зупинку(ок) недосяжно жодним режимом
флоту "
        f"(або вичерпана вантажомісткість) – пропущено: {names}"
    )
    logger.warning(
        "%d stop(s) could not be assigned to any vehicle: %s",
        len(unreachable), [s.address[:40] for s in unreachable],
    )

```

Продовження додатку А

```

# — Phase 2: GA per mode

```

```

mode_to_vehicles: dict[str, list] = {}
for v in fleet:
    mode_to_vehicles.setdefault(v.mode, []).append(v)

routes: list[VehicleRoute] = []

for mode, vehicles in mode_to_vehicles.items():
    pool = mode_stop_pool.get(mode, [])

    if not pool:
        for v in vehicles:
            warnings.append(f" i {v.name} не має призначених зупинок
(простоює).")
            logger.info("Vehicle %r (%s) is idle – no compatible stops.", v.name,
mode)
            continue

    mode_routes = _build_ga_routes(pool, vehicles, depot, graphs)
    routes.extend(mode_routes)

active_names = {vr.vehicle.name for vr in mode_routes}
for v in vehicles:
    if v.name not in active_names:
        warnings.append(f" i {v.name} не має призначених зупинок
(простоює).")

```

Продовження додатку А

```
if not routes:
    raise RuntimeError(
        "Жоден транспорт не вдалося маршрутизувати. Перевірте, що режими
ТЗ сумісні "
        "з дорожньою мережею області доставки, та що вантажомісткість
флоту достатня."
    )

obj_val = compute_objective(routes)
logger.info(
    "GA solution: %d active vehicle(s), objective F = %.1f",
    len(routes), obj_val,
)

return routes, warnings
```

ДОДАТОК Б

